FEMaster User Manual

Finite Elements in Mechanical Analysis and Simulation for Technical Engineering Research

Version 2.9.67

17 October 2026

Contents

1 Overview **1**

1.1 Introduction . . . 2
1.2 Design philosophy and implementation . . . 3
1.3 Input format and compatibility . . . 4
1.4 Supported element families . . . 5
1.5 Supported analysis families . . . 6
1.6 Degree-of-freedom convention . . . 8
1.7 Units . . . 9
1.8 Manual organization . . . 10

2 Getting Started **11**

2.1 Installation . . . 12
2.2 Building FEMaster . . . 13
2.2.1 Available build presets . . . 14
2.3 Running FEMaster . . . 15
2.4 First example: linear static truss . . . 16
2.5 Result files . . . 17
2.6 Built-in documentation . . . 18

3 FEMaster Architecture **19**

3.1 The finite-element model . . . 20
3.2 Model organization . . . 21
3.3 Models and analyses . . . 22
3.4 Typical workflow . . . 23

4 Keyword Reference **24**

4.1 Input Structure . . . 25
4.1.1 Command lines . . . 26
4.1.2 Data lines and multiline records . . . 27
4.1.3 Scopes . . . 28
4.1.4 Names, identifiers, and references . . . 29
4.1.5 FEMaster and Abaqus examples in this manual . . . 30

4.1.6 *HEADING . . . 31
4.2 Model Definition . . . 32
4.2.1 *PART . . . 33
4.2.2 *END PART . . . 34
4.2.3 *ASSEMBLY . . . 35
4.2.4 *END ASSEMBLY . . . 36
4.2.5 *INSTANCE . . . 37
4.2.6 *END INSTANCE . . . 38
4.2.7 *NODE . . . 39
4.2.8 *ELEMENT . . . 40
4.2.8.1 Supported element types . . . 41
4.2.8.2 Solid elements . . . 42
C3D4: 4-node tetrahedral solid . . . 43
C3D5: 5-node pyramid input . . . 44
C3D6: 6-node wedge solid . . . 45
C3D8: 8-node hexahedral solid . . . 46
C3D8I: incompatible-mode 8-node hexahedral solid . . . 47
C3D8R: reduced-integration 8-node hexahedral solid . . . 48
C3D10: 10-node quadratic tetrahedral solid . . . 49
C3D13: 13-node quadratic pyramid solid . . . 50
C3D15: 15-node quadratic wedge solid . . . 51
C3D20: 20-node quadratic hexahedral solid . . . 52
C3D20R: 20-node reduced-integration hexahedral solid . . . 53
4.2.8.3 Shell elements . . . 54
MITC3FRT / S3 / S3R: 3-node finite-rotation triangular shell . . . 55
MITC4FRT / S4 / S4R / MITC4: 4-node finite-rotation quadrilateral shell . . . 56
MITC6FRT / S6 / S6R: 6-node finite-rotation triangular shell . . . 57
MITC8FRT / S8 / S8R / MITC8: 8-node finite-rotation quadrilateral shell . . . 58
QSPT: quadrilateral shear-panel element . . . 59
4.2.8.4 Beam and truss elements . . . 60
B31: 2-node geometrically exact 3D beam . . . 61
B33: 3D beam element . . . 62
T3: 2-node truss element . . . 63
4.2.8.5 Point elements . . . 64
MASS: 1-node concentrated-mass element . . . 65
ROTARYI: 1-node rotary-inertia element . . . 66
SPRING1: 1-node point-to-ground spring . . . 67
4.2.9 *NSET . . . 68
4.2.10 *ELSET . . . 71
4.2.11 *SURFACE . . . 72

4.2.12 *SFSET . . . 74
4.3 Model Properties . . . 76
4.3.1 Materials . . . 77
4.3.1.1 *MATERIAL . . . 78
4.3.1.2 *ELASTIC . . . 79
4.3.1.3 *PLASTIC . . . 81
4.3.1.4 *HYPERELASTIC . . . 82
4.3.1.5 *DENSITY . . . 83
4.3.1.6 *EXPANSION . . . 84
4.3.1.7 *THERMALEXPANSION . . . 86
4.3.2 Sections and Element Properties . . . 87
4.3.2.1 *PROFILE . . . 88
4.3.2.2 *SOLID SECTION . . . 89
4.3.2.3 *SHELL SECTION . . . 90
4.3.2.4 *BEAM SECTION . . . 92
4.3.2.5 *TRUSS SECTION . . . 94
4.3.2.6 *POINTMASS . . . 95
4.3.2.7 *MASS . . . 97
4.3.2.8 *ROTARY INERTIA . . . 98
4.3.2.9 *SPRING . . . 99
4.3.3 Coordinate Systems and Orientations . . . 100
4.3.3.1 *ORIENTATION . . . 101
4.3.3.2 *TRANSFORM . . . 103
4.3.4 Fields . . . 104
4.3.4.1 *FIELD . . . 105
4.3.4.2 *INITIALCONDITION . . . 107
4.3.4.3 *NORMAL . . . 108
4.4 Loads and Boundary Conditions . . . 109
4.4.1 *AMPLITUDE . . . 110
4.4.2 *SUPPORT . . . 112
4.4.3 *BOUNDARY . . . 114
4.4.4 *CLOAD . . . 115
4.4.5 *DLOAD . . . 116
4.4.6 *PLOAD . . . 118
4.4.7 *DSLOAD . . . 119
4.4.8 *VLOAD . . . 120
4.4.9 *INERTIALOAD . . . 121
4.4.10 *TEMPERATURE . . . 123
4.4.11 *HEATFLUX . . . 124
4.4.12 *CONVECTION . . . 125

4.5 Constraints and Contact . . . 126
4.5.1 *COUPLING . . . 127
4.5.2 *KINEMATIC . . . 129
4.5.3 *DISTRIBUTING . . . 130
4.5.4 *EQUATION . . . 131
4.5.5 *MPC . . . 132
4.5.6 *RIGID BODY . . . 133
4.5.7 *TIE . . . 134
4.5.8 *CONNECTOR . . . 135
4.5.9 *RBM . . . 136
4.5.10 *CONTACT . . . 137
4.6 Steps and Loadcases . . . 139
4.6.1 Analysis Procedures . . . 140
4.6.1.1 *LOADCASE . . . 141
4.6.1.2 *SUPPORTS . . . 143
4.6.1.3 *LOADS . . . 144
4.6.1.4 *END . . . 145
4.6.1.5 *STEP . . . 146
4.6.1.6 *STATIC . . . 148
4.6.1.7 *FREQUENCY . . . 150
4.6.1.8 *BUCKLE . . . 151
4.6.1.9 *DYNAMIC . . . 153
4.6.1.10 *STEADY STATE DYNAMICS . . . 155
4.6.1.11 *END STEP . . . 157
4.6.2 Solver and Numerical Controls . . . 158
4.6.2.1 *SOLVER . . . 159
4.6.2.2 *CONSTRAINTMETHOD . . . 161
4.6.2.3 *NONLINEAR . . . 163
4.6.2.4 *TIME . . . 166
4.6.2.5 *NEWMARK . . . 167
4.6.2.6 *DAMPING . . . 168
4.6.2.7 *FREQUENCIES . . . 169
4.6.2.8 *NUMEIGENVALUES . . . 171
4.6.2.9 *SIGMA . . . 172
4.6.2.10 *WRITEEVERY . . . 173
4.6.2.11 *INITIALVELOCITY . . . 174
4.6.2.12 *INERTIARELIEF . . . 175
4.6.2.13 *REBALANCELOADS . . . 176
4.6.3 Topology-weighted Linear Statics . . . 177
4.6.3.1 *TOPODENSITY . . . 178

4.6.3.2 *TOPOORIENT . . . 179
4.6.3.3 *TOPOEXPONENT . . . 180
4.7 Output and Diagnostics . . . 181
4.7.1 *OVERVIEW . . . 182
4.7.2 *REQUESTSTIFFNESS . . . 183
4.7.3 *REQUESTSTGEOM . . . 184
4.7.4 *CONSTRAINTSUMMARY . . . 185
4.7.5 Abaqus and CalculiX output-request keywords . . . 186
4.7.6 *NODE OUTPUT . . . 187
4.7.7 *ELEMENT OUTPUT . . . 188
4.7.8 *NODE FILE . . . 189
4.7.9 *EL FILE . . . 190
4.7.10 *EL OUTPUT . . . 191
4.7.11 *NODE PRINT . . . 192
4.7.12 *EL PRINT . . . 193
4.7.13 Native .res files . . . 194
4.7.14 FEMR binary result format . . . 199
4.7.14.1 Chunk envelope . . . 200
4.7.14.2 Chunks . . . 201
4.7.14.3 Stable mesh and field indexing . . . 202
4.7.14.4 Lazy access . . . 203
4.7.15 FRD visualization output . . . 204
4.8 Keyword Index . . . 205
4.8.1 Supported element formulations . . . 208
4.8.2 Coverage notes . . . 209

5 Examples **210**
5.1 Linear static truss . . . 211
5.2 Nonlinear axial response with tabulated J2 plasticity . . . 212
5.3 Reusable Part and translated Instance . . . 214
5.4 Homogeneous shell cantilever . . . 216
5.5 Solid cube under pressure . . . 218
5.6 Eigenfrequency analysis . . . 220
5.7 Eigenfrequency with a concentrated MASS element . . . 222
5.8 Linear buckling of a compressed member . . . 224
5.9 Arc-length snap-through . . . 226
5.10 Linear transient ramp load . . . 228
5.11 Linear harmonic frequency sweep . . . 230
5.12 Using the examples as templates . . . 232

A Mathematical Derivations **233**

A.1 Element stiffness 234
A.2 Compliance 235
A.3 Density penalization 236
A.4 Orientation-dependent stiffness 237
A.5 Voigt transformation 238
A.6 Orientation sensitivity 239
A.7 Rotation derivatives 240

1 Overview

1.1 Introduction

FEMaster (*Finite Elements in Mechanical Analysis and Simulation for Technical Engineering Research*) is an open-source finite-element solver for structural mechanics, implemented in modern C++. It is designed for engineering applications, scientific research, and education, providing a common framework for the numerical analysis of structures ranging from simple mechanical components to large and complex finite-element models. Its capabilities include different structural element formulations, static and dynamic analyses, eigenvalue problems, and nonlinear structural behavior.

The project originated during the author's university studies, following an internship at MTU where working with CalculiX provided the motivation to develop an independent finite-element solver. Rather than simply reproducing existing functionality, the objective was to implement established numerical methods within a modern, maintainable, and accessible codebase. FEMaster is intended to serve both as a practical engineering tool and as a platform for studying, developing, and verifying finite-element methods. Particular emphasis is placed on numerical transparency, computational performance, and a broad selection of element formulations, allowing users to investigate both the physical response of a structure and the numerical methods used to obtain it.

1.2 Design philosophy and implementation

The development of FEMaster is guided by three closely related objectives: numerical transparency, maintainability, and computational efficiency. Numerical transparency means that the methods used by the solver should be accessible and understandable, rather than hidden behind an opaque implementation. The source code is organized around clearly defined mechanical and numerical concepts, with an emphasis on readable implementations, meaningful interfaces, and documentation. Mathematical formulations and numerical procedures are described throughout this manual where appropriate, providing a connection between the theoretical foundations of finite-element analysis and their implementation in the solver. This is particularly relevant for educational applications, method development, and the verification of numerical results.

Maintainability is supported by a modular C++ architecture that separates model definitions, element formulations, material behavior, analysis procedures, and numerical solution methods. This separation allows individual components to be developed and examined without requiring detailed knowledge of the entire application. It also facilitates the implementation of additional element formulations, constitutive models, and analysis capabilities. The intention is to maintain a codebase that remains approachable as the solver grows in complexity, enabling contributions and modifications without sacrificing the clarity of existing implementations.

Computational efficiency is addressed through parallel execution, efficient numerical algorithms, and established high-performance libraries. FEMaster uses Eigen for linear algebra operations and Spectra for eigenvalue computations, while Intel oneMKL provides high-performance multithreaded CPU solution capabilities. GPU acceleration is supported through NVIDIA CUDA and cuDSS, allowing computationally demanding solution tasks to be executed on compatible graphics hardware. The available numerical backends depend on the platform and build configuration. These technologies are intended to provide efficient solutions for both small models, where computational overhead can be significant relative to the actual problem size, and large systems, where memory requirements and the performance of sparse numerical operations become increasingly important.

1.3 Input format and compatibility

FEMaster uses a unified keyword-based input format derived from the conventions established by Abaqus and also used by CalculiX. A finite-element model is described through a text file containing definitions for geometry, element connectivity, materials, structural properties, loads, boundary conditions, and analysis procedures. The format is particularly suitable for automated model generation, parameter studies, and reproducible numerical investigations, while remaining sufficiently readable for small models to be created and modified manually.

The input language supports a subset of the Abaqus keyword syntax together with FEMaster-specific extensions. Both forms are interpreted within the same input language and can generally be combined in a single input deck. This allows existing Abaqus-style models to be used where their definitions are supported, while additional FEMaster keywords provide access to functionality beyond the conventional syntax. Full compatibility with arbitrary Abaqus input files is not implied, since the supported keywords, parameters, element formulations, and numerical procedures depend on the capabilities implemented in FEMaster. The Keyword Reference documents these capabilities and presents alternative Abaqus-compatible and FEMaster-specific syntax wherever applicable.

Several extensions introduce concepts inspired by preprocessing environments such as Altair HyperWorks. In particular, loads and supports can be organized into named collectors that are defined independently of individual analyses. These collectors can subsequently be referenced by analysis steps, allowing different combinations of loads and boundary conditions to be applied without repeatedly defining the same conditions. Additional keywords provide explicit control over analysis procedures, constraint handling, solver selection, and the use of CPU or GPU acceleration. Together, these extensions are intended to make input decks easier to organize, reuse, and inspect, particularly when a model contains several loading configurations or analysis procedures.

1.4 Supported element families

FEMaster provides a range of structural finite elements for modelling one-, two-, and three-dimensional mechanical behavior. The supported families include truss, beam, shell, continuum solid, and point elements, each offering different capabilities and levels of geometric detail. The choice of element formulation determines which deformation modes can be represented, which physical properties are required, and how accurately a particular structural response can be captured.

An important characteristic of FEMaster is its support for six mechanical degrees of freedom per node, consisting of three translations and three rotations. Individual elements activate only the degrees of freedom required by their formulation. In particular, beam and shell elements use independent rotational degrees of freedom and are implemented as dedicated structural elements. Unlike the approach used for many conventional beam and shell formulations in CalculiX, these elements are not internally expanded into three-dimensional continuum elements. Instead, their stiffness, internal forces, and other mechanical quantities are evaluated directly from their respective structural formulations.

Continuum solid elements represent the full three-dimensional deformation and stress state of a body. FEMaster supports tetrahedral, hexahedral, wedge, and pyramid elements with different interpolation orders and integration schemes, including reduced integration and enhanced formulations. They are particularly suitable for complex geometries, local stress concentrations, and regions where three-dimensional effects are important.

Beam and truss elements provide computationally efficient representations of slender structures. Truss elements carry axial forces, while beam elements additionally represent bending, torsion, and transverse shear, depending on the formulation. Their cross-sectional behavior is defined through geometric and material properties rather than an explicitly meshed cross-section. FEMaster supports both classical beam formulations and geometrically exact formulations capable of representing finite rotations, making them suitable for frames, stiffeners, spars, and other structural members.

Shell elements describe thin-walled structures through a two-dimensional reference surface and associated thickness properties. They capture membrane deformation, bending, and transverse shear without requiring a volume mesh through the thickness. FEMaster includes shell formulations based on the Mixed Interpolation of Tensorial Components (MITC) approach with finite-rotation kinematics, making them suitable for plates, panels, curved shells, and lightweight structures.

Point elements introduce concentrated properties such as masses, rotary inertias, and spring stiffnesses at individual nodes. They allow attached equipment, payloads, and simplified elastic supports to be represented without explicitly modelling their geometry.

The different element families can be combined within a single finite-element model, allowing an appropriate balance between computational efficiency and mechanical detail. The available element types, their formulations, numerical characteristics, and limitations are documented in Section 4.2.8.1.

1.5 Supported analysis families

FEMaster supports several analysis procedures covering static structural behavior, stability, free vibration, and time- or frequency-dependent response. An analysis procedure determines the governing equations to be solved, the required system matrices, and the numerical strategy used to obtain the solution. The same finite-element model can be evaluated using different procedures, allowing structural stiffness, deformation, stability, and dynamic characteristics to be investigated without redefining the underlying geometry and material properties. The currently supported analysis families are summarized below.

FEMaster analysis	**Description**
`LINEARSTATIC`	Linear static equilibrium analysis under prescribed loads and boundary conditions. The procedure determines displacements, reactions, strains, and stresses using a linearized structural model. It is suitable for stiffness evaluation, load-path investigations, and structural assessments in which nonlinear effects can be neglected.
`LINEARSTATICTOPO`	Linear static analysis with topology-dependent element properties. Element-wise density values, penalization, and optional orientation Fields allow structural responses to be evaluated for different material distributions, providing a basis for topology-related investigations and optimization workflows.
`NONLINEARSTATIC`	Incremental static equilibrium analysis accounting for nonlinear structural behavior. The solution is obtained through iterative equilibrium corrections, with support for load-controlled increments and arc-length path following. Depending on the element formulations and model definitions, nonlinear effects may include large displacements, finite rotations, material nonlinearities, and contact.
`EIGENFREQ`	Free-vibration analysis based on the generalized eigenvalue problem of the structural stiffness and mass matrices. The procedure computes natural frequencies and their associated mode shapes, allowing the dynamic characteristics of a structure to be investigated independently of a prescribed time-dependent excitation.
`LINEARBUCKLING`	Linearized stability analysis using the structural stiffness and the geometric stiffness associated with a reference loading state. The resulting eigenvalues describe critical load factors within the linearized buckling approximation, while the corresponding eigenvectors identify possible instability modes.
`LINEARTRANSIENT`	Linear time-domain dynamic analysis including inertial and optional damping effects. The equations of motion are integrated using an implicit Newmark scheme, allowing the response to time-dependent loading, initial velocities, and transient excitation to be evaluated over a prescribed time interval.
`LINEARHARMONIC`	Linear steady-state response to harmonic excitation. The procedure solves the frequency-dependent complex equation system directly at specified excitation frequencies, providing displacement and other supported response amplitudes together with their phase information. Optional damping can be included through Rayleigh damping.

The analysis procedures differ not only in their numerical solution methods but also in the physical assumptions underlying their results. Linear static analysis and linearized eigenvalue procedures are generally efficient and suitable for a wide range of engineering investigations, provided their assumptions are appropriate for the problem. Nonlinear static analysis extends the equilibrium formulation to situations in which the structural response changes with deformation, loading, or contact conditions, but typically requires incremental loading and iterative convergence. Dynamic procedures additionally account for mass

and inertia, either by integrating the response over time or by evaluating the steady-state response at selected excitation frequencies. These distinctions should be considered when selecting a procedure, since the most computationally demanding analysis is not necessarily the most appropriate one for a particular engineering question.

Both FEMaster-specific analysis definitions and supported Abaqus procedure keywords are available within the input language. The corresponding Abaqus procedures include `*STATIC`, `*FREQUENCY`, `*BUCKLE`, `*DYNAMIC`, and `*STEADY STATE DYNAMICS`. Their accepted parameters, available numerical controls, and relationship to the FEMaster analysis types are documented in Section 4.6.1.

1.6 Degree-of-freedom convention

FEMaster uses a generalized six-component nodal convention consisting of three translations and three rotations:

`Ux, Uy, Uz, Rx, Ry, Rz`.

The first three components represent translational displacements along the global coordinate axes, while the last three represent rotational degrees of freedom about those axes. This convention is used for generalized nodal loads, supports, coupling definitions, initial velocities, and selected result Fields. It does not imply that every node necessarily contributes six degrees of freedom to the global equation system. The active degrees of freedom depend on the connected element formulations and associated model definitions. For example, continuum solid elements generally require only translations, whereas beam and shell elements additionally use rotations. This distinction permits different structural element families to be represented within a common generalized framework without introducing unnecessary degrees of freedom throughout the model.

The six-component nodal convention must not be confused with the six-component engineering stress and strain convention, which represents the independent components of symmetric second-order tensors rather than generalized displacements. Stress and strain component ordering is documented in Section 4.7.

1.7 Units

FEMaster does not prescribe a particular system of units or perform automatic unit conversions. All geometric dimensions, material properties, loads, masses, and numerical parameters must therefore be expressed using a consistent system of units. A common choice for structural analysis is millimetres, newtons, tonnes, and seconds, resulting in stresses and elastic moduli expressed in megapascals. Other systems, such as metres, newtons, kilograms, and seconds, are equally valid when applied consistently. Particular attention is required for density, rotary inertia, dynamic loading, damping coefficients, thermal expansion, and stiffness-related parameters, since inconsistent units may produce numerical results that appear plausible despite having no meaningful physical interpretation. Responsibility for unit consistency remains with the user, and the solver interprets all supplied quantities according to the governing finite-element equations.

1.8 Manual organization

This manual begins with Chapter 2, which introduces installation, command-line execution, and a first complete calculation, including the interpretation and verification of its results. Chapter 3 then explains the fundamental concepts of the FEMaster model architecture and the relationships between its principal objects. Building on these concepts, Chapter 4 provides the detailed input language reference, including supported keywords, parameters, data records, and alternative Abaqus-compatible definitions. The individual modelling and analysis concepts are brought together in Chapter 5, which presents complete applications and demonstrates how the available functionality can be used to investigate engineering problems. Additional mathematical background, numerical formulations, and derivations are provided in the appendices.

2 Getting Started

This chapter explains how to obtain FEMaster, compile it from source, and perform a first finite-element analysis. FEMaster is a command-line application that reads a keyword-based input file, performs the requested calculations, and writes the resulting data to output files. No graphical interface is required, allowing the solver to be used both interactively and as part of automated simulation workflows.

2.1 Installation

FEMaster is available for Linux, Windows, and macOS. The source code, precompiled releases, and additional project information are hosted on GitHub at https://github.com/Luecx/FEMaster.

The simplest way to get started is to download a precompiled version from https://github.com/Luecx/FEMaster/releases. Where available, release archives contain executables for the corresponding operating system and architecture. After extracting the archive, the executable can be launched directly from a terminal. Depending on the selected release, additional runtime libraries may be required.

Alternatively, FEMaster can be compiled from source. This allows the solver to be configured for the available hardware, including optional multithreaded CPU and GPU acceleration, and provides access to the latest development version. The source code can be obtained through Git:

```
git clone https://github.com/Luecx/FEMaster.git
cd FEMaster
```

2.2 Building FEMaster

FEMaster uses CMake to manage its compilation and external dependencies. The repository provides predefined build configurations, called *presets*, for the supported operating systems and numerical backends. A preset selects the compiler configuration, optimization settings, and optional libraries required for a particular build.

From the repository root, configuring and compiling FEMaster requires only two commands. For example, a basic Linux build can be created using:

```
cmake --preset linux
cmake --build --preset linux -j8
```

The name `linux` is the selected preset and can be replaced by any of the available presets listed below. The first command configures the project and prepares its dependencies, while the second compiles the executable. The optional argument `-j8` specifies eight parallel build jobs. Subsequent builds generally require only the second command, unless the build configuration changes.

Compilation requires CMake 3.24 or newer, Git, and a compiler supporting C++17. The presets using Ninja additionally require Ninja to be installed. Standard dependencies, including Eigen, Spectra, argparse, and LZ4, are downloaded automatically by CMake during configuration. Optional numerical libraries such as Intel oneMKL, NVIDIA CUDA, and cuDSS must be installed separately when their corresponding backends are enabled. On macOS, Apple Accelerate is provided by the system SDK.

2.2.1 Available build presets

The predefined presets provide different combinations of CPU parallelization, sparse solvers, and GPU acceleration. All regular presets produce optimized Release builds, with the exception of the profiling configuration. The available presets are summarized below.

Preset	Configuration
Linux	
`linux`	Standard CPU build.
`linux-omp`	CPU build with OpenMP parallelization.
`linux-profile`	CPU build with profiling and debugging symbols.
`linux-mkl`	CPU build with sequential Intel oneMKL.
`linux-mkl-omp`	Threaded Intel oneMKL with OpenMP.
`linux-cuda`	NVIDIA CUDA acceleration.
`linux-cuda-cudss`	CUDA with NVIDIA cuDSS sparse direct solving.
`linux-cuda-mkl`	CUDA with sequential Intel oneMKL.
`linux-cuda-mkl-omp`	CUDA with threaded Intel oneMKL and OpenMP.
`linux-full`	CUDA, cuDSS, threaded Intel oneMKL, and OpenMP.
Windows	
`windows-msvc`	Standard CPU build using MSVC.
`windows-msvc-cuda`	MSVC build with CUDA acceleration.
`windows-msvc-cuda-cudss`	MSVC build with CUDA and cuDSS.
`windows-intel`	Standard CPU build using IntelLLVM.
`windows-intel-mkl`	IntelLLVM with sequential Intel oneMKL.
`windows-intel-mkl-omp`	IntelLLVM with threaded Intel oneMKL and OpenMP.
macOS	
`macos`	Standard CPU build.
`macos-omp`	CPU build with OpenMP parallelization.
`macos-accelerate`	CPU build with Apple Accelerate.
`macos-omp-accelerate`	OpenMP with Apple Accelerate.

The available configurations depend on the installed toolchains and hardware. Windows MSVC presets use Visual Studio, while the IntelLLVM presets require the Intel compiler toolchain. CUDA configurations require a compatible NVIDIA GPU and CUDA installation, and cuDSS configurations additionally require the cuDSS library. Apple Accelerate is available only on macOS.

CMake places the compiled executable in the build directory associated with the selected preset, typically `build/<preset>/bin`. Multi-configuration generators, such as the Visual Studio presets, place the Release executable in `build/<preset>/bin/Release`. The executable is named `FEMaster` on Linux and macOS and `FEMaster.exe` on Windows.

The exact preset definitions and additional configuration options can be found in `CMakePresets.json` and `CMakeLists.txt`.

2.3 Running FEMaster

FEMaster is executed by passing the path of an input deck to the command-line application. Input files conventionally use the `.inp` extension and contain the model definition together with the requested analysis procedures. Both supported Abaqus keywords and FEMaster-specific extensions are interpreted by the same parser, so no explicit input-format selection is required.

If the executable is available on the system path, a model can be executed using:

```
FEMaster example.inp
```

Alternatively, the executable can be called using its location in the build directory. For example, from the repository root after compiling with the `linux` preset:

```
./build/linux/bin/FEMaster example.inp
```

Several optional command-line arguments control execution and result storage. The most commonly used options are:

Argument	Description
`-ncpus N`	Set the number of CPU threads available to the solver.
`-output FILE`	Specify an alternative output file base name or path.
`-output-format`	Select one or more result formats: `res`, `frd`, or `femr`.

For example, the following command limits execution to eight CPU threads and requests FRD and FEMR output:

```
FEMaster example.inp --ncpus 8 --output-format frd femr
```

The selected numerical backend is configured through the input deck and must be supported by the compiled executable. The complete command-line interface can be inspected using `-help`.

2.4 First example: linear static truss

As a first calculation, consider a straight truss element subjected to an axial tensile force. The model consists of two nodes connected by a single element with a length of 1000 mm and a cross-sectional area of 100 mm^2. The material is linear elastic steel with Young's modulus of 210000 MPa. One end is fixed, while the opposite end is allowed to move only in the axial direction and is subjected to a force of 1000 N.

Save the following input deck as `example.inp`:

Syntax

```
*HEADING
First FEMaster example

*NODE
1, 0., 0., 0.
2, 1000., 0., 0.

*ELEMENT, TYPE=T3, ELSET=BAR
1, 1, 2

*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3

*TRUSS SECTION, ELSET=BAR, MATERIAL=STEEL
100.

*STEP, NAME=AXIAL_TENSION
*STATIC
*BOUNDARY
1, 1, 3, 0.
2, 2, 3, 0.
*CLOAD
2, 1, 1000.
*END STEP
```

The example combines FEMaster-specific element and section definitions with Abaqus-style analysis keywords. Both are supported within the unified input language. To perform the analysis, execute:

```
FEMaster example.inp
```

FEMaster reads the model, assembles the finite-element system, solves the static equilibrium equations, and writes the calculated results to output files. The expected axial displacement at the loaded node is approximately 0.047619 mm, providing a simple reference value for checking the calculation.

2.5 Result files

By default, FEMaster writes the results of an analysis using the input filename as the output base name. Running `example.inp` therefore produces `example.res` and `example.frd`. An additional binary result format, `.femr`, is available through the `-output-format` option.

The `.res` format stores numerical result Fields in a text-based representation, while `.frd` provides results in the CalculiX/CGX format for visualization and postprocessing. The `.femr` format provides structured binary storage with compression for more efficient handling of larger result datasets. The available result quantities, output controls, and file conventions are described in Section 4.7.

2.6 Built-in documentation

In addition to this manual, FEMaster provides a command-line documentation mode for inspecting the keywords and input variants recognized by the executable. This can be useful when working with an unfamiliar command or checking which syntax forms are supported by a particular build.

Documentation mode is activated using `-document`, followed by an action specifying the requested information:

```
FEMaster --document --doc-list
FEMaster --document --doc-show CLOAD
FEMaster --document --doc-variants ELASTIC
FEMaster --document --doc-search SECTION
```

These commands list available keywords, display the definition of a particular command, inspect its accepted data variants, or search the command registry. Documentation mode does not require an input deck and does not perform a finite-element analysis. For detailed explanations, modelling conventions, and practical examples, the Keyword Reference remains the primary source of information.

3 FEMaster Architecture

This chapter explains the finite-element objects used by FEMaster and how they form a model and its analyses. The exact input syntax is documented in Chapter 4.

3.1 The finite-element model

A structural finite-element model consists of several layers of information that work together.

Nodes define the reference geometry and carry the mechanical degrees of freedom. **Elements** connect those nodes and define the interpolation and kinematic assumptions used over lines, surfaces, or volumes. The element formulation determines which deformation modes can be represented and how strains, curvatures, internal forces, mass, and other element quantities are evaluated.

Node sets, element sets, and surfaces give reusable names to parts of the mesh. They are used to assign Sections, supports, loads, couplings, ties, contact pairs, and other model features without repeatedly listing individual IDs.

Materials define constitutive behaviour and density. **Sections** connect element regions to their physical properties. A solid Section assigns a material; a shell Section additionally defines thickness or generalized shell stiffness; a truss Section defines area; and a beam Section combines a material with a profile and local cross-section direction.

Coordinate systems and orientations define local directions for anisotropic material behaviour, beam and shell properties, loads, supports, nodal transformations, and other directional quantities.

Fields store quantities that vary spatially over nodes, elements, element-local nodes, integration points, or material points. They are used when a model feature requires more information than can be expressed by one scalar keyword parameter, for example topology variables or prescribed shell directors.

Loads and boundary conditions define the external forcing and prescribed motion. **Constraints** introduce relations between different degrees of freedom, such as couplings, ties, idealized connectors, rigid-body-mode suppression, and unilateral contact.

Finally, an **analysis procedure** determines which equilibrium or eigenvalue problem is solved. Numerical controls select the solution strategy, constraint treatment, time or load stepping, convergence criteria, damping, and related settings.

3.2 Model organization

Model definition describes the geometry and topology on which the analysis is built. A simple FEMaster model may be completely flat: nodes and elements can be defined directly without an explicit Part, Assembly, or Instance. The optional structured Part/Assembly form represents reused components, several placements of the same Part, separate local ID spaces, and imported Abaqus-style assemblies.

From the user's point of view, the two forms describe the same kind of finite-element model. In a flat model there is one direct topology. In a structured model, each Part owns its local nodes, elements, sets, surfaces, and section assignments, while Instances place those Parts into the final assembly. Parts therefore describe *what a component is*; Instances describe *where that component is*.

3.3 Models and analyses

Mesh topology, regions, materials, Sections, orientations, and model Fields define the physical model. Native load and support collectors provide reusable loading and support definitions. A `*LOADCASE` selects the collectors and analysis procedure to use with that model; its numerical controls determine how the resulting problem is solved.

Supported Abaqus input expresses an analysis through `*STEP` and its procedure, load, and boundary-condition keywords. The native and Abaqus forms are documented together in Section 4.6.

3.4 Typical workflow

A practical FEMaster model is usually built in the following order:

1. define the geometry and mesh using nodes, elements, and optionally Parts and Instances;
2. create named regions and surfaces that will be used by later definitions;
3. define materials, profiles, Sections, and coordinate systems;
4. define Fields or other spatial model data where needed;
5. apply loads, supports, and kinematic constraints;
6. define the analysis procedure and its numerical controls;
7. resulting displacement, force, stress, strain, mode-shape, buckling, transient, or harmonic Fields complete the analysis output.

The same order exposes dependencies during diagnosis: geometry and element connectivity precede region definitions, material and Section assignments, coordinate directions, supports and constraints, and finally the numerical analysis settings.

4 Keyword Reference

This chapter documents the unified FEMaster input language: supported Abaqus-compatible definitions, CalculiX-compatible aliases, and FEMaster-specific extensions can be used together. Keywords are organized by modelling role. Section 4.8 provides a compact index.

Reading the keyword pages

Each keyword has one canonical entry, beginning with its modelling purpose and supported parameters or data lines. An example of that keyword follows. Where a genuinely useful alternative exists, it is shown directly below the primary example. Abaqus compatibility and FEMaster extensions are identified without presenting separate input languages. The `*ELEMENT` reference additionally documents the individual numerical formulations and their accepted type aliases.

4.1 Input Structure

FEMaster reads a unified keyword-based input language. A command begins with an asterisk, optional parameters follow after commas, and keyword-specific data are provided on subsequent lines. Supported Abaqus and CalculiX input forms coexist with FEMaster extensions in the same deck; selecting a particular file format or parser is not required.

Familiar Parts, Assemblies, Steps, loads, and boundary conditions can therefore be combined with FEMaster features such as named load and support collectors, explicit LOADCASE definitions, generic Fields, and solver controls. Each keyword page presents a preferred example and, where meaningful, additional supported alternatives below it rather than in parallel columns.

4.1.1 Command lines

A command line begins with `*`. The command name follows immediately and may be followed by comma-separated parameters. Parameters can be key/value pairs or flags. Data lines continue until the next command line begins.

Syntax

```
*COMMAND, KEY=value, FLAG
value, value, value
value, value, value
```

Command and parameter names are case insensitive. Whitespace inside command names is ignored, so compact names such as `*SOLIDSECTION` and spaced names such as `*SOLID SECTION` refer to the same command where that keyword is supported. The manual normally uses the spaced form when it improves readability and the compact form where it is customary in native FEMaster decks.

Parameter values retain their meaning. Numerical values are read as integer or floating-point quantities according to the keyword definition. Boolean-style options may appear as flags or as explicit values where documented. Names are used to reference Parts, Instances, sets, surfaces, materials, sections, coordinate systems, Fields, amplitudes, collectors, and other model entities.

4.1.2 Data lines and multiline records

The number and meaning of data-line entries depend on the keyword. Some commands accept one record per line, such as `*NODE`. Others read a fixed-size group of values, a matrix, a list of identifiers, or multiple records. The keyword pages describe the required ordering explicitly.

Comma-separated values preserve the visibility of optional empty entries. Whitespace around values is ignored. Several native FEMaster commands use the literal `NAN` to mean that a generalized component is not prescribed. This is different from zero: zero is a physical numerical value, whereas an empty or NaN component may mean “leave this degree of freedom unspecified” for the corresponding keyword.

Long connectivity or data records may continue over more than one physical line where that command permits multiline input. This representation accommodates higher-order elements and large fixed-size material or Section definitions.

4.1.3 Scopes

Several keywords open scopes. A Part contains reusable mesh topology and part-level regions. An Assembly contains Instance placement and assembly-level region definitions. Material subcommands apply to the active material. Native FEMaster `*LOADCASE` opens an analysis definition. Abaqus `*STEP` opens an Abaqus analysis step.

Scopes are defined by keywords, not by indentation. Indentation may be used for readability but has no syntactic meaning. Closing commands such as `*END PART`, `*END ASSEMBLY`, `*END INSTANCE`, `*END STEP`, and native `*END` make the end of the corresponding scope explicit.

Parts, Assemblies, and Instances are optional for simple native FEMaster models. Nodes and elements may be defined directly without constructing an explicit Assembly. The structured form represents reusable Parts, repeated Instances, independent local ID spaces, and assembly-level selections.

Optional native `*MODEL` marker.

Native FEMaster may optionally place `*MODEL` at root level as a descriptive marker. It does not open a scope, create a second model object, or select among several models; omitting it has no effect on the topology or analysis. `NAME` is optional and may be used to attach a descriptive model name.

Syntax

```
*MODEL, NAME=BRACKET_MODEL
```

This marker is part of the native input-file framing rather than a separate model-definition hierarchy level. Abaqus input does not require or use a corresponding wrapper.

4.1.4 Names, identifiers, and references

Node and element IDs are user-defined non-negative integer labels. They may start at **0**; they do not need to start at 1 and do not need to be contiguous. IDs only need to be unique within the topology scope in which they are defined. For example, `0`, `10`, and `1000` are valid node IDs.

Inside separate Parts, the same numerical ID may be reused because each Part has its own local topology. Instances preserve these local labels from the user's point of view. When an assembly-level command needs to identify an entity from a particular Instance, the documented Instance-qualified reference syntax is used.

Named resources are used throughout the rest of the deck. Node sets, element sets, surfaces, materials, profiles, orientations, Fields, amplitudes, collectors, and other definitions are referenced by name. Stable names expose the model structure without repeatedly addressing individual numerical IDs.

User IDs and *local positional indices* represent different concepts. User node and element IDs may be sparse and may include zero. By contrast, indices such as a Field's `local_node`, `local_ip`, or `local_mp` identify a position inside an element and are explicitly zero based where documented.

4.1.5 FEMaster and Abaqus examples in this manual

Each keyword page shows a primary definition with useful alternatives stacked directly below it. A given keyword may have Abaqus-compatible or FEMaster-specific data forms; their placement and limitations are defined by that keyword rather than by separate input dialects. For example, `*SOLVER` belongs inside a `*LOADCASE` or inside a `*STEP` after its procedure definition.

The comparison shows how the same model operation is expressed in either accepted input style without exposing parser implementation details.

4.1.6 *HEADING

*HEADING adds descriptive text to an input deck. It does not change the finite-element equations or the numerical result. The keyword keeps exported, archived, and regression decks self-describing and is accepted in both native FEMaster and Abaqus input.

A heading is metadata conventionally placed near the beginning of a file. Later model commands do not reference it by name, and it does not substitute for a Part name, material name, loadcase name, or other actual model identifier.

Native FEMaster also accepts the optional root-level *MODEL, NAME=... marker described above. *HEADING is free descriptive text, whereas *MODEL is a compact native file marker.

Syntax and semantics

Syntax

```
*HEADING
Descriptive model title
```

The title is metadata for the input deck. It can identify the model or study without changing its assembled equations.

FEMaster and Abaqus example

```
*HEADING
Cantilever verification model
```

4.2 Model Definition

A FEMaster model can be defined directly through nodes and elements or organized into reusable Parts and placed Instances. The optional Part/Assembly hierarchy supports multiple placements of the same component with independent local numbering; simple models can omit the hierarchy. Node and element IDs are non-negative user labels, including `0`, and need not be consecutive.

Named node sets, element sets, and surfaces provide reusable selections for Section assignments, loads, supports, and constraints. The following keywords define the mesh, its organization, and its associated regions.

4.2.1 *PART

*PART begins a reusable finite-element component definition. Nodes, elements, part-level sets, surfaces, and section assignments that follow belong to the active Part until *END PART closes it.

A Part is defined in its own local coordinate system. Its node coordinates describe the undeformed component geometry before any Instance placement. The Part itself has no global translation or rotation. If the same component is needed in several locations, define its topology once and create several *INSTANCE entries.

Part-local IDs are independent. Node 0 in Part LEFT and node 0 in Part RIGHT are different nodes even though they use the same numerical label. The same applies to element IDs. This is one of the main practical advantages of the Part representation: imported or reusable components do not need to be renumbered merely because another Part uses the same IDs.

Explicit Parts are optional in native FEMaster input. If nodes and elements are written directly at root level, they belong to the flat model topology and can be used without writing *PART, *ASSEMBLY, or *INSTANCE.

Part-level sets, surfaces, and Section assignments follow the Part when it is instantiated. Part-level definitions therefore describe properties intrinsic to every placed copy, whereas Assembly-level definitions can select or combine particular Instances.

Parameters

Parameter	Status	Meaning
NAME	required	Unique Part name. The name is referenced by *INSTANCE.

In this example, the truss connectivity refers to the node IDs local to the part. A later INSTANCE can reuse that topology at another placement.

FEMaster and Abaqus example

```
*PART, NAME=ROD
*NODE, NSET=PNODES
0,   0., 0., 0.
1, 100., 0., 0.
*ELEMENT, TYPE=T3D2, ELSET=BAR
0, 0, 1
*END PART
```

4.2.2 *END PART

*END PART closes the active Part scope. Definitions that follow are no longer part-local unless another *PART is opened. The command is distinct from native *END, which terminates a native loadcase.

Syntax and semantics

Syntax

```
*END PART
```

The closing keyword terminates the reusable part definition; model-level or assembly-level declarations follow outside its scope.

FEMaster and Abaqus example

```
*PART, NAME=PLATE
...
*END PART
```

4.2.3 *ASSEMBLY

*ASSEMBLY begins the optional Assembly scope. The Assembly is where placed Instances and regions that refer to the assembled model are defined. A model that does not need explicit Instances may omit this scope entirely.

The Assembly combines several Parts or repeated placements of one Part. Part-level topology remains expressed with local IDs, while Assembly-level sets and surfaces select entities from particular Instances. This preserves local numbering inside each component without forcing a single manually renumbered global mesh.

The Assembly organizes the placed model but is not required by the solver. A flat native model and a structured Assembly represent alternative organizational forms for the same finite-element topology.

NAME is optional and defaults to ASSEMBLY. The name is descriptive for the supported current workflow; Instances and assembly-level regions are the referenced objects inside the Assembly.

Parameters

Parameter	Status	Meaning
NAME	optional	Assembly name; default ASSEMBLY.

The assembly refers to a previously defined part and places one reusable instance of it into the model.

FEMaster and Abaqus example

```
*ASSEMBLY, NAME=ASSEMBLY
*INSTANCE, NAME=ROD_A, PART=ROD
0., 0., 0.
*END INSTANCE
*END ASSEMBLY
```

4.2.4 *END ASSEMBLY

*END ASSEMBLY closes the Assembly scope.

Syntax and semantics

Syntax

```
*END ASSEMBLY
```

The assembly is closed after its instances and assembly-level sets have been defined.

FEMaster and Abaqus example

```
*ASSEMBLY, NAME=ASSEMBLY
*INSTANCE, NAME=LEFT, PART=ROD
0., 0., 0.
*END INSTANCE
*END ASSEMBLY
```

4.2.5 *INSTANCE

*INSTANCE places an existing Part in the Assembly. Native FEMaster intentionally accepts the same placement representation used by the supported Abaqus reader. NAME gives the Instance a unique assembly name and PART selects the reusable Part.

An Instance may be placed by translation, rotation, both, or neither. A three-value data line gives a translation

$$\boldsymbol{t} = (t_x, t_y, t_z).$$

A seven-value line defines a rotation by two points on the rotation axis and an angle in degrees:

$$(x_1, y_1, z_1, x_2, y_2, z_2, \varphi).$$

The two axis points must be distinct. If both translation and rotation are supplied, the translation is applied first and the axis-angle rotation second. With no placement data the Part is instantiated in its original position.

Instance placement changes the position and orientation of the component in the assembled model; it does not require the Part coordinates or Part IDs to be rewritten. Several Instances may therefore reference the same Part with different placements.

The Part-local node and element labels remain associated with every Instance. Assembly-level sets use the Instance qualification to distinguish the same local ID in several placed copies.

Parameters

Parameter	Status	Meaning
NAME	required	Unique Instance name used for assembly-level references.
PART	required	Name of the Part to place.

Data lines

Entry	Meaning
tx, ty, tz	Optional translation vector.
x1,y1,z1,x2,y2,z2,angle	Optional rotation axis through two points and rotation angle in degrees.

The two data lines below first translate the part and then rotate the placed instance. The part itself retains its original local node coordinates.

FEMaster and Abaqus example

```
*INSTANCE, NAME=RIGHT, PART=BRACKET
100., 0., 0.
0., 0., 0., 0., 0., 1., 90.
*END INSTANCE
```

4.2.6 *END INSTANCE

*END INSTANCE closes the current Instance definition.

The keyword closes the instance placement scope. Subsequent assembly-level selections can refer to the placed instance by name.

FEMaster and Abaqus example

```
*INSTANCE, NAME=A, PART=BLOCK
20., 0., 0.
*END INSTANCE
```

4.2.7 *NODE

*NODE defines finite-element nodes by user ID and Cartesian reference coordinates. In a Part scope the nodes belong to that Part. In a flat native model they belong directly to the model topology.

Node IDs are non-negative integers. **ID 0 is valid.** IDs need not be contiguous and may be chosen to preserve meaningful numbering from a mesh generator or external model. They must only be unique within the topology scope in which they are defined.

The optional NSET parameter adds every node in the block to a node set. Native FEMaster defaults this set to NALL. Coordinates are Cartesian reference coordinates. Missing trailing coordinate components default to zero; explicit three-component coordinates expose the complete position in shared examples.

Node order has no mechanical meaning by itself, but the order in which node IDs appear in an element connectivity is crucial because it determines the element's local topology and orientation.

The coordinates define the undeformed geometry. Loads, prescribed displacements, initial velocities, and Instance placement do not change the meaning of the node's Part-local reference coordinates.

Parameters

Parameter	Status	Meaning
NSET	optional	Node set receiving all nodes in the block; native default NALL.

Data lines

Entry	Meaning
id	Non-negative user node ID; 0 is valid.
x, y, z	Cartesian reference coordinates.

These three records create independent user node IDs, even though the coordinates are regularly spaced. Node IDs need not be consecutive.

FEMaster and Abaqus example

```
*NODE, NSET=EDGE
0,  0., 0., 0.
10, 25., 0., 0.
20, 50., 0., 0.
```

4.2.8 *ELEMENT

*ELEMENT defines element topology and connectivity. Each data record starts with an element ID followed by the node IDs in formulation-specific order. The TYPE parameter selects the formulation, while ELSET optionally assigns the elements to a named set. Element IDs are non-negative user labels, may start at 0, and need not be consecutive.

Connectivity order matters: it defines local edges and faces, element orientation, shell normals, and the mapping between the reference element and its physical geometry. The connected nodes must belong to the same topology scope as the element.

Syntax

```
*ELEMENT, TYPE=<element-type>, ELSET=<set>
<element-id>, <node-1>, <node-2>, ...
```

Parameters

Parameter	Status	Meaning
TYPE	required	Element type or formulation identifier listed below.
ELSET	optional	Element set receiving the block; the general set EALL is used by default.

Data lines

Entry	Meaning
id	Non-negative element ID; 0 is valid.
n1, n2, ...	Node connectivity in formulation-specific order.

Element connectivity defines topology and kinematics, but additional physical properties are assigned by the applicable Section or concentrated-property keywords; see Section 4.3.2. The following element reference lists the supported types and their aliases, followed by the individual numerical formulations, connectivity, applications, and limitations.

The selected element type determines kinematics, interpolation, integration scheme, and recoverable mechanical quantities. The overview groups alternative input names of the same implementation; the detailed descriptions cover connectivity, applications, and limitations.

4.2.8.1 Supported element types

Each row describes one implemented formulation. Alternative accepted `TYPE` names are grouped in the same row; they select the same FEMaster implementation rather than distinct numerical elements.

Element type and aliases	Nodes	Formulation
`C3D4`	4	Linear tetrahedral solid.
`C3D5`	5	Five-node pyramid represented by a degenerated hexahedral topology.
`C3D6`	6	Linear wedge solid.
`C3D8`	8	Fully integrated linear hexahedral solid.
`C3D8I`	8	Incompatible-mode hexahedral solid with condensed enhanced modes.
`C3D8R`	8	Reduced-integration hexahedral solid with hourglass stabilization.
`C3D10`	10	Quadratic tetrahedral solid.
`C3D13`	13	Quadratic pyramid solid with rational interpolation.
`C3D15`	15	Quadratic wedge solid.
`C3D20`	20	Fully integrated quadratic hexahedral solid.
`C3D20R`	20	Reduced-integration quadratic hexahedral solid.
`MITC3FRT`, `S3`, `S3R`	3	Finite-rotation triangular shell.
`MITC4FRT`, `S4`, `S4R`, `MITC4`	4	Finite-rotation quadrilateral MITC shell.
`MITC6FRT`, `S6`, `S6R`	6	Quadratic finite-rotation triangular shell.
`MITC8FRT`, `S8`, `S8R`, `MITC8`	8	Quadratic finite-rotation quadrilateral MITC shell.
`B31`	2	Shear-flexible geometrically exact three-dimensional beam.
`B33`	2	Three-dimensional Euler–Bernoulli beam.
`T3`, `T3D2`	2	Three-dimensional axial truss.
`MASS`	1	Concentrated translational mass.
`ROTARYI`	1	Concentrated rotary inertia.
`SPRING1`	1	Point-to-ground spring in a generalized nodal direction.
`QSPT`	4	Specialized quadrilateral shear-panel element.

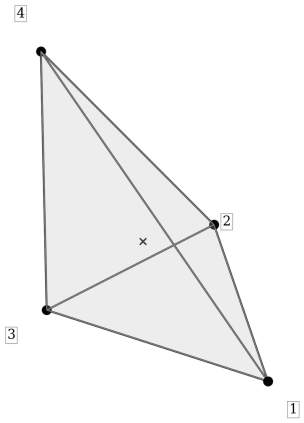
The table describes accepted input names, not numerical equivalence with the corresponding elements in other finite-element solvers. In particular, the `R` suffix of imported shell names such as `S4R` does not select a separate reduced-integration formulation in FEMaster.

4.2.8.2 Solid elements

Solid elements model a three-dimensional continuum. They use translational nodal degrees of freedom and obtain their constitutive response from a `*SOLID SECTION`. They represent the full three-dimensional stress state, through-thickness stresses, and local geometry or load introduction that shell and line elements omit.

Low-order solids are inexpensive but may require significant refinement in bending-dominated problems. Higher-order solids generally represent curvature and stress gradients more efficiently, but they contain more degrees of freedom per element and are more sensitive to poor midside-node placement and distorted mappings. Well-shaped hexahedra are useful for structured or swept meshes, while tetrahedra and pyramids accommodate irregular geometry and mixed-topology transitions. Reduced integration introduces additional sensitivity to low-energy modes and stabilization. Mesh quality and refinement should be assessed using displacements, reactions, energies, and stress trends; convergence of the algebraic solver alone does not establish discretization accuracy.

C3D4: 4-node tetrahedral solid



Element	C3D4
Type	solid
Nodes	4
Faces	4
Edges	6
DOF / node	3
Total DOF	12
Shape functions	linear tetrahedral Lagrange
Integration points	1

Figure 4.1: C3D4 element topology and integration scheme.

Syntax

```
*ELEMENT, TYPE=C3D4, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>
```

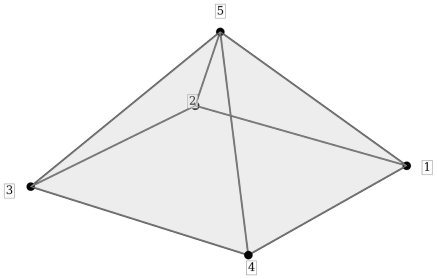
C3D4 is the simplest three-dimensional continuum element in FEMaster. It uses four corner nodes and linear displacement interpolation over a tetrahedral reference domain. The resulting strain field is constant within one element, which makes the element inexpensive and geometrically easy to mesh but limits the deformation and stress gradients that a single element can reproduce.

The principal advantage of C3D4 is meshing flexibility. Tetrahedra can fill complex CAD volumes, narrow transitions, and irregular local details with little manual topology design. This supports early stiffness checks, coarse load-path studies, and robust automatic meshing at the cost of stress accuracy per degree of freedom.

The limitation is stiffness and stress resolution. Linear tetrahedra tend to be overly stiff in bending-dominated problems unless the mesh is fine. Since the strain state is constant in each element, stress fields can appear piecewise constant or patchy and steep gradients near holes, fillets, notches, and concentrated loads require substantial refinement.

Compared with C3D4, C3D10 provides higher-order tetrahedral stress resolution in tetrahedral meshes. The sensitivity of C3D4 results to refinement appears in displacements, reactions, strain energy, and averaged stresses, while a single coarse local peak is not a convergence measure.

C3D5: 5-node pyramid input



Element	C3D5
Type	solid
Nodes	5
Faces	-
Edges	8
DOF / node	3
Total DOF	15
Shape functions	pyramid input mapped to C3D8
Integration points	-

Figure 4.2: C3D5 pyramid input topology.

Syntax

```
*ELEMENT, TYPE=C3D5, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <apex>
```

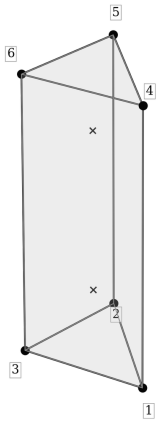
C3D5 provides a five-node pyramid connectivity for transition meshes. FEMaster represents this input with a degenerated eight-node hexahedral topology in which the four upper hexahedral positions coincide at the pyramid apex. The user nevertheless writes the natural five-node pyramid connectivity shown above.

The element primarily acts as a transition cell between hexahedral and tetrahedral regions. In automatically generated mixed meshes, it connects a quadrilateral face to tetrahedral topology without forcing a large surrounding region to change element family.

A degenerated mapping is generally less predictable than a regular hexahedron, wedge, or tetrahedron. Flat pyramids, badly skewed bases, and apex positions close to the base plane can produce poor Jacobians and unreliable stiffness. These properties limit C3D5 primarily to transition regions rather than large stress-critical volumes.

In models with many pyramids in the primary load path, reactions and global stiffness can remain plausible even when local stress quality differs significantly from an alternative regular-element mesh.

C3D6: 6-node wedge solid



Element	C3D6
Type	solid
Nodes	6
Faces	5
Edges	9
DOF / node	3
Total DOF	18
Shape functions	linear wedge / prism
Integration points	2

Figure 4.3: C3D6 wedge element.

Syntax

```
*ELEMENT, TYPE=C3D6, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>
```

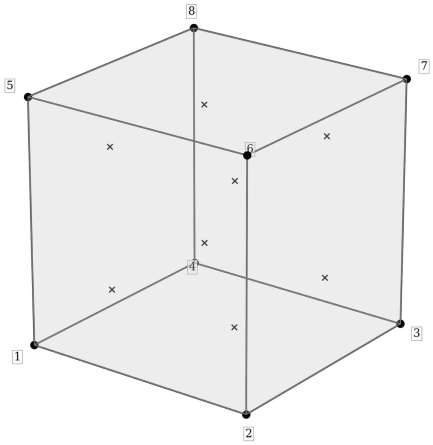
C3D6 is a linear wedge or prism element. It combines triangular end faces with quadrilateral side faces and represents swept meshes, extruded triangular surface meshes, layered regions, and transitions where a purely hexahedral topology is not possible.

A wedge mesh can align naturally with a dominant through-thickness or extrusion direction. In locally layered solids and prismatic regions, its element rows can follow the physical geometry more consistently than an unstructured tetrahedral mesh.

Like other low-order elements, C3D6 has limited bending and gradient resolution. One or two linear wedges through the thickness of a bending-dominated region provide limited stress recovery. Resolution increases with the number of elements in the gradient direction, while collapsed triangular faces, excessive twisting between end faces, and highly skewed side faces degrade the mapping.

C3D6 matches geometries with a natural wedge topology. C3D15 provides the corresponding higher-order interpolation for quadratic swept meshes.

C3D8: 8-node hexahedral solid



Element	C3D8
Type	solid
Nodes	8
Faces	6
Edges	12
DOF / node	3
Total DOF	24
Shape functions	trilinear Lagrange
Integration points	8

Figure 4.4: C3D8 hexahedral solid element.

Syntax

```
*ELEMENT, TYPE=C3D8, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>, <n7>, <n8>
```

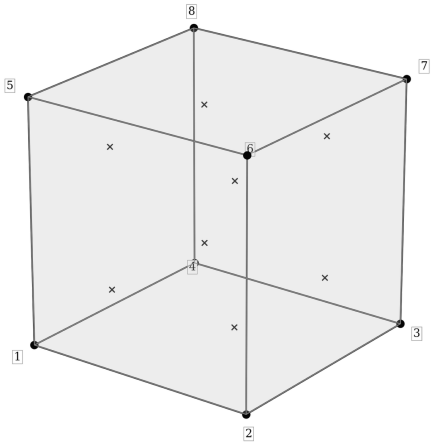
C3D8 is the standard low-order hexahedral continuum element. It uses trilinear displacement interpolation and full integration. For regular block-like, swept, or layered meshes it is often an efficient general-purpose solid because element directions can follow geometry and material orientation naturally.

Hexahedral topology provides direct control of the number of elements through thickness and along structural load paths. A regular C3D8 mesh therefore represents plates modelled with solids, bonded layers, spars, ribs, extruded components, and regions whose orthotropic material axes follow the mesh.

The element remains low order. A coarse mesh can be too stiff in bending, and straight element edges only approximate curved boundaries. Through-thickness resolution and refinement of curved or stress-critical geometry directly control the corresponding bending-stress and geometry errors.

Warping, skew, negative Jacobians, and extreme aspect ratios can remove the numerical advantages of the hexahedral topology. A clean quadratic tetrahedral mesh can provide a more accurate mapping than a badly distorted hexahedral mesh; topology alone does not determine mesh quality.

C3D8I: incompatible-mode 8-node hexahedral solid



Element	C3D8
Type	solid
Nodes	8
Faces	6
Edges	12
DOF / node	3
Total DOF	24
Shape functions	trilinear Lagrange
Integration points	8

Figure 4.5: C3D8I topology; the eight external nodes are identical to C3D8.

Syntax

```
*ELEMENT, TYPE=C3D8I, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>, <n7>, <n8>
```

C3D8I uses the same trilinear eight-node geometry and the same full $2 \times 2 \times 2$ integration rule as C3D8. Its compatible displacement field is augmented by 13 element-local enhanced modes. Nine principal vector-valued modes improve bending behaviour by removing parasitic constraints of the low-order compatible interpolation; four additional volumetric modes improve the response in nearly incompressible states.

The 13 enhanced parameters are internal to each element. FEMaster solves their local stationarity equations and statically condenses them, so a C3D8I still contributes only the 24 translational nodal degrees of freedom of an ordinary eight-node solid to the global system. Unlike C3D8R, the element does not rely on reduced integration or hourglass stabilization.

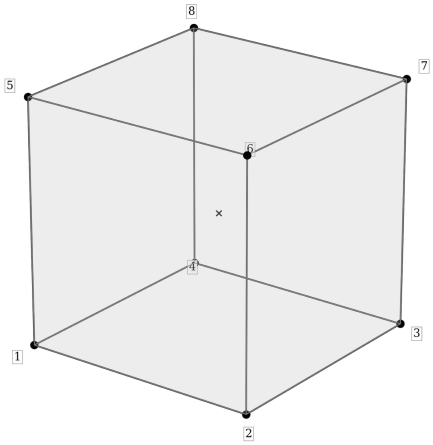
For geometrically nonlinear analysis the enhanced parameters are solved by a local Newton iteration at every element evaluation and the complete material and geometric tangent blocks are condensed by a Schur complement. The finite-strain kinematics use a right multiplicative enhancement

$$\bar{\mathbf{F}} = \mathbf{F}_c \left(\mathbf{I} + \sum_m \alpha_m \mathbf{H}_m \right).$$

The enhanced tensors therefore remain in the reference configuration. A superposed spatial rigid rotation maps $\bar{\mathbf{F}} \to \mathbf{Q}\bar{\mathbf{F}}$, so the enhanced mode space and condensed response remain objective.

C3D8I is intended primarily for low-order hexahedral meshes in which bending or near-incompressibility makes a standard C3D8 unnecessarily stiff. It remains a trilinear geometry mapping and is still sensitive to severe warping and skew. The incompatible modes improve the kinematics; they do not turn a poor eight-node geometry into a higher-order element.

C3D8R: reduced-integration 8-node hexahedral solid



Element	C3D8R
Type	solid
Nodes	8
Faces	6
Edges	12
DOF / node	3
Total DOF	24
Shape functions	trilinear Lagrange with hourglass stabilization
Integration points	1

Figure 4.6: C3D8R reduced-integration hexahedral solid element.

Syntax

```
*ELEMENT, TYPE=C3D8R, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>, <n7>, <n8>
```

C3D8R uses the same eight-node topology as C3D8 but evaluates the continuum contribution with one integration point at the element centre. Reduced integration lowers assembly cost and avoids some of the excessive stiffness that a fully integrated low-order hexahedron can exhibit in bending-dominated states.

One-point integration also introduces zero-energy hourglass deformation modes. FEMaster therefore adds hourglass stabilization. The primitive scalar hourglass patterns are based on the four non-affine modes

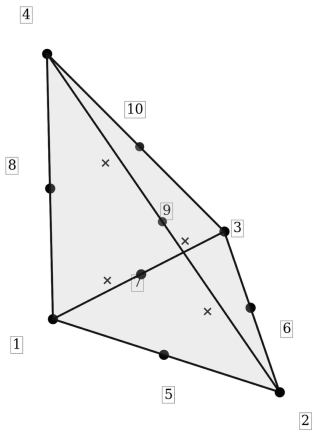
$$\gamma_1 = s\,t, \qquad \gamma_2 = t\,r, \qquad \gamma_3 = r\,s, \qquad \gamma_4 = r\,s\,t.$$

These modes are projected so that affine displacement fields are not penalized. The stabilization scale is based on the element reference volume, the reference geometry gradients, and an effective shear stiffness derived from the constitutive tangent. The default dimensionless hourglass coefficient is $\alpha = 0.005$.

Newton convergence alone does not characterize C3D8R; the deformation pattern and stabilization sensitivity are independent numerical properties. A coarse or badly distorted mesh can converge while exhibiting nonphysical hourglass deformation. Comparison with C3D8 and mesh refinement expose response components associated with hourglass-sensitive deformation.

Severe finite rotations and highly distorted elements increase sensitivity to reduced-integration stabilization. Relative to C3D8, C3D8R exchanges lower integration cost and locking tendency for dependence on hourglass control.

C3D10: 10-node quadratic tetrahedral solid



Element	C3D10
Type	solid
Nodes	10
Faces	4
Edges	6
DOF / node	3
Total DOF	30
Shape functions	quadratic tetrahedral Lagrange
Integration points	4

Figure 4.7: C3D10 quadratic tetrahedral element.

Syntax

```
*ELEMENT, TYPE=C3D10, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>, <n7>, <n8>, <n9>, <n10>
```

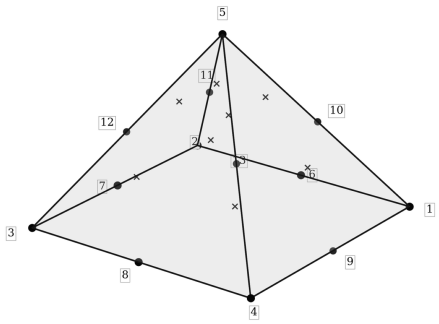
C3D10 is the quadratic tetrahedral solid. Six midside nodes augment the four corner nodes and allow quadratic displacement interpolation. Compared with C3D4, the element can represent curved deformation, bending, and stress gradients far more efficiently.

For automatic tetrahedral meshing, C3D10 retains the geometric flexibility of tetrahedra while reducing the excessive stiffness and piecewise-constant strain behaviour associated with linear tetrahedra.

The quadratic geometry mapping also represents curved boundaries more accurately when midside nodes follow the CAD geometry. Badly displaced midside nodes can create a distorted mapping even if the corner tetrahedron appears acceptable.

The element costs more per cell than C3D4, but substantially fewer cells may be required for comparable displacement and stress accuracy. Refinement is still required near singularities, sharp re-entrant corners, and concentrated loads where no polynomial element can make a mathematical singularity disappear.

C3D13: 13-node quadratic pyramid solid



Element	C3D13
Type	solid
Nodes	13
Faces	5
Edges	8
DOF / node	3
Total DOF	39
Shape functions	quadratic pyramid
Integration points	8

Figure 4.8: C3D13 quadratic pyramid topology and integration scheme.

Syntax

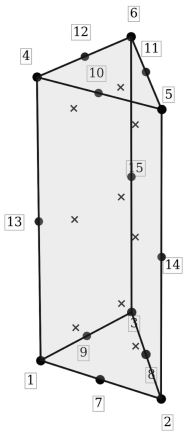
```
*ELEMENT, TYPE=C3D13, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>, <n7>, <n8>, <n9>, <n10>, <n11>, <n12>, <n13>
```

C3D13 is a thirteen-node quadratic pyramid element with rational shape functions in the natural pyramid coordinates. Nodes 1–4 define the quadrilateral base and node 5 the apex. Nodes 6–9 lie on the four base edges, while nodes 10–13 lie between the base corners and the apex. The rational interpolation accommodates the collapsed pyramid geometry and provides a higher-order displacement description than the five-node C3D5 transition element.

A pyramid is useful where a quadrilateral-faced hexahedral mesh must transition to a region containing tetrahedral cells. Compared with C3D5, the midside nodes can represent curved edges and varying deformation fields more accurately, provided their positions are consistent with the intended physical geometry. Strongly distorted pyramids or poor apex placement can still result in inaccurate Jacobians, stiffness, and stress recovery.

The element uses the DOMAIN_ISO_PYRAMID quadrature with ORDER_QUARTIC (27 integration points) and provides five outward-oriented quadratic faces for surface definitions: an eight-node quadrilateral base (S1) and four six-node triangular sides (S2–S5). Recovered stress and strain values are extrapolated from the constitutive integration points. As for other quadratic solids, local stresses should be assessed through mesh refinement rather than a single extrapolated nodal peak.

C3D15: 15-node quadratic wedge solid



Element	C3D15
Type	solid
Nodes	15
Faces	5
Edges	9
DOF / node	3
Total DOF	45
Shape functions	quadratic wedge / prism
Integration points	9

Figure 4.9: C3D15 quadratic wedge element.

Syntax

```
*ELEMENT, TYPE=C3D15, ELSET=<set>
<id>, <n1>, ..., <n15>
```

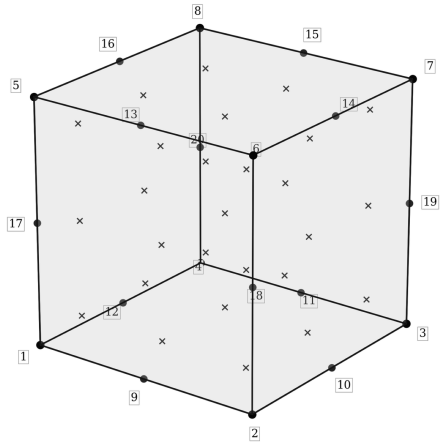
C3D15 is the quadratic counterpart of the six-node wedge. It adds midside nodes and represents curvature, bending, and through-thickness gradients at higher order than C3D6. Its topology matches higher-order swept meshes and prismatic regions created by extruding a quadratic triangular surface mesh.

The element combines wedge topology with higher-order through-thickness interpolation. Curved swept solids, local layered regions, and transition zones can therefore require fewer elements than with C3D6.

Higher-order mapping increases sensitivity to midside-node placement. Poorly shaped triangular end faces, excessive twisting of the side faces, and midside nodes that depart from the intended geometry degrade the mapping. A high-order wedge with a poor mapping is not automatically more accurate than a clean low-order mesh.

C3D15 represents quadratic wedge meshes. A structured quadratic hexahedral mesh provides the more regular C3D20 topology where geometry permits it, whereas C3D10 supports fully unstructured complex geometry.

`C3D20`: 20-node quadratic hexahedral solid



Element	C3D20
Type	solid
Nodes	20
Faces	6
Edges	12
DOF / node	3
Total DOF	60
Shape functions	quadratic serendipity hexahedral
Integration points	27

Figure 4.10: `C3D20` quadratic hexahedral element.

Syntax

```
*ELEMENT, TYPE=C3D20, ELSET=<set>
<id>, <n1>, ..., <n20>
```

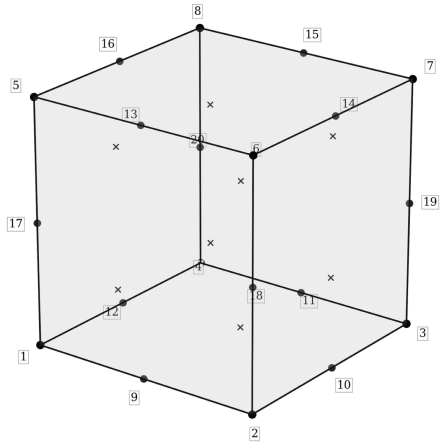
`C3D20` is the full-integration quadratic hexahedral solid. The eight corner nodes are supplemented by twelve midside nodes, giving the element higher-order bending and stress-gradient representation than `C3D8`. Curved geometry is also represented at higher order when midside nodes follow the physical boundary.

For smooth structural fields and high-quality structured meshes, `C3D20` provides high-order solid accuracy. Its quadratic hexahedral interpolation represents curved blocks, swept volumes, thick structural regions, and smooth stress gradients.

The main costs are larger element matrices and stricter geometric quality requirements. Distortion of a quadratic mapping can be more harmful than distortion of a linear one because the midside nodes participate directly in the geometry interpolation. Jacobian quality therefore depends on both corner and midside-node positions.

Relative to `C3D8`, `C3D20` exchanges larger element matrices for higher-order accuracy. A fine regular `C3D8` mesh can remain less expensive for models dominated by linear global stiffness.

C3D20R: 20-node reduced-integration hexahedral solid



Element	C3D20R
Type	solid
Nodes	20
Faces	6
Edges	12
DOF / node	3
Total DOF	60
Shape functions	quadratic serendipity hexahedral
Integration points	8

Figure 4.11: C3D20R reduced-integration quadratic hexahedral solid element.

Syntax

```
*ELEMENT, TYPE=C3D20R, ELSET=<set>
<id>, <n1>, ..., <n20>
```

C3D20R uses the same twenty-node topology and quadratic interpolation as C3D20, but the stiffness contribution is evaluated with reduced integration. The reduced rule lowers cost and can improve behaviour in states where full integration introduces excessive constraint or stiffness.

Reduced integration changes the numerical character of the element; it is not merely a faster switch. Nonphysical low-energy deformation modes and sensitivity to mesh distortion can appear in deformed shapes, reaction balance, strain-energy distribution, and comparison with C3D20.

The formulation combines a quadratic hexahedral mesh with reduced integration. C3D20 provides the corresponding fully integrated comparison formulation.

Solid stress and strain recovery is performed at the constitutive integration points and then extrapolated to element nodes; global nodal results are averaged over element contributions. An element with fewer integration points cannot provide the same recovered spatial variation as its full-integration counterpart. See Section 4.7 for the distinction between integration-point, element-nodal, and averaged nodal fields.

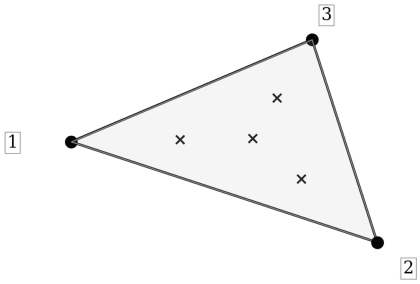
4.2.8.3 Shell elements

Shell elements model thin-walled structures through a two-dimensional reference surface with an assigned thickness or generalized sectional stiffness. Their kinematics represent membrane stretching, bending, and transverse shear without discretizing the thickness with continuum elements. FEMaster's finite-rotation shells use independent nodal translations and rotations, so the current orientation of the shell can be represented directly. Typical applications include plates, aircraft skins, curved panels, and other lightweight structures dominated by midsurface deformation.

The recovered quantities include membrane forces, bending moments, transverse-shear resultants, and supported top/bottom stresses, but not an arbitrary three-dimensional through-thickness stress field. A solid model may be more suitable near concentrated load introductions, connections, or locations with significant thickness effects. The shell normal determines the top/bottom convention, affects pressure direction, and enters the interpretation of bending results. Mesh quality, surface orientation, thickness, and material directions must therefore be defined consistently.

Four finite-rotation formulations are available, covering triangular and quadrilateral topologies with linear and quadratic geometric interpolation. Triangular meshes accommodate irregular boundaries, while quadrilateral meshes are particularly effective on structured surfaces. Higher-order elements resolve smooth curvature more efficiently but increase element cost and require well-positioned midside nodes. Each heading lists all accepted aliases of the same FEMaster implementation.

`MITC3FRT` / `S3` / `S3R`: 3-node finite-rotation triangular shell



Element	MITC3FRT
Type	shell
Nodes	3
Faces	-
Edges	3
DOF / node	6
Total DOF	18
Shape functions	finite-rotation MITC3 shell
Integration points	4

Figure 4.12: Three-node finite-rotation triangular shell geometry and node ordering.

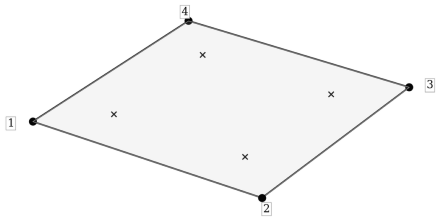
Syntax

```
*ELEMENT, TYPE=MITC3FRT, ELSET=<set>
<id>, <n1>, <n2>, <n3>
```

The three-node triangular shell combines a linear geometric interpolation with finite-rotation kinematics and independent nodal translations and rotations. It represents membrane deformation, bending, and transverse shear within the common shell framework. This compact topology is useful for unstructured surface meshes, irregular boundaries, openings, and transitions where a regular quadrilateral mesh would be difficult to construct.

A single linear triangle has limited resolution of curved geometry and spatially varying bending fields. Bending-dominated or strongly curved models therefore require refinement, with convergence checked using displacements, stresses or shell resultants, and strain energy. The aliases `S3` and `S3R` select the same implementation as `MITC3FRT`; the `R` suffix does not select a separate reduced-integration formulation in FEMaster.

`MITC4FRT` / `S4` / `S4R` / `MITC4`: 4-node finite-rotation quadrilateral shell



Element	MITC4FRT
Type	shell
Nodes	4
Faces	-
Edges	4
DOF / node	6
Total DOF	24
Shape functions	finite-rotation MITC4 shell
Integration points	4

Figure 4.13: Four-node finite-rotation MITC shell geometry and node ordering.

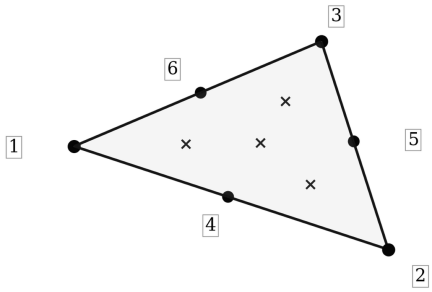
Syntax

```
*ELEMENT, TYPE=MITC4FRT, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>
```

The four-node quadrilateral shell combines finite-rotation kinematics with a Mixed Interpolation of Tensorial Components (MITC) treatment of transverse shear. In thin-shell bending, an ordinary low-order displacement interpolation can become artificially stiff because the shear strains cannot approach their limiting values correctly. This numerical effect, known as shear locking, is mitigated by the MITC interpolation while transverse-shear deformation remains part of the underlying shell model.

The formulation is well suited to plates, skins, stiffened panels, and shell assemblies discretized with regular quadrilaterals. It represents bending and twisting through independent nodal rotations rather than an internally expanded solid mesh. Nevertheless, strong warping, severe element skewness, and insufficient surface resolution can reduce accuracy, especially for curved geometry and nonlinear deformation paths. The names `S4`, `S4R`, and `MITC4` are aliases for `MITC4FRT`; they do not select different integration schemes.

`MITC6FRT` / `S6` / `S6R`: 6-node finite-rotation triangular shell



Element	MITC6FRT
Type	shell
Nodes	6
Faces	-
Edges	3
DOF / node	6
Total DOF	36
Shape functions	finite-rotation MITC6-b shell
Integration points	4

Figure 4.14: Six-node finite-rotation triangular shell geometry and node ordering.

Syntax

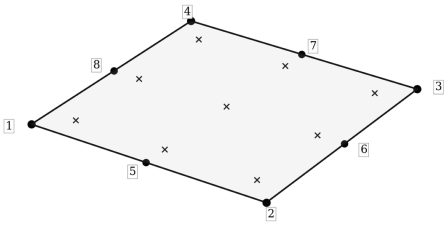
```
*ELEMENT, TYPE=MITC6FRT, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>
```

The six-node triangular shell extends the finite-rotation midsurface description to quadratic geometric interpolation. Its midside nodes allow curved edges and a more detailed representation of smoothly varying displacement and bending fields than the three-node triangle. It is useful for curved shell regions, cutouts, and unstructured higher-order surface meshes where triangular topology is preferred.

The element uses the MITC6-b assumed-strain formulation. Membrane strains and curvatures are independently interpolated through the in-plane assumed-strain operator, while the transverse shear field uses the MITC6-b tying-point interpolation. The formulation evaluates nine in-plane tying-point entries and six transverse-shear entries; these are distinct from the numerical integration points. Its area quadrature is `DOMAIN_ISO_TRI`, `ORDER_QUARTIC`, using **six integration points**, rather than the former four-point cubic rule. The same six-point rule supplies the element's area integration and integration-point result locations.

The higher interpolation order improves geometric flexibility but does not eliminate sensitivity to mesh quality. Midside nodes must follow the intended surface and highly distorted triangles can produce inaccurate stiffness and recovered results. Thin-shell bending calculations should therefore be checked by mesh refinement. The `S6` and `S6R` aliases select exactly the same FEMaster implementation as `MITC6FRT`.

`MITC8FRT` / `S8` / `S8R` / `MITC8`: 8-node finite-rotation quadrilateral shell



Element	MITC8FRT
Type	shell
Nodes	8
Faces	-
Edges	4
DOF / node	6
Total DOF	48
Shape functions	finite-rotation MITC8 shell
Integration points	9

Figure 4.15: Eight-node finite-rotation MITC shell geometry and node ordering.

Syntax

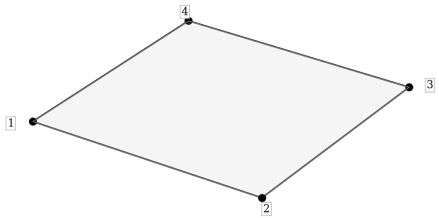
```
*ELEMENT, TYPE=MITC8FRT, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>, <n7>, <n8>
```

The eight-node quadrilateral shell combines higher-order midsurface interpolation, finite-rotation kinematics, and an MITC transverse-shear treatment. The midside nodes provide a richer description of smooth curvature, displacement, and bending fields than the four-node element. This makes the formulation suitable for curved panels and shell regions where accurate geometry and smooth stress-resultant gradients are important.

Compared with `MITC4FRT`, the additional nodes increase element cost but may reduce the number of elements needed for smooth curved geometry. Accuracy still depends on the placement of midside nodes, consistent shell normals, and acceptable element warping. The shell idealization does not resolve detailed through-thickness continuum stresses. The aliases `S8`, `S8R`, and `MITC8` select this same `MITC8FRT` implementation.

Although grouped with the shell and panel formulations, `QSPT` is a specialized shear-panel element rather than a general-purpose shell. Its kinematics and recoverable quantities differ from those of the finite-rotation MITC shells described above.

`QSPT`: quadrilateral shear-panel element



Element	QSPT
Type	shell
Nodes	4
Faces	-
Edges	4
DOF / node	3
Total DOF	12
Shape functions	quadrilateral shear-panel formulation
Integration points	-

Figure 4.16: `QSPT` shear-panel element.

Syntax

```
*ELEMENT, TYPE=QSPT, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>
```

`QSPT` is a specialized four-node quadrilateral panel element developed for shear-flow-oriented structural idealizations. It is intended for simplified panel models where the principal purpose of the element is to transfer in-plane shear rather than reproduce the complete behaviour of a general shell.

Typical applications are preliminary thin-walled structures, webs, aircraft-style shear panels, and reduced-order structural models in which panel shear flow is the primary quantity of interest. In such models `QSPT` can provide a compact load-path representation without introducing a complete shell formulation for every panel.

`QSPT` is not a general replacement for `MITC4FRT` despite sharing a four-node quadrilateral topology. It does not provide the bending stiffness, local shell curvature, full shell resultants, or detailed top/bottom stresses of a shell formulation.

4.2.8.4 Beam and truss elements

Line elements idealize slender three-dimensional members by describing the cross-section through section properties instead of meshing it explicitly. This can reduce the number of degrees of freedom by orders of magnitude compared with shell or solid modelling. The trade-off is that local three-dimensional connection behaviour, detailed cross-section distortion, holes, welds, and local stress concentrations are not resolved.

B31: 2-node geometrically exact 3D beam

Syntax

```
*ELEMENT, TYPE=B31, ELSET=<set>
<id>, <n1>, <n2>
```

B31 is a two-node shear-flexible spatial beam with three translational and three rotational degrees of freedom per node. The rotational degrees of freedom use FEMaster's total global axis–angle convention. The element evaluates finite rotations on $SO(3)$, interpolates the relative nodal rotation objectively along the shortest rotation path, and carries the six generalized section strains

$$[\varepsilon_x, \gamma_y, \gamma_z, \kappa_x, \kappa_y, \kappa_z]^T.$$

The formulation uses the two end points as order-one Gauss–Lobatto integration points. Axial, transverse-shear, torsional and bending stiffnesses are obtained from the assigned isotropic material and *PROFILE. The profile values A_{sy} and A_{sz} define the effective shear areas; if they are omitted, both default to $5A/6$.

Finite section offsets are part of the beam kinematics. With SP the shear point, SMP the section material/centroid point and REF the nodal reference line, the stored profile data define

$$SP - SMP = (0, e_y, e_z), \qquad REF - SMP = (0, \mathrm{ref}_y, \mathrm{ref}_z).$$

The vector from the nodal reference line to the shear point is therefore rotated with the current section orientation. The $SP - SMP$ eccentricity also enters the sectional constitutive matrix so axial/bending coupling is represented without applying a small-rotation offset matrix after assembly.

For nonlinear static analysis the internal force and the complete tangent are evaluated at the supplied base state. A superposed finite rigid-body translation and rotation therefore produces zero generalized strain. Linear static, eigenfrequency, harmonic and buckling operators use the same B31 reference tangent and mass/geometric operators. The current mass operator is the order-one collocated sectional mass matrix including reference-line eccentricity.

The relative rotation between the two B31 nodes must remain below 180 degrees so the shortest $SO(3)$ interpolation branch is unique. Large overall rigid-body rotations are not restricted by this condition; it applies only to the relative nodal rotation within one element.

B33: 3D beam element



Element	B33
Type	line
Nodes	2
Faces	-
Edges	1
DOF / node	6
Total DOF	12
Shape functions	linear 2-node beam
Integration points	2

Figure 4.17: B33 beam element.

Syntax

```
*ELEMENT, TYPE=B33, ELSET=<set>
<id>, <n1>, <n2>
```

B33 is a two-node three-dimensional beam element with translational and rotational nodal degrees of freedom. It represents axial deformation, bending, torsion, and the section response defined by *BEAM SECTION and the referenced *PROFILE.

The current *ELEMENT syntax uses exactly two connectivity nodes for B33. Beam cross-section orientation is supplied by the orientation vector in *BEAM SECTION; an additional orientation node is not part of the current connectivity grammar.

Beam elements represent slender frames, stiffeners, spars, ribs, idealized support members, and other structures through Section forces and global deformation. They include axial area, bending inertias, torsional stiffness, product of inertia, Section offsets, and reference-line offsets without meshing the full cross-section.

The beam idealization does not resolve local connection stresses, bolt or weld geometry, detailed cross-section deformation, or three-dimensional load introduction. Those effects require a corresponding shell or solid representation. The beam local axes determine the interpretation of bending moments, section forces, and profile properties.

The Abaqus-compatible B33 name uses the same two-node connectivity.

T3: 2-node truss element



Element	T3
Type	line
Nodes	2
Faces	-
Edges	1
DOF / node	3
Total DOF	6
Shape functions	linear 2-node truss
Integration points	1

Figure 4.18: T3 truss element.

Syntax

```
*ELEMENT, TYPE=T3, ELSET=<set>
<id>, <n1>, <n2>
```

T3 is a two-node truss element. It carries axial force along its current line direction and uses translational nodal degrees of freedom. The material and cross-sectional area are assigned by *TRUSS SECTION.

The element represents pin-jointed members, rods, braces, links, and other components whose structural action is axial tension or compression. Its mechanical quantities are axial force, axial strain, and axial stress.

The limitations are deliberate. T3 has no bending stiffness, no torsional stiffness, and no rotational continuity. It cannot model fixed-end moments, eccentric frame action, or transverse stiffness of a physical beam; those behaviours require B33 or a shell/solid representation.

The Abaqus-compatible name T3D2 selects the same FEMaster truss formulation as T3.

```
*ELEMENT, TYPE=T3D2, ELSET=BARS
10, 1, 2
```

4.2.8.5 Point elements

Point elements are zero-dimensional structural elements attached to a single node. They preserve element identity and ELSET membership while their physical contribution is supplied by an assigned point-mass section rather than by continuum geometry or a material law. The unified input language accepts three regular one-node types: `MASS`, `ROTARYI`, and `SPRING1`. Internally all three use the same `PointElement`; the corresponding property keyword determines which generalized diagonal coefficient is populated.

A point element without an assigned point property is intentionally inert. It has zero stiffness, zero mass, no stress or strain state, and activates no degrees of freedom. Property definitions are documented in Section 4.3.2; this section focuses on the element topology and kinematic role.

MASS: 1-node concentrated-mass element

Syntax

```
*ELEMENT, TYPE=MASS, ELSET=<set>
<id>, <node>
*MASS, ELSET=<set>
<mass>
```

MASS is an Abaqus-compatible one-node point element. The *ELEMENT record creates only topology: an element ID, one connected node, and normal ELSET membership. The scalar physical mass is assigned separately by *MASS, ELSET=....

For the supported isotropic property, the element mass matrix is

$$M_e = \mathrm{diag}(m, m, m, 0, 0, 0),$$

so a positive mass activates the three translational components at the connected node. Several MASS elements may reference the same node; each represents an independent concentrated contribution and the assembled mass is additive.

The point element has zero geometric volume, but its density-scaled field integration interprets m as the complete concentrated mass measure. Consequently, a regular MASS element contributes to the global mass matrix, eigenfrequency and transient analyses, inertia relief, and element-targeted mass-proportional body or inertial loads. The Abaqus-compatible *DLOAD with GRAV produces the concentrated force $m\boldsymbol{a}$ when the selected element region contains the point element. FEMaster-specific *VLOAD instead denotes force density and does not load this zero-volume point element.

Part-local MASS elements and their *MASS property assignments are copied to every Instance during model compilation. An assembly-level *MASS assignment instead targets compiled point elements through an assembly ELSET.

Alternative: For concentrated mass that does not require an explicit element ID or ELSET membership, the FEMaster-specific *POINTMASS keyword assigns a mass directly to an NSET:

```
*POINTMASS, NSET=PAYLOAD_NODES
0.025
```

This creates auxiliary point elements during model compilation. The specified value is applied independently to each node in the set; load selection and element-set behavior differ from a regular MASS element. See Section 4.3.2.

ROTARYI: 1-node rotary-inertia element

Syntax

```
*ELEMENT, TYPE=ROTARYI, ELSET=<set>
<id>, <node>
*ROTARY INERTIA, ELSET=<set>
<I11>, <I22>, <I33>, <I12>, <I13>, <I23>
```

ROTARYI is the rotational-inertia counterpart of MASS. Its one-node topology is identical, but the property acts on the rotational degrees of freedom. FEMaster's supported diagonal mapping produces

$$M_e = \mathrm{diag}(0, 0, 0, I_{11}, I_{22}, I_{33}).$$

Each nonzero diagonal inertia activates its corresponding nodal rotation.

Abaqus permits the full symmetric rotary-inertia tensor. The current FEMaster point section stores only the diagonal moments, so the data positions I_{12}, I_{13}, and I_{23} are accepted only when zero. Orientation-dependent rotary inertia is likewise outside the current mapping.

A ROTARYI element can share its connected node with a MASS element. This is the regular element-based way to represent translational mass and rotary inertia at the same physical point while preserving the Abaqus separation between the two element families.

SPRING1: 1-node point-to-ground spring

Syntax

```
*ELEMENT, TYPE=SPRING1, ELSET=<set>
<id>, <node>
*SPRING, ELSET=<set>
<dof>
<stiffness>
```

SPRING1 is a one-node spring between the connected node and ground. The first *SPRING data line selects the generalized nodal direction; the second gives the constant linear stiffness. Degrees of freedom 1–3 act on translations and degrees of freedom 4–6 act on rotations.

For one selected direction the point-element stiffness matrix contains exactly one nonzero diagonal entry. Translational DOFs therefore represent ordinary point-to-ground springs, whereas rotational DOFs represent torsional point-to-ground springs. Multiple independent directions are represented by multiple SPRING1 elements, which may all reference the same node.

The current mapping is deliberately narrower than the complete Abaqus spring family. It supports only real, constant, linear SPRING1 behavior in the global nodal system. SPRING2, SPRINGA, local orientations, nonlinear force–displacement curves, frequency or field dependence, and complex stiffness are not mapped.

The FEMaster-specific *POINTMASS keyword provides an alternative NSET-based convenience syntax when mass, rotary inertia, and several diagonal translational or rotational springs should be combined directly on the same target nodes. It creates auxiliary post-compile point elements rather than regular ELSET members.

4.2.9 *NSET

*NSET creates or extends a named node set. The same basic syntax is accepted by native FEMaster and by the supported Abaqus input format. NSET gives the set name; NAME is accepted as an alternative spelling. Data may contain explicit node references or generated arithmetic ranges.

Inside a Part, numerical entries refer to that Part's local node IDs. Separate Parts may therefore contain sets with the same numerical members without referring to the same physical nodes. If a Part is instantiated more than once, its Part-level node sets remain available for every copy.

At Assembly level, INSTANCE=<name> qualifies local IDs with one particular Instance. This is the most direct way to create a support or load region on one placed copy of a reusable Part. Assembly-level sets may also combine selections from different Instances by using the supported qualified references or by building larger sets from previously named regions.

With GENERATE, every data row contains

start, end, increment.

The end value is inclusive. The increment defaults to 1 and must not be zero. Positive and negative increments are supported, so generated sequences can run in either direction as long as the sign agrees with the requested range.

Part-level sets represent regions intrinsic to the component, such as a bolt row, root edge, or material domain, and are available on every Instance. Assembly-level sets represent placement-dependent selections, such as one support on a particular Instance or a region combining nodes from several components.

Parameters

Parameter	Status	Meaning
NSET / NAME	required	Name of the node set to create or extend.
INSTANCE	optional	Assembly-level qualifier for local node references.
GENERATE	optional flag	Interpret each data row as start, end, increment.

Data lines

Entry	Meaning
Explicit form	One or more node IDs or supported named/qualified node references.
Generated form	start, end, increment; the increment defaults to 1 and must be nonzero.

Node sets are reusable selections for supports, loads, or output. The first definition selects endpoints explicitly; the second generates a sequence of node IDs.

FEMaster and Abaqus example

```
*NSET, NSET=ENDS
0, 3

*NSET, NSET=ALLN, GENERATE
0, 3, 1
```

Instance-qualified node set

```
*NSET, NSET=ROOT, INSTANCE=LEFT
1, 2, 3
```

Assembly-level Instance selection

A typical Assembly selection looks like

Syntax

```
*ASSEMBLY
*INSTANCE, NAME=BEAM_A, PART=BEAM
0., 0., 0.
*END INSTANCE
*INSTANCE, NAME=BEAM_B, PART=BEAM
100., 0., 0.
*END INSTANCE

*NSET, NSET=SUPPORT_A, INSTANCE=BEAM_A
0
*NSET, NSET=SUPPORT_B, INSTANCE=BEAM_B
0
*END ASSEMBLY
```

Both sets contain a local node with ID `0`, but they refer to different placed Instances.

4.2.10 *ELSET

*ELSET is the element-domain counterpart of *NSET. It creates or extends a named element set from explicit element references or generated ranges. ELSET supplies the name and NAME is accepted as an alternative.

Element sets are central to property assignment. Solid, shell, beam, and truss Sections normally target an *ELSET, so the element-set structure can mirror the physical construction of the model through separate names for different materials, thicknesses, beam profiles, or design domains.

Inside a Part, entries refer to that Part's local element IDs. At Assembly level, INSTANCE qualifies local IDs with one placed Instance. This allows two Instances of the same Part to be selected independently even though they use identical local element numbering.

GENERATE follows the same inclusive start/end/increment rule as *NSET. ID 0 is valid, so for example 0, 99, 1 generates one hundred element IDs if they exist in the intended topology.

A Section assigned to a Part-level element set belongs to the component and follows every Instance of the Part. Assembly-level element sets instead identify loads, diagnostics, or other selections whose meaning depends on the particular placed copy.

Parameters

Parameter	Status	Meaning
ELSET / NAME	required	Name of the element set to create or extend.
INSTANCE	optional	Assembly-level qualifier for local element references.
GENERATE	optional flag	Generate an arithmetic sequence of element IDs.

Data lines

Entry	Meaning
Explicit form	One or more element IDs or supported named/qualified element references.
Generated form	start, end, increment; inclusive end, nonzero increment.

Generated element sets save long ID lists when the numbering follows a regular stride. The referenced element IDs must already exist within the relevant topology scope.

FEMaster and Abaqus example

```
*ELSET, ELSET=DESIGN_DOMAIN, GENERATE
0, 99, 1

*ELSET, ELSET=EVEN_ELEMENTS, GENERATE
0, 98, 2
```

4.2.11 *SURFACE

*SURFACE defines a geometric boundary from element sides. A surface is more than a node set: it retains which element face is selected and therefore has a well-defined interpolation, area or line measure, and orientation. Surface information is required for pressure loads, distributed tractions, surface couplings, ties, and contact.

A side is selected by an element or element-set target together with a side label such as S1, S2, and so on. The actual face represented by a side label depends on the element type and its connectivity. Reversing or changing the element node order can therefore change the physical direction of a surface normal.

For shell elements the two sides of the midsurface can also be described by SPOS and SNEG. These represent the positive and negative shell sides that determine pressure direction and top/bottom interpretation.

Native FEMaster additionally supports an explicit surface-ID form in which a user surface ID is supplied together with the parent element and side. Explicit surface IDs are non-negative and may start at 0. A surface named directly from an element or element set avoids the separate numerical surface-ID space of the explicit form.

At Assembly level, INSTANCE can qualify the target element or element set. This allows a face of one particular Instance to be selected even when several copies use the same local element IDs and set names.

The Abaqus example uses the supported element-based *SURFACE, TYPE=ELEMENT syntax. Only the specialized surface features documented here are included in the supported Abaqus subset.

An unordered node set contains no element-side information and therefore cannot define a pressure or contact normal. The selected element side determines the surface orientation. For shell models, normal consistency additionally controls the positive and negative shell faces and the sign convention of bending-related quantities.

Parameters

Parameter	Status	Meaning
SFSET / NAME	optional	Name of the destination surface region; native default SFALL.
INSTANCE	optional	Assembly-level Instance qualifier.

Data lines

Entry	Meaning
Target form	element-or-elset, side.
Explicit native form	surface_id, element_id, side.

This definition selects face S2 from each element in SOLIDS. Face numbering depends on the element formulation; consult the element schematics when selecting the loaded side.

FEMaster and Abaqus example

```
*SURFACE, NAME=TOP, TYPE=ELEMENT
SOLIDS, S2
```

Direct surface references

```
*SURFACE, NAME=TOP
SOLIDS, S2

*SURFACE, NAME=ONE_FACE
0, 42, S1
```

4.2.12 *SFSET

*SFSET groups existing native FEMaster surface entities into a named surface region. It combines surfaces created with explicit user surface IDs or several previously defined surface groups into one target.

*SURFACE and *SFSET therefore serve different purposes: *SURFACE creates geometric boundary entities from elements, while *SFSET collects already existing surface entities. A direct named *SURFACE does not require the separate numerical surface-ID space used by *SFSET.

The destination name is supplied by SFSET or NAME; the native default is SFALL. Explicit surface references can be listed directly. With GENERATE, arithmetic ranges of surface IDs can be produced using the same inclusive start/end/increment convention as node and element sets. INSTANCE can be used for Assembly-level selection where applicable.

*SFSET is a FEMaster extension. A regular *SURFACE definition can instead select the same physical faces directly, as shown in the alternative below. The two forms do not require separate input modes.

Parameters

Parameter	Status	Meaning
SFSET / NAME	optional	Destination surface-region name; native default SFALL.
INSTANCE	optional	Assembly-level Instance qualifier.
GENERATE	optional flag	Generate an arithmetic sequence of surface IDs.

Data lines

Entry	Meaning
Explicit form	Existing surface IDs or supported named surface references.
Generated form	start, end, increment; inclusive end, nonzero increment.

The first definitions create surface entities; SFSET then groups their identifiers into one reusable region. A regular SURFACE can describe the same geometric selection directly.

FEMaster and Abaqus example

```
*SURFACE, NAME=PATCH_A
0, 20, S1
1, 21, S1
*SURFACE, NAME=PATCH_B
2, 22, S1

*SFSET, NAME=CONTACT_FACE
0, 1, 2
```

Equivalent element surface selection

```
*SURFACE, NAME=CONTACT_FACE, TYPE=ELEMENT
20, S1
21, S1
22, S1
```

4.3 Model Properties

4.3.1 Materials

Materials define constitutive and inertial properties independently of the mesh. A material is created once, given a name, and referenced by one or more Sections. The same material may therefore be used by several element regions, Parts, or Instances without duplicating its constants.

`*MATERIAL` begins a material definition. The material-property keywords that follow—for example `*ELASTIC`, `*PLASTIC`, `*HYPERELASTIC`, `*DENSITY`, and thermal expansion—belong to that material until the material definition ends with the next appropriate root-level model definition. A material may contain only the properties required by the intended analyses; for example, density is unnecessary for a purely elastic static stiffness calculation but is required when mass participates in the mechanics.

4.3.1.1 *MATERIAL

*MATERIAL creates a named material and makes it the current target for subsequent material-property keywords. Native FEMaster accepts the identifier through MATERIAL or NAME; Abaqus input uses NAME.

The command itself does not define a constitutive law. Elasticity, plasticity, hyperelasticity, density, and thermal expansion are added with separate keywords. This makes the material definition modular and keeps the physical meaning of each property explicit.

Material names must be unique. Sections reference the material by name, so one definition can be reused throughout the model.

Materials are independent of Parts. The corresponding Section assignment connects a material to a particular mesh region.

Parameters

Parameter	Status	Meaning
MATERIAL / NAME	required	Unique material identifier.

The material definition becomes the active target for ELASTIC and DENSITY. The assigned unit system must be consistent with all geometry, loads, and mass data.

FEMaster and Abaqus example

```
*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3
*DENSITY
7.85e-9
```

4.3.1.2 *ELASTIC

*ELASTIC defines a linear-elastic constitutive law. Native FEMaster supports isotropic elasticity, generalized isotropic elasticity with an independent shear modulus, orthotropic engineering constants, and an orthotropic stiffness-coefficient form. The supported Abaqus input covers the corresponding documented Abaqus forms.

Isotropic elasticity

For TYPE=ISOTROPIC or ISO, the data are Young's modulus E and Poisson's ratio ν. This is the default form. The shear modulus is

$$G = \frac{E}{2(1+\nu)},$$

and the three-dimensional isotropic constitutive law follows from E and ν in the usual way.

For ordinary stable isotropic elasticity, $E > 0$ and $-1 < \nu < 0.5$. Values very close to $\nu = 0.5$ describe nearly incompressible behaviour and may require element formulations and meshes that can represent that regime without excessive volumetric locking.

Generalized isotropic elasticity

Native FEMaster also accepts GENERALISEDISOTROPIC, GENERALISED_ISOTROPIC, or GENISO. This form reads E, ν, and an independent G. It preserves the isotropic-style normal relation while allowing the shear response to differ from the classical identity $G = E/[2(1+\nu)]$.

This is a FEMaster-specific convenience for constitutive models with a physically independent shear modulus. It does not compensate for an inappropriate element formulation or mesh.

Orthotropic engineering constants

TYPE=ENGINEERINGCONSTANTS defines the nine constants

$$E_1, E_2, E_3, \nu_{12}, \nu_{13}, \nu_{23}, G_{12}, G_{13}, G_{23}.$$

These quantities are expressed in the material coordinate system. A Section orientation therefore determines which physical directions of the model correspond to material axes 1, 2, and 3.

The Poisson ratios are directional. For example, ν_{12} describes strain in direction 2 caused by stress in direction 1. The reciprocal values follow from elastic symmetry, e.g.

$$\frac{\nu_{12}}{E_1} = \frac{\nu_{21}}{E_2}.$$

Transferred orthotropic data are compatible only when the axis ordering and Poisson-ratio convention agree with this definition.

Orthotropic stiffness coefficients

TYPE=ORTHOTROPIC or ORTHO accepts the nine coefficients

$$D_{1111}, D_{1122}, D_{2222}, D_{1133}, D_{2233}, D_{3333}, D_{1212}, D_{1313}, D_{2323}.$$

The first six values form the symmetric normal-stiffness block

$$\begin{bmatrix} D_{1111} & D_{1122} & D_{1133} \\ D_{1122} & D_{2222} & D_{2233} \\ D_{1133} & D_{2233} & D_{3333} \end{bmatrix},$$

while D_{1212}, D_{1313}, and D_{2323} are the three shear stiffnesses. The normal block must be nonsingular. This form accepts constitutive stiffness coefficients directly rather than through engineering constants.

Elastic constants follow the unit system of the model. If stresses are measured in MPa, the elastic moduli and stiffness coefficients are also entered in MPa. FEMaster does not attach units to numerical values, so dimensional consistency is determined by the complete set of input quantities.

Parameters

Parameter	Status	Meaning
`TYPE`	optional	Elasticity form. Native default `ISOTROPIC`; supported forms and aliases are described above.

Data lines

Entry	Meaning
`E, nu`	Isotropic Young's modulus and Poisson ratio.
`E, nu, G`	Native generalized-isotropic form.
Engineering constants	$E_1, E_2, E_3, \nu_{12}, \nu_{13}, \nu_{23}, G_{12}, G_{13}, G_{23}$.
Orthotropic stiffness	$D_{1111}, D_{1122}, D_{2222}, D_{1133}, D_{2233}, D_{3333}, D_{1212}, D_{1313}, D_{2323}$.

Isotropic material example

The first block defines an isotropic steel with Young's modulus and Poisson ratio. The second block supplies nine directional engineering constants for an orthotropic material; its section must use the intended material axes.

```
*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3
```

Orthotropic material example

```
*MATERIAL, NAME=COMPOSITE
*ELASTIC, TYPE=ENGINEERING CONSTANTS
135000., 10000., 10000., 0.30, 0.30, 0.40, 5000., 5000., 3500.
```

4.3.1.3 *PLASTIC

*PLASTIC adds associative isotropic von Mises (J_2) elastoplasticity with tabulated isotropic hardening to the current material. The command is registered in *both* the native FEMaster and supported Abaqus parsers. It requires an isotropic *ELASTIC definition for the same material; orthotropic, generalized-isotropic, and hyperelastic definitions cannot be combined with the current *PLASTIC implementation. The material keeps the Young's modulus and Poisson ratio defined by *ELASTIC.

Each data row gives the current uniaxial *true (Cauchy) yield stress* followed by the *accumulated equivalent plastic strain*, not total strain. This is the same stress meaning used by Abaqus-style *PLASTIC input. The first row must have zero equivalent plastic strain. Later strain values must increase strictly, while yield stresses must be positive and non-decreasing. FEMaster interpolates the yield stress linearly between rows and holds it constant beyond the last row (perfect plasticity after the supplied range). A single row describes a perfectly plastic material from initial yield.

TYPE is optional and defaults to J2; accepted selectors are J2, MISES, VONMISES, and VON_MISES. The material implementation provides small-strain and finite-strain constitutive responses for the supported axial, solid, and integrated-shell evaluation paths. A nonlinear analysis is required to evolve plastic history; a linear static analysis does not turn into an elastoplastic analysis merely because this keyword is present.

For finite strain, the return map uses Mandel/Kirchhoff equivalent stress internally. Because J2 plastic flow is isochoric, the tabulated Cauchy yield radius is converted consistently as $\tau_y = J\sigma_y$, where $J = \det \mathbf{F}$. This conversion and its deformation derivative are included in the finite-strain algorithmic tangent.

When converting uniaxial test data to this table, use true stress and subtract the elastic strain from the corresponding logarithmic total strain before specifying the plastic-strain ordinate. Stress values and Young's modulus must use the same stress unit.

Parameters

Parameter	Status	Meaning
TYPE	optional	J2 (default), MISES, VONMISES, or VON_MISES.

Data lines

Entry	Meaning
true (Cauchy) yield stress, equivalent plastic strain	One hardening point per row; the first plastic strain is zero and subsequent strains are strictly increasing.

The table specifies true yield stress against equivalent plastic strain, with the first plastic strain equal to zero. The second row introduces hardening beyond initial yield.

FEMaster and Abaqus example

```
*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3
*PLASTIC
250., 0.
300., 0.02
350., 0.05
```

4.3.1.4 *HYPERELASTIC

*HYPERELASTIC defines the currently supported finite-strain Neo-Hooke material law. Native FEMaster accepts the Neo-Hooke selector through the documented spellings NEOHOOKE, NEO_HOOKE, and NEO-HOOKE. The supported Abaqus input uses the corresponding NEO HOOKE form.

The material is parameterized by C_{10} and D_1. C_{10} controls the deviatoric stiffness and D_1 controls compressibility. These parameters are distinct from Young's modulus and Poisson's ratio and do not accept E and ν as direct substitutes.

For the conventional compressible Neo-Hooke parameterization, the small-strain shear modulus is related to C_{10} by

$$\mu = 2C_{10}.$$

The precise relation between D_1 and bulk stiffness follows the Neo-Hooke energy convention used by the model. Converted test data or material cards are compatible only when they use the same parameter convention rather than merely matching parameter names.

Hyperelasticity is a finite-strain constitutive model and requires a nonlinear analysis and an element formulation capable of evaluating the finite-deformation response. Defining a hyperelastic material does not by itself make a linear static loadcase nonlinear.

For nearly incompressible rubber-like materials, the ratio between bulk and shear stiffness can become very large. Element formulation, mesh quality, nonlinear convergence, and contact penalty stiffness are then sensitive to the volumetric stiffness.

Parameters

Parameter	Status	Meaning
Neo-Hooke selector	required	Selects the supported Neo-Hooke constitutive model.

Data lines

Entry	Meaning
C10	Deviatoric Neo-Hooke coefficient.
D1	Compressibility coefficient.

The supported Neo-Hooke model is intended for finite-strain elastic behavior. Its constants must be expressed in the same mechanical unit system as the remainder of the model.

FEMaster and Abaqus example

```
*MATERIAL, NAME=RUBBER
*HYPERELASTIC, NEO HOOKE
0.348, 5.0e-6
```

4.3.1.5 *DENSITY

*DENSITY assigns mass density ρ to the current material. The keyword is shared by native FEMaster and the supported Abaqus input format.

Density contributes wherever the model requires mass. For an element occupying volume V, the total distributed mass scales with

$$m = \int_V \rho \, \mathrm{d}V.$$

The corresponding element mass matrix enters eigenfrequency and transient analyses, and density also affects gravity/body loads, inertial loading, and inertia-relief calculations where those features are used.

Density must be consistent with the chosen length, force, mass, and time units. A common source of dynamic errors is to enter geometry in millimetres and elastic modulus in MPa while leaving density in kg/m^3 without conversion. Such a model may have a plausible static stiffness but eigenfrequencies that are wrong by orders of magnitude.

A purely stiffness-based linear static analysis can omit density. Eigenfrequency, transient, inertia-relief, and mass-dependent loadcases require mass data from density, explicitly modelled concentrated masses, or both.

Data lines

Entry	Meaning
rho	Scalar mass density.

For eigenfrequency, inertia, and transient analyses, this value enters the mass matrix. A static stress analysis without inertial loading need not use mass density.

FEMaster and Abaqus example

```
*MATERIAL, NAME=STEEL
*DENSITY
7.85e-9
```

4.3.1.6 *EXPANSION

*EXPANSION defines one constant isotropic coefficient of thermal expansion α in both FEMaster and Abaqus input. The coefficient describes the free thermal strain produced by a temperature change. In the isotropic case,

$$\boldsymbol{\varepsilon}_{\mathrm{th}} = \alpha\,\Delta T\begin{bmatrix}1 & 1 & 1 & 0 & 0 & 0\end{bmatrix}^{T}.$$

The thermal strain uses $\Delta T = T - T_{\mathrm{zero}}$, where T_{zero} is set by ZERO. A nodal temperature Field can be assigned to the model with *INITIALCONDITION, TYPE=TEMPERATURE.

The coefficient has units of inverse temperature, for example K^{-1}. Temperature differences in kelvin and degrees Celsius are numerically identical, so either scale may be used for ΔT as long as the coefficient is consistent.

The supported property is scalar and isotropic. Direction-dependent or tabulated temperature-dependent expansion is not described by this keyword. TYPE is optional and accepts ISO or ISOTROPIC; the default is ISO.

Thermal stress arises when free thermal expansion is restrained or when temperature varies spatially. A uniformly heated completely free body expands without developing stress in an ordinary thermoelastic model.

Parameters

Parameter	Status	Meaning
TYPE	optional	ISO or ISOTROPIC; default ISO.
ZERO	optional	Stress-free reference temperature; default 0.

Data lines

Entry	Meaning
alpha	Constant isotropic coefficient of thermal expansion.

The coefficient describes free thermal strain per unit change in temperature. The older THERMALEXPANSION spelling shown below selects the same underlying property.

FEMaster and Abaqus example

```
*MATERIAL, NAME=ALUMINIUM
*EXPANSION, TYPE=ISO
2.30e-5
```

Assigning a temperature state

For a previously defined two-node model, the following Field specifies a spatial temperature distribution. *INITIALCONDITION assigns it to the model, and the material's thermal expansion is evaluated relative to its ZERO temperature.

```
*FIELD, NAME=TEMP_STATE, TYPE=NODE,
 COLS=1, FILL=ZERO
1, 20.
2, 70.
*INITIALCONDITION, TYPE=TEMPERATURE,
 FIELD=TEMP_STATE
```

Legacy keyword

```
*MATERIAL, NAME=ALUMINIUM
*THERMALEXPANSION
2.30e-5
```

4.3.1.7 *THERMALEXPANSION

*THERMALEXPANSION is a legacy alias for *EXPANSION. Both keywords accept the same constant isotropic coefficient, optional TYPE selector, and stress-free reference temperature ZERO. For new decks, prefer *EXPANSION; both forms remain usable within the unified input language.

Parameters

Parameter	Status	Meaning
TYPE	optional	ISO or ISOTROPIC; default ISO.
ZERO	optional	Stress-free reference temperature; default 0.

Data lines

Entry	Meaning
alpha	Constant isotropic coefficient of thermal expansion.

This legacy keyword accepts the same value as EXPANSION. New input decks should use the latter, while older files remain readable.

FEMaster and Abaqus example

```
*MATERIAL, NAME=ALUMINIUM
*THERMALEXPANSION
2.30e-5
```

Recommended keyword

```
*MATERIAL, NAME=ALUMINIUM
*EXPANSION
2.30e-5
```

4.3.2 Sections and Element Properties

Element connectivity defines interpolation and kinematics, but it does not by itself provide a complete structural model. Sections connect element regions to material and geometric properties. For solids the Section selects a material and optionally a material orientation. Shells additionally require thickness or a generalized shell stiffness. Trusses require cross-sectional area. Beams combine material data with cross-section constants and a local section direction.

A Section defined for an element set inside a Part belongs to that Part. If the Part is instantiated several times, every Instance carries the same Section assignment. Material, thickness, area, and profile data thereby remain properties of the physical component, while the Instance placement determines where that component is located.

4.3.2.1 *PROFILE

*PROFILE defines the cross-section constants used by native FEMaster beam Sections. A Profile is referenced by name from *BEAM SECTION. It stores area, bending second moments, torsional constant, optional product of inertia, shear-centre offsets, reference-line offsets, and optional effective transverse-shear areas.

The data order is

$$A,\ I_y,\ I_z,\ J_t,\ I_{yz},\ e_y,\ e_z,\ \mathrm{ref}_y,\ \mathrm{ref}_z,\ A_{sy},\ A_{sz}.$$

Only the first four values are required. Product of inertia and offsets default to zero. If A_{sy} or A_{sz} is omitted or non-positive, FEMaster uses $5A/6$. The shear areas are used by shear-flexible beam formulations such as B31; B33 does not use them.

FEMaster uses the convention

$$I_{yz} = \int_A yz \, \mathrm{d}A$$

without a leading minus sign. Some texts and tools define the product of inertia with the opposite sign, so imported Section properties can differ by this convention.

The offsets e_y, e_z describe the relative position used for the section/shear-centre definition, while $\mathrm{ref}_y, \mathrm{ref}_z$ describe the location of the beam reference line relative to the section reference point. In unsymmetric or eccentric Sections these offsets determine the different points about which axial force, shear, bending, and torsion act.

Profile constants follow the unit system of the model. In a consistent N–mm model, A, A_{sy}, and A_{sz} are in mm^2, I_y, I_z, J_t, and I_{yz} are in mm^4, and all offsets are in mm.

Parameters

Parameter	Status	Meaning
PROFILE / NAME	required	Unique Profile name referenced by *BEAM SECTION.

Data lines

Entry	Meaning
A	Cross-sectional area.
Iy, Iz	Second moments of area about the local section axes.
Jt	Torsional constant.
Iyz	Product of inertia using $I_{yz} = \int_A yz \, \mathrm{d}A$.
ey, ez	Optional section/shear-centre offsets; default zero.
refy, refz	Optional reference-line offsets; default zero.
Asy, Asz	Optional effective transverse-shear areas; default $5A/6$.

The profile supplies section constants that a BEAM SECTION can reference by name. It does not by itself assign a section to any element.

FEMaster and Abaqus example

```
*PROFILE, NAME=RECT_EQUIV
200., 6666.67, 1666.67, 4577.,
0., 0., 0., 0., 0., 166.667, 166.667.
```

4.3.2.2 *SOLID SECTION

*SOLID SECTION assigns material properties to a solid element region. Native FEMaster requires ELSET and MATERIAL; MAT is accepted as an alias. ORIENTATION is optional and defines the local basis in which directional material properties are interpreted.

No thickness or area is required because a solid element represents its physical volume directly through the mesh geometry. The Section determines the constitutive law of the elements and the material coordinate system in which an anisotropic law is evaluated.

For isotropic elasticity, changing the material orientation has no effect on the constitutive response because the stiffness is direction independent. For orthotropic or fully anisotropic material data, the orientation is physically significant and must be chosen consistently with the material axes.

*SOLID SECTION also accepts a positive area data line when its ELSET contains only truss elements, including T3 and T3D2. This special case is equivalent to an explicit *TRUSS SECTION; for an ordinary solid region no area data line is permitted.

A Section applies one material system and orientation to its target element set. Different material systems or orientations within one geometric component therefore require separate named element sets and Section assignments. A target set containing incompatible element families does not define a uniform Section interpretation.

Parameters

Parameter	Status	Meaning
ELSET	required	Solid element set receiving the Section.
MATERIAL / MAT	required	Material assigned to the region.
ORIENTATION	optional	Named material coordinate system.

Solid element section

The first form assigns material and, where needed, material orientation to a continuum-element set. For a pure truss ELSET the same keyword takes an area data line instead.

```
*SOLID SECTION, ELSET=SOLID,
 MATERIAL=STEEL,
 ORIENTATION=MAT_AXES
```

Truss cross-sectional area

```
*SOLID SECTION, ELSET=BARS, MATERIAL=STEEL
100.
```

Explicit truss section

```
*TRUSS SECTION, ELSET=BARS, MATERIAL=STEEL
100.
```

4.3.2.3 *SHELL SECTION

*SHELL SECTION assigns thickness and constitutive properties to shell elements. Native FEMaster supports two forms:

- TYPE=INTEGRATED, where a material law is integrated through the physical shell thickness;
- TYPE=ABD, where the generalized membrane–bending and transverse-shear stiffnesses are supplied directly.

For TYPE=INTEGRATED, ELSET identifies the shell region and MATERIAL identifies the constitutive law. The data line supplies the physical shell thickness t. The shell uses this thickness to form membrane, bending, shear, mass, and stress-recovery quantities according to the shell formulation.

For a homogeneous linear elastic shell, the familiar generalized relation has the form

$$\begin{bmatrix} \boldsymbol{N} \\ \boldsymbol{M} \end{bmatrix} = \begin{bmatrix} A & B \\ B & D \end{bmatrix} \begin{bmatrix} \boldsymbol{\varepsilon}_0 \\ \boldsymbol{\kappa} \end{bmatrix},$$

with additional transverse-shear resultants governed by a 2×2 shear stiffness. In an integrated Section these generalized stiffnesses arise from the material and thickness. In an ABD Section they are entered directly.

For TYPE=ABD, FEMaster reads 40 numerical values: all 36 entries of the full 6×6 generalized stiffness matrix in row-major order, followed by the four entries of the 2×2 transverse-shear stiffness matrix. The matrix is not assumed to be symmetric from input; enter all values explicitly in the required order.

ORIENTATION selects the local Section/material coordinate system. The ABD matrix is interpreted in this local shell basis. This is critical for laminates and anisotropic panels: an ABD matrix entered in one material direction but assigned with another orientation describes a different structure. CSYSAXIS=1,2,3 selects which axis of the named coordinate system establishes the in-plane reference direction; the default is axis 1.

The supported Abaqus *SHELL SECTION form represents a homogeneous integrated Section. Its data line gives thickness and optionally the number of Simpson integration points. FEMaster currently supports five through-thickness points for this imported form; if the count is omitted, five are used. Layered composite layups and other Abaqus-specific shell-section options are not implied by this syntax.

The shell normal and in-plane Section direction together define the local shell basis in which N_{xx}, N_{yy}, M_{xx}, M_{yy}, top/bottom stresses, and an anisotropic ABD matrix are interpreted. On curved meshes both directions vary with the surface geometry. A 90-degree orientation difference can exchange strong and weak laminate directions while leaving the input syntactically valid.

Parameters

Parameter	Status	Meaning
TYPE	native optional	INTEGRATED (default) or ABD.
ELSET	required	Shell element set.
MATERIAL / MAT	integrated required	Material used by the integrated Section; optional for native ABD input.
THICKNESS	native optional	Nominal thickness parameter; default 1.0.
ORIENTATION	optional	Named Section/material coordinate system.
CSYSAXIS	native optional	Coordinate-system axis 1, 2, or 3 used for the in-plane reference direction; default 1.

Data lines

Entry	Meaning
Integrated FEMaster	One positive shell thickness.
ABD FEMaster	36 row-major generalized-stiffness entries followed by 4 row-major transverse-shear entries.
Abaqus homogeneous form	Thickness and optional integration-point count; the current supported count is 5.

Integrated shell section

An integrated shell section uses a material and thickness; a directly specified ABD section instead supplies membrane, bending, coupling, and transverse-shear stiffnesses.

```
*SHELL SECTION, ELSET=PANEL,
 MATERIAL=ALUMINIUM
2.0, 5
```

Direct ABD stiffness

```
*SHELL SECTION, ELSET=PANEL, TYPE=ABD
1000000., 0., 0., 0., 0., 0., 0., 1000000., 0., 0., 0., 0., 0., 0., 400000., 0., 0., 0., 0.,
    0., 0., 5000., 0., 0., 0., 0., 0., 0., 5000., 0., 0., 0., 0., 0., 0., 2000., 100000., 0.,
     0., 100000.
```

The data line contains 40 values: the complete 6×6 generalized membrane–bending stiffness matrix in row-major order, followed by the complete 2×2 transverse-shear stiffness matrix.

4.3.2.4 *BEAM SECTION

*BEAM SECTION and *BEAM GENERAL SECTION both create the same generalized linear BeamSection in FEMaster. Unlike Abaqus, FEMaster does not yet integrate nonlinear constitutive behavior through a beam cross-section. Both input keywords therefore use the same parsing and section-property code path.

There are three mutually exclusive section descriptions:

- PROFILE=name references a previously declared native FEMaster *PROFILE; a named MATERIAL is required.
- SECTION=GENERAL (or an omitted SECTION without PROFILE) reads $(A, I_{11}, I_{12}, I_{22}, J)$ directly on the first data line.
- SECTION=RECT, CIRC, PIPE, BOX, I, CHANNEL, or L reads a geometric data line and converts it to generalized section constants.

For geometric types, use Abaqus dimension order: RECT: a, b; CIRC: radius; PIPE: outer radius, wall thickness; BOX: a, b, t for uniform walls or a, b, t_1, t_2, t_3, t_4; I: $l, h, b_1, b_2, t_1, t_2, t_3$, with a T-section represented by $b_1 = t_1 = 0$ or $b_2 = t_2 = 0$; CHANNEL: $l, h, b_1, b_2, t_1, t_2, t_3, o$; L: a, b, t_1, t_2. Shapes not listed here, including ARBITRARY, HEX, HAT, and TRAPEZOID, are not yet supported.

For GENERAL and geometric input, the first data line is followed by an optional (n_{1x}, n_{1y}, n_{1z}) direction (default $0, 0, -1$). When MATERIAL is omitted, provide a following data line with E, G, α; α is optional. The solver currently represents these direct constants using an equivalent isotropic material, so they must imply a valid Poisson ratio.

A DENSITY override may be specified without changing a shared named material. If PROFILE is used, the only data line is the optional section direction.

For closed circular sections, torsion is calculated from the polar second moment. For solid rectangles and thin-walled open/closed library shapes, torsion uses engineering approximations. The currently generic transverse shear areas remain $5A/6$ and nontrivial shear-center corrections are not derived. Consequently, these inputs are linear generalized-section approximations, not a claim of full Abaqus library-section equivalence.

The nonzero orientation vector must not be parallel to the beam tangent. It defines the local cross-section basis in which section inertias and offsets are interpreted.

Parameters

Parameter	Status	Meaning
ELSET	required	Target beam element set.
MATERIAL / MAT	optional	Named material; required with PROFILE.
PROFILE	optional	Previously defined native Profile; mutually exclusive with SECTION.
SECTION	optional	GENERAL (default without PROFILE), or one of the supported library shapes.
DENSITY	optional	Section-local nonnegative density override.

Geometric beam section

This beam section defines the geometry of a rectangular profile and its local reference direction. A separately defined PROFILE can be used when the section constants are already known.

```
*BEAM SECTION, ELSET=BEAMS, MATERIAL=STEEL, SECTION=RECT
10., 20.
0., 1., 0.
```

The keyword `*BEAM GENERAL SECTION` is also accepted with the same `SECTION` geometry and data lines; only the keyword spelling changes. No separate definition is required.

Predefined profile

```
*PROFILE, NAME=BEAM_PROFILE
200., 6666.7, 1666.7, 4577.
*BEAM SECTION, ELSET=BEAMS, MATERIAL=STEEL, PROFILE=BEAM_PROFILE
0., 1., 0.
```

4.3.2.5 *TRUSS SECTION

Native *TRUSS SECTION assigns a material and a positive cross-sectional area to a truss element set. A truss carries only axial force, so cross-sectional area is the only geometric Section property required by the current formulation.

For an element of length L, area A, and Young's modulus E, the small-strain axial stiffness scales as

$$k = \frac{EA}{L}.$$

The same area also determines material mass per unit length through ρA and converts axial force to average axial stress through $\sigma = N/A$.

The area must therefore use the same unit system as the geometry and material constants and must be strictly positive.

*TRUSS SECTION is the explicit FEMaster form for assigning a truss material and area. Alternatively, *SOLID SECTION accepts an area data line when the ELSET consists exclusively of truss elements (T3 or T3D2). The resulting truss-section properties are the same. Ordinary solid elements, in contrast, use *SOLID SECTION without an area line.

Parameters

Parameter	Status	Meaning
MATERIAL / MAT	required	Truss material.
ELSET	required	Truss element set.

Data lines

Entry	Meaning
area	Strictly positive cross-sectional area.

A truss carries axial force only. The area given on the data line is the area used for axial stiffness and mass; SOLID SECTION can assign the same area to an exclusively truss-based set.

FEMaster and Abaqus example

```
*TRUSS SECTION, ELSET=BARS,
 MATERIAL=STEEL
100.0
```

Solid section for truss elements

```
*SOLID SECTION, ELSET=BARS,
 MATERIAL=STEEL
100.0
```

4.3.2.6 *POINTMASS

*POINTMASS is the native NSET-based concentrated-property command. It is evaluated on the compiled model and creates one auxiliary point element for every node in the target NSET. All point elements created by one command share the supplied mass, rotary inertia, and optional ground-spring properties.

The first value is one translational point mass m. It contributes equally to the three translational inertia directions. The next three values are rotary inertias

$$(I_x, I_y, I_z)$$

associated with the nodal rotational degrees of freedom. Translational and rotational spring constants may then be supplied. Missing values default to zero, so a pure lumped mass requires only the first scalar.

For one target node, the diagonal generalized contributions are

$$M_p = \mathrm{diag}(m, m, m, I_x, I_y, I_z),$$

and

$$K_p = \mathrm{diag}(k_x, k_y, k_z, k_{rx}, k_{ry}, k_{rz}).$$

All coefficients must be non-negative.

The point element activates only generalized components that carry a nonzero physical coefficient. A nonzero translational mass activates U_x, U_y, U_z; a nonzero rotary inertia activates its corresponding rotational component; translational and rotational springs likewise activate their associated directions. A point element with all coefficients zero is mechanically inactive.

If the target set contains several nodes, the complete property is applied independently at every target node. The specified mass is therefore mass *per node*, not the total mass of the NSET. A total payload distributed over four target nodes requires four appropriately chosen nodal masses whose sum equals the intended total.

The auxiliary point elements created by *POINTMASS are deliberately not inserted into the public compiled element array or any ELSET. This preserves ordinary element numbering and element-domain Field offsets. They nevertheless participate in active-DOF enumeration, global stiffness assembly, global mass assembly, eigenfrequency and transient analyses, and the dedicated point-mass-aware inertia paths.

This distinction matters for loads. Native *VLOAD and supported Abaqus *DLOAD, GRAV select ordinary element regions and therefore do not select auxiliary *POINTMASS objects. *VLOAD is a force-per-volume load and, unlike density-scaled *DLOAD, GRAV, makes no concentrated contribution to a zero-volume regular MASS point element. Native *INERTIALOAD includes them only when CONSIDER_POINT_MASSES=1. Likewise, native *INERTIARELIEF includes them in total mass, center of gravity, and rotary inertia only when its CONSIDER_POINT_MASSES option is enabled.

Because the auxiliary point elements activate their own DOFs, a model may consist entirely of nodes plus *POINTMASS definitions without requiring an unrelated structural element merely to activate translational or rotational components. Node-only models can also be written to FRD; the writer omits the element-connectivity block when no regular output element topology exists.

Point masses represent equipment, payloads, actuators, engines, sensors, or other concentrated inertia without explicitly meshing the attached body. Translational stiffness has force/length units, rotational stiffness has moment/radian units, and rotary inertia has mass-length^2 units in the chosen consistent system.

Parameters

Parameter	Status	Meaning
NSET	required	Target compiled node set.

Data lines

Entry	Meaning
`mass`	Concentrated translational mass; default 0.
`Ix, Iy, Iz`	Rotary inertia components; default 0.
`kx, ky, kz`	Translational ground-spring constants; default 0.
`krx, kry, krz`	Rotational ground-spring constants; default 0.

The command creates one auxiliary point element at every node of PAYLOAD. The listed coefficients are assigned per node, not divided by the number of nodes in the set.

FEMaster and Abaqus example

```
*POINTMASS, NSET=PAYLOAD
0.025,
1.2e-3, 1.2e-3, 2.0e-3,
1000., 1000., 0.,
0., 0., 0.
```

For a translational mass alone, a regular `MASS` element and its `*MASS` property assignment can be used instead. The additional rotary inertias and ground springs in the example above are not represented by this replacement.

Regular mass element

```
*ELEMENT, TYPE=MASS, ELSET=PAYLOAD_MASS
100, 2
*MASS, ELSET=PAYLOAD_MASS
0.025
```

4.3.2.7 *MASS

*MASS assigns an isotropic concentrated translational mass to one-node MASS elements. One scalar value m is applied independently to every point element in the target ELSET and produces

$$M_e = \mathrm{diag}(m, m, m, 0, 0, 0).$$

A positive mass activates the three translational nodal directions. Several MASS elements may reference the same node; their assembled contributions are additive.

The supported mapping is the isotropic form only. Abaqus anisotropic mass, orientation-dependent mass, and damping parameters are outside the current subset. Native FEMaster also retains *POINTMASS as the legacy NSET-based variant when several diagonal point properties should be assigned together.

Parameters

Parameter	Status	Meaning
ELSET	required	Nonempty element set containing point elements.
TYPE	optional	ISOTROPIC; currently the only supported form and the default.

Data lines

Entry	Meaning
mass	Non-negative isotropic translational mass assigned to every target element.

An explicit MASS element is referenced through its element set and given a translational mass. The POINTMASS alternative avoids a separate regular element definition.

FEMaster and Abaqus example

```
*ELEMENT, TYPE=MASS, ELSET=PAYLOAD_MASS
100, 2
*MASS, ELSET=PAYLOAD_MASS
0.01
```

POINTMASS assigned directly to a node set

```
*POINTMASS, NSET=PAYLOAD_NODES
0.01
```

4.3.2.8 *ROTARY INERTIA

*ROTARY INERTIA assigns concentrated rotary inertia to one-node ROTARYI elements. The first three data values are the diagonal moments I_{11}, I_{22}, and I_{33}, which map directly to the three rotational nodal directions:

$$M_e = \mathrm{diag}(0, 0, 0, I_{11}, I_{22}, I_{33}).$$

The Abaqus data format additionally contains the products of inertia I_{12}, I_{13}, and I_{23}. FEMaster parses these fields for input compatibility but currently requires all three to be zero because PointMassSection stores only diagonal rotary inertia. Missing off-diagonal fields are treated as zero. The ORIENTATION form is not supported by this mapping, so the three supported inertia directions are the point element's global nodal rotational directions. Native FEMaster also retains *POINTMASS as the legacy NSET-based variant for diagonal rotary inertia.

Parameters

Parameter	Status	Meaning
ELSET	required	Nonempty element set containing ROTARYI point elements.

Data lines

Entry	Meaning
I11, I22, I33	Non-negative rotary inertias about the three supported axes.
I12, I13, I23	Optional Abaqus tensor fields; currently required to be zero.

The listed diagonal inertias act about the nodal rotational axes. They do not supply translational mass unless combined with a MASS element or POINTMASS values.

FEMaster and Abaqus example

```
*ELEMENT, TYPE=ROTARYI, ELSET=PAYLOAD_INERTIA
101, 2
*ROTARY INERTIA, ELSET=PAYLOAD_INERTIA
1.2e-3, 1.2e-3, 2.0e-3, 0., 0., 0.
```

POINTMASS with diagonal rotary inertia

```
*POINTMASS, NSET=PAYLOAD_NODES
0.,
1.2e-3, 1.2e-3, 2.0e-3
```

4.3.2.9 *SPRING

*SPRING assigns a constant linear ground stiffness to one-node SPRING1 elements. The first data line selects one nodal degree of freedom and the second data line gives the corresponding scalar stiffness. Degrees of freedom 1–3 create translational springs; degrees of freedom 4–6 create rotational springs:

DOF	1	2	3	4	5	6
coefficient	k_x	k_y	k_z	k_{rx}	k_{ry}	k_{rz}

The current mapping supports only real, constant, non-negative linear stiffness in the global nodal system. Abaqus SPRING2 and SPRINGA elements, local ORIENTATION, nonlinear force–displacement data, frequency or field dependence, and complex stiffness are outside the supported subset. Native FEMaster also retains *POINTMASS as the legacy NSET-based variant for diagonal translational and rotational ground springs.

Parameters

Parameter	Status	Meaning
ELSET	required	Nonempty element set containing SPRING1 point elements.

Data lines

Entry	Meaning
dof	Integer nodal degree of freedom from 1 through 6.
stiffness	Non-negative constant linear spring stiffness.

The two SPRING1 elements act in different generalized directions at the same node. POINTMASS can express several diagonal ground-spring components in one nodal property.

FEMaster and Abaqus example

```
*ELEMENT, TYPE=SPRING1, ELSET=SPRING_X
102, 2
*SPRING, ELSET=SPRING_X
1
1000.0

*ELEMENT, TYPE=SPRING1, ELSET=ROTSPRING_Z
103, 2
*SPRING, ELSET=ROTSPRING_Z
6
25.0
```

POINTMASS with translational and rotational ground springs

```
*POINTMASS, NSET=SUPPORT_NODES
0.,
0., 0., 0.,
1000., 0., 0.,
0., 0., 25.
```

4.3.3 Coordinate Systems and Orientations

Coordinate systems define local directions used by materials, Sections, loads, supports, and nodal transformations. They determine the interpretation of anisotropy, beam or shell local axes, cylindrical loading, and boundary-condition components expressed outside the global Cartesian basis.

`*ORIENTATION` and `*TRANSFORM` act on different concepts. `*ORIENTATION` defines a named coordinate system that can be referenced by Sections and other orientation-aware native FEMaster features. Abaqus `*TRANSFORM` assigns a local degree-of-freedom basis to a node set. Neither command moves the geometry; rigid placement of a Part is defined by `*INSTANCE`.

4.3.3.1 *ORIENTATION

Native *ORIENTATION creates a named rectangular or cylindrical coordinate system. NAME identifies the system and TYPE selects RECTANGULAR or CYLINDRICAL.

A rectangular coordinate system describes a fixed orthogonal basis. FEMaster accepts one, two, or three supplied direction vectors, corresponding to three, six, or nine scalar values. In most user-written models, two non-collinear directions are the clearest representation: the first vector defines the first local axis and the second vector defines the plane in which the second local axis lies. The final orthogonal triad is then determined from those directions.

For example, the vectors

$$\boldsymbol{a} = (1, 0, 0), \qquad \boldsymbol{b} = (0, 1, 0)$$

define a local basis aligned with the global Cartesian axes. Rotating these vectors rotates the local material or Section basis without changing any node coordinates.

A cylindrical coordinate system defines an axis and corresponding radial/circumferential directions. Unlike a rectangular system, the radial and circumferential directions depend on the position at which the system is evaluated. This represents axisymmetric-like components, cylindrical material directions, and directional quantities that follow a curved geometry.

The supported Abaqus *ORIENTATION form uses DEFINITION=COORDINATES and SYSTEM=RECTANGULAR. Its data contain points a, b, and optionally c. The vector $a - c$ defines the first direction and $b - c$ defines the second in-plane direction. If c is omitted, the global origin is used. The points must define nonzero, non-collinear directions.

A material orientation changes the basis in which directional material constants are interpreted. For orthotropic elasticity, local axes 1, 2, and 3 correspond to the directions associated with E_1, E_2, E_3, G_{12}, G_{13}, G_{23}, and the Poisson-ratio convention of the material definition.

For shell Sections, the selected coordinate-system axis is projected into the shell plane to establish the local in-plane material/Section direction. The shell normal completes the local shell basis. Both directions determine the interpretation of anisotropic stiffness and shell resultants on curved shells.

Parameters

Parameter	Status	Meaning
NAME	required	Coordinate-system name used by Sections and other orientation-aware features.
TYPE	native required	RECTANGULAR or CYLINDRICAL.
DEFINITION	native optional	Accepted legacy parameter.
DEFINITION	Abaqus optional	COORDINATES; default COORDINATES.
SYSTEM	Abaqus optional	RECTANGULAR; default RECTANGULAR.

Data lines

Entry	Meaning
Native rectangular	3, 6, or 9 scalar components defining one to three directions.
Native cylindrical	9 scalar components defining the cylindrical system geometry.
Abaqus rectangular	$a_1, a_2, a_3, b_1, b_2, b_3, c_1, c_2, c_3$; c defaults to the origin.

This example defines local material directions. The orientation can subsequently be selected by a section that needs directional constitutive properties. A nodal TRANSFORM instead changes the basis used for nodal DOFs.

FEMaster and Abaqus example

```
*ORIENTATION, NAME=MAT_AXES,
 DEFINITION=COORDINATES,
 SYSTEM=RECTANGULAR
1., 0., 0., 0., 1., 0., 0., 0., 0.
```

Compact rectangular orientation

```
*ORIENTATION, NAME=MAT_AXES,
 TYPE=RECTANGULAR
1., 0., 0.,
0., 1., 0.
```

4.3.3.2 *TRANSFORM

*TRANSFORM is supported in Abaqus input and assigns a local degree-of-freedom basis to a node set. The nodal basis differs from a named *ORIENTATION: local support and load directions may also be specified by referencing named coordinate systems.

NSET identifies the nodes that use the transformation. TYPE=R selects a rectangular transform and is the default. The six data values define two directions or points that establish the local rectangular basis. The directions must be nonzero and non-collinear.

TYPE=C selects a cylindrical transform. The two three-component points a and b define the cylindrical axis through

$$\boldsymbol{e}_z \parallel \boldsymbol{b} - \boldsymbol{a}.$$

The points must therefore be distinct. At each transformed node, the local radial and circumferential directions follow from the node position relative to that axis.

The transform affects how nodal vector components are interpreted. For example, a *BOUNDARY or *CLOAD component applied to a transformed node is expressed in the node's local transformed basis rather than directly in the global Cartesian basis. The transform does not rotate or translate the mesh itself.

A node has one effective nodal basis. Overlapping transformed sets with incompatible coordinate systems make that basis ambiguous.

*TRANSFORM differs from *INSTANCE: *INSTANCE changes the actual placement of a Part, whereas *TRANSFORM leaves the geometry unchanged and only changes the local basis used to interpret generalized nodal components.

Parameters

Parameter	Status	Meaning
NSET	required	Node set receiving the local nodal transformation.
TYPE	optional	R for rectangular or C for cylindrical; default R.

Data lines

Entry	Meaning
a1,a2,a3,b1,b2,b3	Two directions/points defining the rectangular basis or cylindrical axis.

Rectangular transformation

The selected node set receives a local basis for nodal degrees of freedom. The two variants describe different coordinate systems rather than competing file formats.

```
*TRANSFORM, NSET=EDGE_NODES, TYPE=R
1., 0., 0., 0., 1., 0.
```

Cylindrical transformation

```
*TRANSFORM, NSET=CYLINDER_NODES, TYPE=C
0., 0., 0., 0., 0., 100.
```

4.3.4 Fields

Fields store spatially varying data that belong to the finite-element model. They are more general than sets: a set only answers whether an entity belongs to a region, whereas a Field stores one or more numerical values at every addressed location. Typical uses are topology variables, prescribed shell directors, initial conditions, and other data that vary from node to node, element to element, or within an element.

A Field has a *domain* and a fixed number of components. The domain determines what one row refers to. For ordinary nodal and elemental Fields the rows are addressed by the same user-defined node and element identifiers that are used elsewhere in the input deck. For element-local Fields, such as values at local nodes or integration points, an element is named first and the location inside that element is given by a zero-based local index.

User node and element identifiers are independent of these local indices. They are non-negative deck labels, may start at **0**, and do not have to be contiguous. For example, node IDs `0`, `10`, and `1000` are perfectly valid if they are unique in their scope. By contrast, the local node indices of a four-node element are `0`, `1`, `2`, and `3` because they refer to positions in that element's connectivity.

4.3.4.1 *FIELD

*FIELD creates and optionally populates a named Field. It is a native FEMaster command; the supported Abaqus input format currently has no generic equivalent. The command is accepted at model level and in the native *LOADCASE contexts documented by the consuming feature.

NAME identifies the Field. TYPE selects its domain, COLS specifies the number of numerical components per location, and FILL defines how entries are initialized before explicit data rows are applied. FILL=ZERO is the default. FILL=NAN initializes unspecified components with NaN so that missing data remain distinguishable from a physical zero.

The supported domains are:

- NODE: one row per addressed node;
- ELEMENT: one row per addressed element;
- ELEMENT_NODAL: values stored separately for the local nodes of an element;
- ELEMENT_IP: values stored at an element's integration points;
- ELEMENT_MP: values stored at material points associated with an integration point.

Aliases without underscores and the short forms IP and MP are accepted where documented. COLS must be positive and is currently limited to 64 components.

For NODE and ELEMENT, every data row begins with the node or element target followed by the Field components. A target may be an ordinary user ID or, for an assembled model, an instance-qualified reference such as INSTANCE.ID. Empty component entries leave the existing value unchanged. This permits sparse updates of a Field that was first initialized with FILL.

For ELEMENT_NODAL, every row starts with an element reference and a zero-based local node index. The local index refers to the position in that element's connectivity, not to the numerical ID of the connected node. Thus, for a four-node element with connectivity 10,20,30,40, local node 0 refers to node 10, local node 1 to node 20, and so on.

ELEMENT_IP uses the same principle for integration points. The local integration-point index is zero based and must exist for the selected element formulation. ELEMENT_MP additionally specifies a zero-based material-point index. This representation allows different element formulations to expose different numbers of local storage positions without inventing global IDs for integration or material points.

Field values may be ordinary floating-point numbers and may also use NAN, +NAN, -NAN, INF, +INF, and -INF. Non-finite values represent partially prescribed or diagnostic Fields only where the consuming feature defines corresponding handling.

User IDs and element-local indices have different meanings. User node and element IDs may include 0 and may be sparse. Element-local node, integration-point, and material-point indices are positional indices beginning at 0.

Parameters

Parameter	Status	Meaning
NAME	required	Unique Field name.
TYPE	required	NODE, ELEMENT, ELEMENT_NODAL, ELEMENT_IP, or ELEMENT_MP.
COLS	required	Number of components per location; must be positive and is currently limited to 64.
FILL	optional	Initial value mode ZERO or NAN; default ZERO.

Data lines

Entry	Meaning
Node/element Field	`target, v1, v2, ....`
Element-nodal Field	`element, local_node, v1, v2, ...`; `local_node` is zero based.
Integration-point Field	`element, local_ip, v1, v2, ...`; `local_ip` is zero based.
Material-point Field	`element, local_ip, local_mp, v1, v2, ...`; both local indices are zero based.

The first field stores one scalar value per element; the second stores a three-component vector for each element node. A consuming keyword selects the named field, for example TOPODENSITY or NORMAL.

FEMaster and Abaqus example

```
*FIELD, NAME=RHO, TYPE=ELEMENT,
 COLS=1, FILL=ZERO
0, 0.8
10, 1.0

*FIELD, NAME=DIRECTORS,
 TYPE=ELEMENT_NODAL,
 COLS=3, FILL=NAN
10, 0, 0., 0., 1.
10, 1, 0., 0., 1.
```

4.3.4.2 *INITIALCONDITION

*INITIALCONDITION assigns model-level initial nodal temperatures or generalized velocities. The spelling *INITIALCONDITIONS is equivalent. Both commands are processed after the geometry and named Fields are compiled.

TYPE=TEMPERATURE supplies one scalar per node; TYPE=VELOCITY supplies six components in the order $(v_x, v_y, v_z, \omega_x, \omega_y, \omega_z)$. With FIELD, an existing NODE Field with one or six components, respectively, becomes the initial state; no data lines follow the command. Without FIELD, the command allocates a zero-initialized nodal Field, and each row addresses one compiled node by ID or INSTANCE.ID. Omitted or empty component values retain their current values, so later rows may selectively update the same node. A transient analysis uses the model's initial velocity state by default; a loadcase-local *INITIALVELOCITY, FIELD=... selects the specified nodal Field for that transient analysis instead.

Parameters

Parameter	Status	Meaning
TYPE	required	TEMPERATURE or VELOCITY.
FIELD	optional	Name of an existing NODE Field with 1 or 6 components, matching TYPE.

Data lines

Entry	Meaning
Temperature	node, temperature for inline data.
Velocity	node, vx, vy, vz, wx, wy, wz for inline data.
Named Field	No data rows; the named Field supplies the values.

The model below uses an existing temperature Field as the initial thermal state. A material's *EXPANSION coefficient and reference temperature ZERO determine the associated free thermal strain.

FEMaster and Abaqus example

```
*FIELD, NAME=START_TEMP, TYPE=NODE,
 COLS=1, FILL=ZERO
1, 20.
2, 60.
*INITIALCONDITION, TYPE=TEMPERATURE,
 FIELD=START_TEMP
```

Initial nodal velocity

In a transient model, the second node begins with a downward velocity.

```
*INITIALCONDITIONS, TYPE=VELOCITY
1, 0., 0., 0., 0., 0., 0.
2, 0., 0., -2., 0., 0., 0.
```

4.3.4.3 *NORMAL

*NORMAL selects an existing three-component ELEMENT_NODAL Field as the source of shell element-nodal normals or directors. The referenced Field must already exist, must use TYPE=ELEMENT_NODAL, and must contain exactly three components.

This mechanism represents shell orientations that cannot be described by a single normal per geometric node. Because the data are element-nodal, two neighbouring shell elements may prescribe different director vectors at the same geometric node, as occurs at folds, discontinuous shell normals, imported midsurfaces, or other locations where a single averaged nodal direction is not defined.

The vectors supplied in the Field are used as prescribed shell-normal information. Missing entries may be left as NaN when the shell-normal workflow is intended to complete them from the element geometry. The resulting normal direction determines shell top/bottom interpretation, pressure direction, local Section directions, and the sign of bending-related result quantities.

The local node index is the zero-based position inside the element connectivity rather than the connected node's user ID. This distinction allows two shell elements sharing one geometric node to store different element-nodal directors at that location.

Parameters

Parameter	Status	Meaning
FIELD	required	Name of an existing three-component ELEMENT_NODAL Field.

The example defines a three-component normal at each element-local node of shell element 10, then assigns the field to the shell-normal workflow. The FIELD declaration must precede NORMAL.

FEMaster and Abaqus example

```
*FIELD, NAME=SHELL_NORMALS,
 TYPE=ELEMENT_NODAL, COLS=3, FILL=NAN
10, 0, 0., 0., 1.
10, 1, 0., 0., 1.
10, 2, 0., 0., 1.
10, 3, 0., 0., 1.

*NORMAL, FIELD=SHELL_NORMALS
```

4.4 Loads and Boundary Conditions

Loads and boundary conditions define how the finite-element model is forced and supported. FEMaster supports reusable named load and support collectors, step-local native load definitions, and persistent Abaqus-style condition histories. Named collectors are activated with `*LOADS` and `*SUPPORTS`. Unnamed native vector loads and native supports participate in the current analysis, while Abaqus-style `*BOUNDARY`, single-DOF `*CLOAD`, labeled `*DLOAD`, and `*DSLOAD` definitions update persistent conditions that are inherited by subsequent analyses.

For persistent direct conditions, `OP=MOD` (the default) replaces matching components or targets; `OP=NEW` retires the inherited entries in that condition family before applying the new ones. For step-local native definitions, the supplied values apply to the current analysis; `OP=NEW` also clears any inherited direct entries of that family. Named collector definitions are reusable between analyses. Thermal prescriptions maintain separate temperature, heat-flux, and convection histories.

Throughout this section, remember that a numerical zero is a real physical value. For a native `*SUPPORT`, `0` means that the corresponding displacement or rotation is constrained to zero, while an omitted or `NAN` component means that the degree of freedom remains unconstrained.

4.4.1 *AMPLITUDE

*AMPLITUDE defines a reusable scalar function that can scale a load as the analysis variable changes. In a transient analysis this variable is physical time. Both input readers accept up to four (t, a) pairs per data line. TYPE selects LINEAR, STEP, or NEAREST interpolation; the default is LINEAR.

For linear interpolation, a value between two samples (t_i, a_i) and (t_{i+1}, a_{i+1}) is obtained from

$$a(t) = a_i + \frac{t - t_i}{t_{i+1} - t_i} \left(a_{i+1} - a_i\right).$$

STEP holds the preceding sample value between tabulated points. NEAREST uses the closest sample. The amplitude is dimensionless unless the user deliberately assigns another interpretation; it multiplies the referenced load magnitude.

An amplitude does not create a load by itself. It must be referenced by a supported load or boundary-condition keyword. This allows one time history to be reused by several load components.

The supported Abaqus form is *AMPLITUDE, DEFINITION=TABULAR. TIME=STEPTIME and VALUE=RELATIVE are accepted by both readers. These options use the same tabular amplitude representation as the FEMaster interpolation modes.

In a transient analysis, amplitude sample times use the same time unit as *TIME. A model expressed in seconds therefore interprets the amplitude abscissa in seconds.

Parameters

Parameter	Status	Meaning
NAME	required	Unique amplitude name.
TYPE	optional	LINEAR, STEP, or NEAREST; default LINEAR.
DEFINITION	optional	Supported value TABULAR.
TIME	optional	Supported value STEPTIME.
VALUE	optional	Supported value RELATIVE.

Data lines

Entry	Meaning
Both readers	One to four time, value pairs per physical line.

The first table describes a ramp from zero to full magnitude during the first 0.1 units of step time, followed by a constant factor. A load references the amplitude by its NAME.

FEMaster and Abaqus example

```
*AMPLITUDE, NAME=RAMP,
 DEFINITION=TABULAR, TIME=STEPTIME
0.0, 0.0, 0.1, 1.0,
1.0, 1.0
```

Stepwise interpolation

```
*AMPLITUDE, NAME=HOLD, TYPE=STEP
0., 0., 0.5, 1., 1., 1.
```

4.4.2 *SUPPORT

Native *SUPPORT prescribes generalized nodal displacements and rotations. Every row contains a target node or node set followed by six components in the order

$$(U_x, U_y, U_z, R_x, R_y, R_z).$$

A finite number prescribes that component. 0 therefore creates a homogeneous constraint. An omitted entry or NAN leaves the component free.

SUPPORT_COLLECTOR selects a reusable support collector activated by *SUPPORTS. Within an active analysis the same six-component syntax can prescribe an unnamed step-local support. OP=NEW also clears inherited direct supports before these step-local values are applied.

ORIENTATION optionally specifies a local coordinate system. The prescribed components are then expressed in that basis, including cylindrical supports, sliders, or boundaries whose constrained direction is not aligned with the global axes.

Every constrained degree of freedom removes an admissible motion. Constraints without a corresponding physical support increase the model stiffness, while missing support constraints can leave a structural nullspace.

Parameters

Parameter	Status	Meaning
SUPPORT_COLLECTOR/NAME	optional in analysis	Named reusable support collector; required outside an analysis.
OP	optional	MOD (default) for ordinary selection; NEW also clears inherited direct supports.
ORIENTATION	optional	Named local basis for the prescribed components.

Data lines

Entry	Meaning
target	Node ID or node set.
Six values	$U_x, U_y, U_z, R_x, R_y, R_z$; omitted or NAN means free.

This collector stores generalized nodal prescriptions for later activation by SUPPORTS. NAN means that a component is left free; zero fixes that component.

FEMaster and Abaqus example

```
*SUPPORT, SUPPORT_COLLECTOR=BC
FIXED, 0., 0., 0., 0., 0., 0.
GUIDE, NAN, 0., 0., NAN, NAN, NAN
```

Equivalent boundary conditions

```
*BOUNDARY
FIXED, 1, 6, 0.
GUIDE, 2, 3, 0.
```

4.4.3 *BOUNDARY

*BOUNDARY is the supported Abaqus displacement/rotation constraint keyword. TYPE currently supports DISPLACEMENT and defaults to that form.

Each data row contains a target, a first degree of freedom, an optional last degree of freedom, and an optional prescribed value. Abaqus structural DOF numbers are one based:

$$1 \to U_x, \quad 2 \to U_y, \quad 3 \to U_z, \quad 4 \to R_x, \quad 5 \to R_y, \quad 6 \to R_z.$$

If the last DOF is omitted, only the first is constrained. If the value is omitted it defaults to zero.

If the target nodes have an Abaqus *TRANSFORM, the boundary components are interpreted in that local nodal basis. Nonzero prescribed values are currently supported for the documented static procedures. Amplitude-dependent prescribed values are subject to the restrictions described for the corresponding analysis procedure.

Native *SUPPORT presents all six generalized components in one row. Abaqus *BOUNDARY uses DOF ranges, so the same constraint can require several rows.

Parameters

Parameter	Status	Meaning
TYPE	optional	Supported form DISPLACEMENT; default DISPLACEMENT.
AMPLITUDE	optional	Named amplitude for the prescribed boundary history.
OP	optional	MOD (default) or NEW for direct support history.

Data lines

Entry	Meaning
target, first_dof, last_dof, value	DOF range 1–6; last DOF defaults to first and value defaults to zero.

The boundary condition data use numbered DOFs, with inclusive ranges. In this example one displacement is prescribed and two other components are fixed.

FEMaster and Abaqus example

```
*BOUNDARY, TYPE=DISPLACEMENT
TIP, 1, 1, 1.5
TIP, 2, 3, 0.
```

Support vector

```
*SUPPORT, SUPPORT_COLLECTOR=BC
TIP, 1.5, 0., 0., NAN, NAN, NAN
```

4.4.4 *CLOAD

*CLOAD applies concentrated nodal force and moment components. Native FEMaster writes all six generalized load components in one row:

$$(F_x, F_y, F_z, M_x, M_y, M_z).$$

Missing components default to zero.

The target may be an individual node or a node set. When a node set is used, the specified vector is applied to *every* node in the set. It is not divided by the number of nodes. A reference node with a structural/distributing *COUPLING instead transfers one definite resultant force or moment to a surface.

ORIENTATION optionally defines a local load basis. AMPLITUDE optionally multiplies the load by a named amplitude during analyses that evaluate time-dependent loading.

Abaqus *CLOAD uses one DOF and one magnitude per row. DOFs 1–3 represent forces and 4–6 represent nodal moments. The default and REAL forms both apply real-valued, fixed-direction concentrated loading.

The native six-component vector form supplies a step-local load when no collector is named. Abaqus-style target, dof, magnitude rows instead update persistent load history per target and DOF. OP=MOD replaces matching DOF entries; OP=NEW retires inherited direct CLOAD entries before the current command.

When a concentrated nodal load represents a physically distributed load, the idealization can create local stress singularities or mesh-dependent peaks. Surface loading and distributing couplings represent load introduction over a finite area.

Parameters

Parameter	Status	Meaning
NAME/LOAD_COLLECTOR	optional in analysis	Named reusable collector; required outside an analysis.
OP	optional	MOD (default) or NEW for direct load history.
ORIENTATION	native optional	Named local load basis.
AMPLITUDE	optional	Named amplitude where supported.
REAL	optional flag	Real-valued, fixed-direction concentrated loading.

A concentrated load can be expressed by DOF number or by a generalized force vector. When a node set is targeted, the specified force is applied separately to every node in the set.

FEMaster and Abaqus example

```
*CLOAD
TIP, 1, 1000.
TIP, 2, -250.
```

Vector notation

```
*CLOAD, LOAD_COLLECTOR=TIP_LOAD
TIP, 1000., -250., 0., 0., 0., 0.
```

4.4.5 *DLOAD

*DLOAD supports several distributed-load data forms in the unified input language. The Abaqus-compatible GRAV label applies mass-proportional acceleration, BX/BY/BZ prescribe global body-force densities, and P1, P2, etc. apply pressure to the numbered element face. FEMaster also accepts a direct three-component surface-traction vector on a named surface.

The native surface, tx, ty, tz form supplies an unnamed step-local traction, while labeled Abaqus rows element_region, label, magnitude, ... create matching persistent history entries. OP=MOD updates those labeled entries by target and load label; OP=NEW clears inherited direct DLOAD entries.

The vector form specifies traction force per unit area in the selected basis. Where ORIENTATION is given, those vector components are resolved in the local coordinate system. AMPLITUDE optionally scales a load in time.

The consistent nodal force generated by a traction $\boldsymbol{t}$ is obtained conceptually from the surface integral

$$\boldsymbol{f}_e = \int_{A_e} N^T \boldsymbol{t} \, \mathrm{d}A,$$

so mesh refinement changes the number of nodal force contributions but not the intended continuous traction magnitude.

For the GRAV form, the acceleration is

$$\boldsymbol{a} = g\,\boldsymbol{d}.$$

Selected regular MASS elements also receive concentrated gravity loads. Auxiliary *POINTMASS objects are not members of an ELSET and therefore are not selected by *DLOAD, GRAV. For surface pressure on a named surface set, the corresponding Abaqus-compatible command is *DSLOAD, P.

Traction has units of force per unit area. Its total resultant equals the surface integral and therefore coincides numerically with the traction magnitude only when the loaded area is one in the chosen unit system.

Parameters

Parameter	Status	Meaning
NAME/LOAD_COLLECTOR	optional in analysis	Named reusable collector; required outside an analysis.
OP	optional	MOD (default) or NEW for direct load history.
ORIENTATION	native optional	Local traction basis.
AMPLITUDE	optional	Named time scaling where supported.
REAL	optional flag	Real-valued distributed loading.

Gravity acceleration

The four examples below describe distinct physical load types: gravity acceleration, force density, surface traction and element-face pressure. Their units and target regions are not interchangeable.

```
*DLOAD
EALL, GRAV, 9810., 0., 0., -1.
```

Body-force density

```
*DLOAD
EALL, BZ, -0.01
```

Surface traction vector

```
*DLOAD
SIDE, 0., 2.5, 0.
```

Element-face pressure

```
*DLOAD
SOLIDS, P2, 5.0
```

4.4.6 *PLOAD

Native *PLOAD applies scalar pressure to a surface or surface set. Unlike a vector traction, pressure acts normal to the selected finite-element surface. The sign therefore depends on the element side and surface orientation.

For pressure p and oriented unit normal $\boldsymbol{n}$, the surface traction is conceptually

$$\boldsymbol{t} = p\boldsymbol{n}.$$

The resulting nodal forces are obtained by surface integration. The load is therefore independent of how many nodes happen to lie on the surface, provided the discretization converges to the same geometry.

AMPLITUDE may scale the pressure where supported. Abaqus input expresses the corresponding supported pressure through *DSLOAD with type P.

The surface side and element connectivity determine the pressure direction. Reversing the load sign can compensate numerically for an opposite orientation, but the same orientation continues to govern contact and shell-result conventions.

Parameters

Parameter	Status		Meaning
NAME/LOAD_COLLECTOR	optional in analysis		Named reusable collector; required outside an analysis.
OP	optional		MOD (default) for ordinary selection; NEW also clears inherited direct pload entries.
AMPLITUDE	optional		Named pressure scaling function.

The named surface must already exist. PLOAD applies a scalar pressure along its surface normal, whereas DSLOAD expresses the supported pressure through the P load label.

FEMaster and Abaqus example

```
*PLOAD, LOAD_COLLECTOR=PRESSURE
INNER_FACE, 2.0
```

Equivalent DSLOAD pressure

```
*DSLOAD
INNER_FACE, P, 2.0
```

4.4.7 *DSLOAD

*DSLOAD applies an Abaqus-compatible scalar pressure to a named surface set. The supported load label is P; each data record specifies a surface, the label P, and a pressure magnitude. The surface normal determines the load direction.

FOLLOWER=NO selects the fixed reference-surface-normal pressure definition and is also the default. AMPLITUDE may provide time scaling. For a prescribed surface traction vector, use the three-component *DLOAD form.

Parameters

Parameter	Status		Meaning
NAME/LOAD_COLLECTOR	optional analysis	in	Named reusable collector; required outside an analysis.
AMPLITUDE	optional		Named amplitude for time scaling.
FOLLOWER	optional		NO (default) for fixed-reference-surface pressure.
OP	optional		MOD or NEW for load history.

Data lines

Entry	Meaning
Pressure	surface, P, magnitude.

The named FACE_A surface is an existing geometric region. The pressure magnitude is applied in the normal direction of that surface; its orientation therefore matters.

FEMaster and Abaqus example

```
*DSLOAD, FOLLOWER=NO
FACE_A, P, 5.0
```

Equivalent PLOAD pressure

```
*PLOAD
FACE_A, 5.0
```

4.4.8 *VLOAD

Native *VLOAD prescribes a three-component *body-force density* on an element or element set. The components have units of force per unit volume (for example, N/mm^3), *not* acceleration. FEMaster integrates the load over the physical element volume without multiplying by material density:

$$\boldsymbol{f}_e = \int_{\Omega_e} \boldsymbol{N}^T \boldsymbol{b} \, \mathrm{d}\Omega.$$

The resulting force is independent of the mass density assigned to the material. This applies equally to globally defined loads and loads expressed in a local basis via ORIENTATION. AMPLITUDE optionally scales the prescribed force density.

For a gravity load, use a mass-proportional inertial load rather than entering acceleration components as *VLOAD. The supported Abaqus *DLOAD with GRAV interprets its magnitude and direction as acceleration and includes material density and selected regular concentrated masses via the InertialLoad path. The two commands therefore *must not* be treated as numerically interchangeable. Auxiliary native *POINTMASS definitions are not selected through a regular element set.

Parameters

Parameter	Status	Meaning
NAME/LOAD_COLLECTOR	optional in analysis	Named reusable collector; required outside an analysis.
OP	optional	MOD (default) for ordinary selection; NEW also clears inherited direct vload entries.
ORIENTATION	native optional	Named local vector basis for the force density.
AMPLITUDE	native optional	Named time-scaling function.

Data lines

Entry	Meaning
target, bx, by, bz	Element or element-set target followed by three body-force-density components.

The supplied components are forces per unit volume rather than accelerations. Consequently the equivalent DLOAD BX/BY/BZ form does not multiply the value by mass density.

FEMaster and Abaqus example

```
*VLOAD, LOAD_COLLECTOR=BODY_FORCE
EALL, 0., 0., -0.01
```

DLOAD body-force component

```
*DLOAD
EALL, BZ, -0.01
```

4.4.9 *INERTIALOAD

The current native keyword is spelled *INERTIALOAD. It defines rigid-body inertial loading over an element set. Every row specifies a reference centre, translational acceleration, angular velocity $\boldsymbol{\omega}$, and angular acceleration $\boldsymbol{\alpha}$.

For position $\boldsymbol{r}$ measured from the reference centre, the rigid-body acceleration field contains the familiar contributions

$$\boldsymbol{a}(\boldsymbol{r}) = \boldsymbol{a}_0 + \boldsymbol{\alpha} \times \boldsymbol{r} + \boldsymbol{\omega} \times (\boldsymbol{\omega} \times \boldsymbol{r}),$$

for the prescribed instantaneous translational, tangential, and centrifugal terms. The corresponding inertial forces depend on the model's mass distribution.

Regular TYPE=MASS point elements are part of normal element topology. If such an element belongs to the target ELSET, its concentrated translational mass contributes directly to the rigid-body inertia load. A point-mass section with rotary inertia also contributes the corresponding Euler and gyroscopic moments when present through the native point-property path.

CONSIDER_POINT_MASSES has a narrower meaning: it controls whether the auxiliary point elements generated by native *POINTMASS, NSET=... are added in addition to the ordinary target ELSET. The default is false. It does not disable regular TYPE=MASS elements already selected by the target region.

Density, concentrated masses, geometry, angular velocity, and acceleration must all use one consistent unit system. Centrifugal acceleration scales with $\omega^2 r$, so a unit error in angular velocity or length can become especially severe.

The spelling *INERTIALOAD matches the currently accepted native keyword exactly.

Parameters

Parameter	Status		Meaning
NAME/LOAD_COLLECTOR	optional	in analysis	Named reusable collector; required outside an analysis.
OP	optional		MOD (default) for ordinary selection; NEW also clears inherited direct inertial-load entries.
CONSIDER_POINT_MASSES	optional		Include auxiliary native *POINTMASS objects; Boolean, default 0.

Data lines

Entry	Meaning
target	Element set receiving the inertial load; regular MASS elements in this set participate automatically.
cx,cy,cz	Reference centre.
ax,ay,az	Translational acceleration of the centre.
wx,wy,wz	Angular velocity.
alphax,alphay,alphaz	Angular acceleration.

The example describes inertial loading by translational acceleration, rotation and angular acceleration. It is meant for the previously defined region EALL, with optional inclusion of auxiliary point masses.

FEMaster and Abaqus example

```
*INERTIALOAD, LOAD_COLLECTOR=ROT_INERTIA,
 CONSIDER_POINT_MASSES=1
EALL,
0., 0., 0.,
0., 0., 0.,
0., 0., 10.,
0., 0., 2.
```

4.4.10 *TEMPERATURE

*TEMPERATURE prescribes the absolute temperature of a node or node set. It maintains a temperature-condition history that is distinct from the model's initial temperature Field set by *INITIALCONDITION, TYPE=TEMPERATURE. Each data row provides one compiled node reference or node-set name and one temperature value. OP=MOD replaces matching targets and OP=NEW begins a new set of temperature prescriptions.

Parameters

Parameter	Status	Meaning
OP	optional	MOD (default) or NEW.

Data lines

Entry	Meaning
target, temperature	Node ID or node set and absolute temperature.

For a model with the assembled node set HEATED_NODES, the command prescribes a uniform temperature of 80 using the model's temperature units.

FEMaster and Abaqus example

```
*TEMPERATURE, OP=MOD
HEATED_NODES, 80.
```

4.4.11 *HEATFLUX

*HEATFLUX prescribes a heat-input density on an existing surface region. A positive heat flux q acts into the thermal balance and contributes through $\int_A N^T q \, \mathrm{d}A$ over the reference surface. AMPLITUDE references a scalar history multiplying the heat-input density. OP=MOD replaces the matching surface target; OP=NEW starts a new heat-flux history.

Parameters

Parameter	Status	Meaning
OP	optional	MOD (default) or NEW.
AMPLITUDE	optional	Named amplitude multiplying the heat-input density.

Data lines

Entry	Meaning
surface, heat_flux	Named surface or surface set and heat input per unit area.

This model-level definition scales a heat flux on the existing HOT_FACE surface from zero to its nominal value during one unit of step progression.

FEMaster and Abaqus example

```
*AMPLITUDE, NAME=POWER_RAMP,
 DEFINITION=TABULAR
0., 0., 1., 1.
*HEATFLUX, AMPLITUDE=POWER_RAMP
HOT_FACE, 0.25
```

4.4.12 *CONVECTION

*CONVECTION defines Newton cooling on an existing surface region. The surface heat input is

$$q = h(T_\infty - T),$$

where h is the film coefficient, T_∞ is the absolute ambient temperature, and T is the local surface temperature. Each row supplies the target surface, film coefficient, and ambient temperature. A named AMPLITUDE scales h while T_∞ remains the specified value. OP=MOD replaces the matching surface target; OP=NEW starts a new convection history.

Parameters

Parameter	Status	Meaning
OP	optional	MOD (default) or NEW.
AMPLITUDE	optional	Named amplitude multiplying the film coefficient.

Data lines

Entry	Meaning
surface, h, ambient	Named surface, film coefficient, and absolute ambient temperature.

The existing surface COOLED_FACE exchanges heat with an environment at temperature 20 through a film coefficient of 0.015, expressed in the model's consistent thermal units.

FEMaster and Abaqus example

```
*CONVECTION
COOLED_FACE, 0.015, 20.
```

4.5 Constraints and Contact

Constraints introduce kinematic relations between degrees of freedom that go beyond ordinary supports. A support prescribes selected degrees of freedom directly. A constraint instead relates the motion of different nodes or regions, for example by connecting a reference node to a surface, tying two independently meshed regions together, defining an idealized joint, or removing rigid-body motion from an otherwise free component.

The distinction between these constraint types is physical, not merely syntactic. A kinematic coupling imposes a rigid relation between selected motions. A structural/distributing coupling transfers generalized motion and loading between a reference point and a distributed region. A Tie creates a permanent compatibility relation between associated geometry. A Connector idealizes a joint between two points. RBM removes the rigid-body nullspace of a selected structural region. Contact, described at the end of this section, is unilateral and allows separation.

Constraints are used by the active analysis together with the selected support collectors. The numerical method used to enforce the resulting equations is controlled separately by `*CONSTRAINTMETHOD`; see Section 4.6.2.

4.5.1 *COUPLING

*COUPLING relates one master/reference node to a slave region. It is accepted at root and Assembly level after the model has been compiled. The reference is supplied by MASTER; REF NODE is accepted as an alias. The reference may name a node set containing exactly one node or a direct compiled node reference such as 42 or INSTANCE.42.

The coupled target is supplied by SURFACE. The legacy spellings SLAVE and SFSET remain aliases for the same target. FEMaster first resolves the name as a surface region and falls back to a node set. Thus element-based surfaces, node-based coupling regions, and existing native coupling decks use one common constraint path. If the same name exists in both namespaces, the surface region takes precedence.

Native FEMaster accepts two syntax styles. The legacy direct form places TYPE=KINEMATIC or TYPE=STRUCTURAL on *COUPLING and follows it with exactly six selector values in the order

$$(U_x, U_y, U_z, R_x, R_y, R_z).$$

A component is active when the supplied value is greater than zero. Missing values are inactive. These six values are selectors rather than prescribed displacement or rotation magnitudes.

The scoped form is shared by the native and Abaqus readers. If TYPE is omitted, *COUPLING opens a scope and exactly one child command selects its behavior: *KINEMATIC or *DISTRIBUTING. The child form uses inclusive Abaqus-style degree-of-freedom ranges. A line first,last activates all DOFs in that range, a single value activates only that DOF, and several lines may select several ranges. With no child data lines all six DOFs are selected.

*KINEMATIC creates a kinematic coupling. The selected slave degrees of freedom follow the rigid kinematics induced by the reference node. This represents an idealized rigid flange, load-introduction point, or boundary where deformation of the coupled patch is suppressed in the selected directions.

*DISTRIBUTING creates FEMaster's structural load-distribution coupling. The optional COUPLING=CONTINUUM and COUPLING=STRUCTURAL spellings are both accepted and deliberately map to the same internal STRUCTURAL coupling implementation. This transfers generalized loads and motions between the reference point and distributed region without making the complete target region rigid in the same sense as a kinematic coupling.

For a surface target, the distribution is based on the selected surface geometry. For a node-set target, the supplied node region itself defines the coupled region. A reference-point load combined with a distributing coupling represents a definite resultant force or moment independently of surface mesh density, whereas applying identical concentrated loads at every surface node changes the total load under mesh refinement.

The Abaqus reader supports the scoped *COUPLING/*KINEMATIC/*DISTRIBUTING subset documented here. This does not imply support for every Abaqus coupling option; parameters outside the documented subset are not part of the FEMaster compatibility contract.

Parameters

Parameter	Status	Meaning
MASTER / REF NODE	required	Reference node or one-node node set.
SURFACE / SLAVE / SFSET	required	Nonempty slave surface or node set.
TYPE	optional	Native legacy direct form: KINEMATIC or STRUCTURAL. Omit to use a child scope.

Data lines

Entry	Meaning
Direct form	Six positive/inactive selectors for $U_x, U_y, U_z, R_x, R_y, R_z$.
Scoped form	Data belong to `*KINEMATIC` or `*DISTRIBUTING` and use inclusive DOF ranges.

The reference node REF and target surface `LOADED_FACE` must already have been defined. The distributing child command controls which generalized DOFs participate.

FEMaster and Abaqus example

```
*COUPLING, REF NODE=REF, SURFACE=LOADED_FACE
*DISTRIBUTING, COUPLING=STRUCTURAL
1, 3
```

Direct coupling form

```
*COUPLING, MASTER=REF, SURFACE=LOADED_FACE, TYPE=STRUCTURAL
1., 1., 1., 0., 0., 0.
```

4.5.2 *KINEMATIC

*KINEMATIC is a child command of a *COUPLING for which TYPE was omitted. It selects the kinematic coupling behavior and optionally restricts the active generalized degrees of freedom. The command is shared by the native and Abaqus readers.

Each data line contains an inclusive DOF range first,last. Valid DOF numbers are 1 through 6 and correspond to $U_x, U_y, U_z, R_x, R_y, R_z$. If last is omitted, only first is selected. If all data lines are omitted, all six DOFs are active.

Data lines

Entry	Meaning
first, last	Optional inclusive range of generalized DOFs 1–6.

KINEMATIC is a child of a scoped COUPLING definition. Here translations 1–3 and rotation 6 participate in the kinematic constraint.

FEMaster and Abaqus example

```
*COUPLING, REF NODE=RP, SURFACE=FLANGE
*KINEMATIC
1, 3
6
```

4.5.3 *DISTRIBUTING

*DISTRIBUTING is the structural/distributing child of a scoped *COUPLING and is shared by the native and Abaqus readers. The optional COUPLING parameter accepts CONTINUUM or STRUCTURAL; both currently select FEMaster's structural load-distribution implementation.

Its optional DOF data uses the same inclusive 1–6 range syntax as *KINEMATIC. With no range lines all generalized DOFs are selected.

Parameters

Parameter	Status	Meaning
COUPLING	optional	CONTINUUM (default) or STRUCTURAL; both map to FEMaster STRUCTURAL.

Data lines

Entry	Meaning
first, last	Optional inclusive range of generalized DOFs 1–6.

The CONTINUUM and STRUCTURAL selector values currently use the same distributing-coupling implementation; a second input deck is therefore unnecessary.

Unlike a kinematic tie, distributing coupling transfers generalized loads through a weighted relation; the selected node set or surface must already exist.

FEMaster and Abaqus example

```
*COUPLING, REF NODE=RP, SURFACE=LOADED_FACE
*DISTRIBUTING, COUPLING=CONTINUUM
1, 3
```

4.5.4 *EQUATION

*EQUATION defines a homogeneous linear multi-point constraint from weighted nodal degrees of freedom. It is accepted at root or Assembly level after model topology has been compiled.

The first data line contains the total number of terms and must declare at least two. Following physical data lines contain one to four complete triples

$$\text{target}, \quad \text{dof}, \quad \text{coefficient},$$

until exactly the declared number of terms has been read. The degree of freedom is an integer from 1 through 6, corresponding to $U_x, U_y, U_z, R_x, R_y, R_z$, and coefficients must be finite.

A target may be a direct node reference or a node set. If the first target is a single node, one scalar constraint equation is created. If the first target is an NSET, FEMaster expands the input into one equation per member of that set. Subsequent NSET targets are then paired with the first set in their existing compiled order and must have the same number of entries. Subsequent single-node targets are reused in every expanded equation.

For example, two equally sized sets LEFT and RIGHT can impose pairwise equality of one displacement component without manually writing one equation for every node pair. The ordering of the two sets therefore forms part of the equation definition.

Data lines

Entry	Meaning
First line	Number of terms $N \geq 2$.
Following lines	One to four `target, dof, coefficient` triples per line until N terms are complete.

This two-term constraint relates displacement component 1 on two named node sets. Coefficients define the equation sign and magnitude; more terms can be provided for general linear MPCs.

FEMaster and Abaqus example

```
*EQUATION
2
LEFT, 1, 1.0, RIGHT, 1, -1.0
```

4.5.5 *MPC

*MPC defines a two-node kinematic multi-point constraint at root or Assembly scope. Each row specifies the relation type and two compiled node references: the first is the dependent node and the second the independent node. The three relation types have the following kinematic meaning:

- PIN equates the available translational components of both nodes;
- TIE equates their available translational and rotational components;
- BEAM applies a small-rotation rigid-link relation with reference offset $\boldsymbol{r} = \boldsymbol{x}_{dep} - \boldsymbol{x}_{ind}$: $\boldsymbol{u}_{dep} = \boldsymbol{u}_{ind} + \boldsymbol{\theta}_{ind} \times \boldsymbol{r}$, and equates available rotations.

FEMaster expands these relations into constraint equations alongside supports, ties, and couplings.

Data lines

Entry	Meaning
type, dependent_node, independent_node	One two-node relation per row; BEAM, TIE, or PIN.

For the already defined node pairs, the first relation connects them with beam kinematics and the second defines a pin connection.

FEMaster and Abaqus example

```
*MPC
BEAM, 101, 102
PIN, 201, 202
```

4.5.6 *RIGID BODY

*RIGID BODY couples all six generalized degrees of freedom of a node set to a reference node. It is a compact CalculiX-compatible way to define a complete kinematic rigid-body constraint. The NSET parameter specifies the coupled nodes; REF NODE selects the reference node (or a one-node set). The optional CalculiX ROT NODE feature is not supported.

Parameters

Parameter	Status	Meaning
NSET	required	Node set to be coupled rigidly.
REF NODE	required	Reference node or one-node set.

The example rigidly constrains the nodes in FLANGE_NODES to the motion of RP. The full six-DOF kinematic coupling shown below represents the same intended rigid-body motion.

FEMaster and Abaqus example

```
*RIGID BODY, NSET=FLANGE_NODES, REF NODE=RP
```

Equivalent kinematic coupling

```
*COUPLING, REF NODE=RP, SURFACE=FLANGE_NODES
*KINEMATIC
1, 6
```

4.5.7 *TIE

*TIE creates a permanent kinematic connection between slave geometry and master geometry located within a prescribed search distance. The slave may be a node set or a surface set. The master may be a surface set or a line set. FEMaster selects the appropriate supported tie relation from these region types.

DISTANCE defines the geometric search tolerance used to associate slave locations with the master geometry. A distance below the interface separation leaves intended locations unmatched, whereas a distance extending to unrelated geometry admits additional candidates. ADJUST=YES permits the supported initial adjustment of slave geometry to the tie relation; the default is NO. The adjustment is performed once when the tie definition is materialized, before automatic shell reference normals are generated. Subsequent constraint collection does not project the slave nodes again or mutate the reference geometry.

A Tie represents a bonded or otherwise permanently connected interface. Once the relation has been established, opening or sliding that violates the selected compatibility equations is not permitted. It therefore represents idealized bonded interfaces, mesh transitions, or connections that do not separate. *CONTACT instead represents interfaces that may open, close, or lose contact.

The accuracy of a Tie depends on the geometric relation between the two discretizations. Highly nonmatching or strongly curved meshes and competing nearby master surfaces increase the sensitivity to the search distance.

Tie and contact solve different modelling problems. A Tie is an equality-type connection and remains active. Contact is unilateral: it transmits normal interaction only while active and permits subsequent separation. The stiffness of a tied interface is governed by the surrounding structure rather than an artificial penalty spring; physical compliance of an adhesive layer or joint is not represented by the Tie relation itself.

Parameters

Parameter	Status	Meaning
MASTER	required	Nonempty master surface or line set.
SLAVE	required	Nonempty slave node or surface set.
DISTANCE	required	Search/projection distance used to associate slave and master geometry.
ADJUST	optional	NO (default) or YES.

The two regions must already be defined. The distance parameter controls the search tolerance used to identify nodes or faces to tie; ADJUST determines whether the selected nodes may be moved.

FEMaster and Abaqus example

```
*TIE, MASTER=MASTER_FACE,
 SLAVE=SLAVE_NODES,
 DISTANCE=0.5, ADJUST=YES
```

4.5.8 *CONNECTOR

*CONNECTOR defines an idealized kinematic joint between two nodes. NSET1 and NSET2 each identify a node set containing exactly one node. COORDINATESYSTEM specifies the local connector axes, and TYPE selects which relative translations and rotations are constrained or released.

The supported connector types are BEAM, HINGE, CYLINDRICAL, TRANSLATOR, JOIN, and JOINRX. They represent different idealized joint kinematics. For example, a hinge is intended to release a rotational degree of freedom about its local hinge axis while constraining the remaining relative motion required by that joint idealization. A translator and a cylindrical connector likewise rely on their local axis system to define the permitted relative motion.

The coordinate system is therefore part of the physical definition, not a cosmetic orientation. Different local connector axes produce different joint kinematics even when the connector type and nodal positions remain unchanged.

Connectors idealize joints without resolving their internal geometry. Bearing deformation, bolt bending, local contact, joint clearance, and detailed connection stress require corresponding beam, shell, solid, spring, or contact representations.

A coupling can reduce a distributed region to a reference node before two reference nodes are connected. This construction makes the idealized joint definition independent of the surface mesh density.

Parameters

Parameter	Status	Meaning
TYPE	required	BEAM, HINGE, CYLINDRICAL, TRANSLATOR, JOIN, or JOINRX.
NSET1	required	First one-node set.
NSET2	required	Second one-node set.
COORDINATESYSTEM	required	Named coordinate system defining the connector axes.

The two node sets define the connector endpoints. The named orientation supplies the local axes along which the HINGE connector constrains and releases relative motion.

FEMaster and Abaqus example

```
*ORIENTATION, NAME=HINGE_AXES,
 TYPE=RECTANGULAR
1., 0., 0., 0., 1., 0.

*CONNECTOR, TYPE=HINGE,
 NSET1=PIN_A, NSET2=PIN_B,
 COORDINATESYSTEM=HINGE_AXES
```

4.5.9 *RBM

*RBM suppresses the rigid-body modes of a selected structural element region without requiring the user to choose arbitrary fixed nodes. The target is specified by ELSET or its alias SET; if omitted, EALL is used.

A free three-dimensional body has six rigid-body modes: three translations and three rotations. These modes have no elastic strain energy and therefore produce a structural nullspace when no physical support removes them. RBM constructs constraint equations that remove the rigid-body motion of the selected region while avoiding the strong local bias that can arise from fixing one arbitrary corner node in several directions.

RBM removes the rigid-body nullspace without introducing a particular physical support point. It does not represent attachment through bearings, bolts, symmetry, contact, or other supports, and it does not complete missing physical boundary conditions.

RBM is also different from inertia relief. RBM changes the admissible kinematics by suppressing rigid motion. Inertia relief retains the free-body interpretation and balances the applied load with an equivalent rigid-body inertial field.

RBM addresses rigid-body motion itself. Other singularities can instead arise from disconnected parts, released mechanisms, missing Sections, unsupported rotational degrees of freedom, or incorrect constraint definitions and remain separate model conditions.

Parameters

Parameter	Status	Meaning
ELSET / SET	optional	Nonempty target structural element set; default EALL.

This command concerns the rigid-body-mode treatment of an existing element set. It does not turn the selected structural elements into infinitely stiff elements.

FEMaster and Abaqus example

```
*RBM, ELSET=FREE_COMPONENT
```

4.5.10 *CONTACT

Contact is a unilateral constraint between two surface regions. In contrast to a Tie, which permanently enforces compatible motion between associated regions, contact permits the surfaces to separate and transmits normal interaction only when the contact condition becomes active.

The current FEMaster contact formulation is frictionless dual-mortar surface-to-surface contact with an augmented-Lagrange normal-contact law. Contact acts over the physical overlap of slave and master facets and is nonlinear because the active contact region changes with deformation. Its response depends on the master/slave assignment, surface orientation, and penalty scale.

*CONTACT defines one frictionless contact pair between a nonempty master surface set and a nonempty slave surface set. MASTER and SLAVE name the two regions. PENALTY provides the initial contact penalty stiffness. CLEARANCE shifts the zero-gap condition and defaults to zero. FLIP=YES reverses the master-surface normal used by the contact pair; the default is NO.

For each active slave contact degree of freedom, FEMaster uses an augmented-Lagrange relation of the form

$$p_i = \max\left(0, \lambda_i - \varepsilon g_i\right),$$

where g_i is the current normal gap, λ_i is the augmented-Lagrange multiplier, and ε is the effective penalty stiffness. With the sign convention used by the formulation, the max operation suppresses tensile contact: surfaces may separate freely and normal contact pressure is generated only on the compressive side of the constraint.

The mortar formulation determines interaction from the current physical overlap of slave and master facets rather than simply tying a slave node to one permanently selected master node. This makes the contact response considerably less mesh-position dependent than a simple node-to-node contact law, but it also means that changing overlap topology can introduce non-smooth changes into a nonlinear equilibrium iteration.

The user-supplied PENALTY is a numerical stiffness scale, not a physical material constant. A small value permits greater penetration, whereas a large value can make the global tangent system poorly conditioned. The augmented-Lagrange procedure can increase the effective penalty when additional enforcement is required. The relevant scale is therefore set by the structural stiffness of the contacting bodies and evaluated through the resulting penetration and nonlinear conditioning.

Master/slave assignment. The mortar formulation is not algebraically symmetric with respect to the two labels: the dual interpolation and normal multipliers are associated with the slave side. Similar discretizations can admit either assignment, but changing it between comparable models also changes the discrete formulation. With strongly different meshes, assigning the finer or more detailed side as slave places the dual interpolation on that discretization; mesh sensitivity remains part of the numerical comparison.

Normal orientation. Contact sign depends on the surface orientation. A reversed surface orientation can make a physically separated pair appear active or prevent a penetrating pair from developing reaction. FLIP reverses the master normal for the contact pair but does not change the underlying element connectivity or surface side label.

Penalty magnitude. Penetration decreases as contact enforcement becomes stiffer, while excessive stiffness degrades the conditioning of the equilibrium iterations. The input number alone therefore does not define contact accuracy independently of geometry and structural stiffness.

Analysis type. Contact changes with deformation and belongs to nonlinear static analysis. A linear analysis cannot represent opening, closing, changing overlap, or an evolving unilateral active set.

Parameters

Parameter	Status	Meaning
`MASTER`	required	Nonempty master surface set.
`SLAVE`	required	Nonempty slave surface set.
`PENALTY`	required	Initial augmented-Lagrange penalty stiffness.
`CLEARANCE`	optional	Contact-clearance offset; default `0`.
`FLIP`	optional	`NO` (default) or `YES`; reverses the master-surface normal.

The surfaces, contact penalty and clearance determine which nonpenetration constraints become active. Check surface orientations and consistent contact units for the selected penalty.

FEMaster and Abaqus example

```
*SURFACE, NAME=MASTER_FACE
MASTER_ELEMENTS, S1
*SURFACE, NAME=SLAVE_FACE
SLAVE_ELEMENTS, S2

*CONTACT, MASTER=MASTER_FACE,
 SLAVE=SLAVE_FACE,
 PENALTY=1.0e8,
 CLEARANCE=0.0, FLIP=NO
```

4.6 Steps and Loadcases

4.6.1 Analysis Procedures

An analysis procedure determines which finite-element problem FEMaster solves. The same mesh, materials, Sections, loads, and constraints can be reused for different procedures because the governing equations differ even when the physical model is unchanged.

FEMaster supports both `*STEP` with a procedure keyword and `*LOADCASE, TYPE=...` with FEMaster-specific control cards. Each procedure entry shows its own definition first and then a corresponding alternative where appropriate. Both can be used within the unified input language. Constraint handling, nonlinear convergence, damping, time stepping, and eigensolver controls are described in Section 4.6.2.

Analysis families

The principal equations are:

Linear static

$$Ku = f,$$

with the prescribed supports and kinematic constraints applied to the displacement vector u.

Nonlinear static

$$r(u, \lambda) = f_{\text{int}}(u) - \lambda f_{\text{ext}} = 0,$$

solved incrementally with Newton iterations. The load factor λ is prescribed by load control or solved together with u by arc-length control.

Eigenfrequency

$$K\phi_i = \omega_i^2 M\phi_i, \qquad f_i = \frac{\omega_i}{2\pi}.$$

Linear buckling

$$(K + \lambda_i K_G)\,\phi_i = 0,$$

where K_G is the geometric stiffness generated by the reference preload state.

Linear transient dynamics

$$M\ddot{u} + C\dot{u} + Ku = f(t),$$

integrated in time with the implicit Newmark method.

Linear harmonic response

$$\left(K - \omega^2 M + i\omega C\right)\hat{u}(\omega) = \hat{f}(\omega),$$

solved directly for each requested excitation frequency.

4.6.1.1 *LOADCASE

Native *LOADCASE begins an analysis definition. TYPE selects one of the supported procedures:

- LINEARSTATIC;
- NONLINEARSTATIC;
- LINEARBUCKLING;
- LINEARSTATICTOPO;
- EIGENFREQ;
- LINEARTRANSIENT;
- LINEARHARMONIC.

NAME is optional and gives the analysis a descriptive identifier.

Commands inside the loadcase select support and load collectors and configure the numerical procedure. Their validity depends on the analysis type: for example, *TIME and *NEWMARK belong to LINEARTRANSIENT, *FREQUENCIES belongs to LINEARHARMONIC, and *NONLINEAR belongs to NONLINEARSTATIC.

A deck may contain several native loadcases, which FEMaster executes in input order. Persistent direct structural conditions (such as Abaqus-style boundary and single-DOF concentrated-load definitions) carry their target values into subsequent analyses until modified. Unnamed native vector loads and supports participate in their current analysis, while named load and support collectors are activated for each analysis by its own *LOADS and *SUPPORTS selections. The analysis procedure determines which model and solution state it uses.

A loadcase can activate reusable named collectors with *SUPPORTS and *LOADS. The active direct condition history also participates in the analysis, independently of the collector selections.

Parameters

Parameter	Status	Meaning
TYPE	required	Supported native analysis type listed above.
NAME	optional	Descriptive loadcase name.

The following static loadcase assumes that the support collector BC and load collector SERVICE were defined before the analysis. Both are activated explicitly; other collectors in the model remain inactive.

FEMaster and Abaqus example

```
*LOADCASE, TYPE=LINEARSTATIC,
 NAME=SERVICE_LOAD
*SUPPORTS
BC
*LOADS
SERVICE
*END
```

STEP-based analysis

```
*STEP, NAME=SERVICE_LOAD
*STATIC
*BOUNDARY
ROOT, 1, 3, 0.
*CLOAD
TIP, 3, -1000.
*END STEP
```

4.6.1.2 *SUPPORTS

*SUPPORTS selects one or more named support collectors for the active analysis. Their support equations are combined with active direct supports and model constraints such as couplings, ties, connectors, and RBM constraints before solving.

Several collectors may be listed together. This supports modular models, for example one collector for permanent fixture constraints and another for a symmetry condition used only in a particular analysis.

The command is supported by the structural analysis families that use nodal constraints, including static, buckling, eigenfrequency, transient, harmonic, and topology-static analyses.

Abaqus input has no separate collector-selection keyword. Its *BOUNDARY cards define the support conditions associated with the Step directly.

The selected support collectors have been defined earlier with *SUPPORT. Multiple collector names may be activated in one analysis. Previously inherited direct *BOUNDARY prescriptions also remain in the condition history.

Data lines

Entry	Meaning
collector_1, collector_2, ...	Names of previously defined support collectors.

FEMaster and Abaqus example

```
*LOADCASE, TYPE=EIGENFREQ
*SUPPORTS
FIXED_SUPPORTS, SYMMETRY_SUPPORTS
*NUMEIGENVALUES
10
*END
```

Direct boundary conditions can instead be specified with *BOUNDARY in a *STEP. This does not activate a named support collector; see the respective keyword pages.

4.6.1.3 *LOADS

*LOADS activates one or more named load collectors for the current analysis. The assembled external load combines active direct load history with the loads in these selected collectors and any unnamed loads specified in the current analysis.

For a linear static analysis,

$$f = f_{\text{direct}} + \sum_{c} f_c,$$

where f_{direct} includes active direct loads and f_c is the contribution of a selected collector. Unnamed native loads specified inside the active analysis also contribute to the current load vector. For transient or harmonic analysis, amplitudes may make the corresponding load contribution depend on time or excitation frequency according to the supported load definition.

*LOADS is used by linear static, nonlinear static, buckling, transient, and harmonic analyses. A free-vibration eigenfrequency analysis has no applied external-force vector and therefore does not use a load collector.

Abaqus Step-local load cards such as *CLOAD, *DLOAD, and *DSLOAD provide the corresponding Step loading directly.

The loadcase activates three previously defined collectors. This separates reusable load definitions from the choice of load combinations in each analysis.

Data lines

Entry	Meaning
collector_1, collector_2, ...	Names of previously defined load collectors.

FEMaster and Abaqus example

```
*LOADCASE, TYPE=LINEARSTATIC
*LOADS
GRAVITY, PRESSURE, ACTUATOR
*END
```

Direct *CLOAD, *DLOAD, and *DSLOAD definitions may also be specified inside *STEP, without first activating named load collectors.

4.6.1.4 *END

Native *END closes the current *LOADCASE. It is distinct from *END PART, *END INSTANCE, *END ASSEMBLY, and Abaqus *END STEP.

The closing keyword ends the active *LOADCASE. It does not terminate the entire input file and is not used to close an Abaqus-compatible *STEP.

FEMaster and Abaqus example

```
*LOADCASE, TYPE=LINEARSTATIC
*SUPPORTS
BC
*LOADS
SERVICE
*END
```

A *STEP is instead closed by *END STEP. The two closing keywords apply to different analysis scopes.

4.6.1.5 *STEP

*STEP begins an Abaqus-style analysis step. Multiple steps can appear in one deck and are executed consecutively in input order when their respective *END STEP is reached. Each step contains exactly one procedure keyword: *STATIC, *FREQUENCY, *BUCKLE, *DYNAMIC, or *STEADY STATE DYNAMICS. The keyword selects the analysis class and the active procedure controls.

NAME assigns an optional step name. NLGEOM activates the geometrically nonlinear static procedure when present without a value or set to YES; NO and OFF use linear geometry. INC sets the positive maximum number of nonlinear increments (default 100). PERTURBATION selects a linear perturbation step. Loads use a ramped step progression by default; a named *AMPLITUDE may be referenced on supported load cards to prescribe their history.

Abaqus-style *BOUNDARY, single-DOF *CLOAD, labeled *DLOAD and *DSLOAD rows update persistent condition histories. Subsequent steps inherit their target values until matching entries are changed with OP=MOD or the family is reset with OP=NEW. Unnamed native six-component *CLOAD vectors and native *SUPPORT vectors instead apply to the active analysis. Explicit *LOADS and *SUPPORTS selections activate named collectors for the current analysis only. The *SOLVER extension may be placed after the procedure card. *END STEP executes the current analysis and closes its scope.

Parameters

Parameter	Status	Meaning
NAME	optional	Descriptive Step name.
NLGEOM	optional	Bare/YES enables geometric nonlinearity; NO/OFF disables it.
INC	optional	Positive maximum increment count; default 100.
PERTURBATION	optional flag	Selects linear perturbation semantics.

Here a nonlinear static step declares the procedure first, then supplies its boundary conditions and loads, and finally terminates with *END STEP.

FEMaster and Abaqus example

```
*STEP, NAME=LARGE_DEFLECTION,
 NLGEOM=YES, INC=100
*STATIC
0.05, 1.0, 1e-5, 0.1
*BOUNDARY
ROOT, 1, 3, 0.
*CLOAD
TIP, 3, -1000.
*END STEP
```

Sequential steps and condition history

The following two steps use the same node sets ROOT and TIP. The second step retains the support from the first and modifies the previous F_z load on TIP from -100 to -200. Each step executes at its own *END STEP.

```
*STEP, NAME=FIRST
*STATIC
*BOUNDARY
ROOT, 1, 3, 0.
*CLOAD
TIP, 3, -100.
*END STEP
*STEP, NAME=SECOND
*STATIC
*CLOAD, OP=MOD
TIP, 3, -200.
*END STEP
```

Step selecting reusable collectors

The same `BC` and `SERVICE` collectors used in native loadcases may also be selected from a `*STEP` after the procedure card. The collector contents are defined before the analysis with `*SUPPORT` using `SUPPORT_COLLECTOR=BC` and `*CLOAD` using `LOAD_COLLECTOR=SERVICE`.

```
*STEP, NAME=SERVICE_FROM_COLLECTORS
*STATIC
*SUPPORTS
BC
*LOADS
SERVICE
*END STEP
```

LOADCASE-based analysis

```
*LOADCASE, TYPE=NONLINEARSTATIC
*SUPPORTS
BC
*LOADS
SERVICE
*NONLINEAR, CONTROL=LOAD,
 MAX_INCREMENTS=100
*END
```

4.6.1.6 *STATIC

Abaqus *STATIC selects static equilibrium. The supported mapping depends on the Step options.

With neither geometric nonlinearity nor Riks path following, the analysis is linear static:

$$Ku = f.$$

A PERTURBATION Step also uses the linear static formulation.

If NLGEOM is enabled or RIKS is present, the procedure uses nonlinear static equilibrium. The nonlinear residual is

$$r(u, \lambda) = f_{\text{int}}(u) - \lambda f_{\text{ext}},$$

and Newton iterations solve the linearized equilibrium equation

$$K_T \Delta u = -r.$$

Without RIKS, λ is advanced by load control. RIKS activates arc-length path following, in which displacement and load factor are solved together under an additional path constraint. This allows equilibrium paths to continue through limit points where simple monotonic load control may fail.

DIRECT requests fixed nonlinear load increments rather than adaptive stepping where supported.

For ordinary nonlinear static input, the data positions are initial increment, Step period, minimum increment, and maximum increment. The values are converted to the normalized load path represented by the FEMaster nonlinear analysis. For RIKS, the first four entries analogously define initial, period, minimum, and maximum arc-length scales. Abaqus Riks termination controls in the later data positions are currently not supported.

Parameters

Parameter	Status	Meaning
RIKS	optional flag	Select arc-length path following.
DIRECT	optional flag	Request fixed nonlinear load increments.

Data lines

Entry	Meaning
Ordinary static	Optional initial, period, minimum, maximum.
Riks	First four values define the supported path-increment controls; unsupported later termination fields must be omitted.

Linear static analysis

```
*STEP, NAME=SERVICE
*STATIC
*BOUNDARY
ROOT, 1, 6, 0.
*CLOAD
TIP, 3, -1000.
*END STEP
```

Nonlinear arc-length analysis

```
*STEP, NAME=POSTBUCKLING, NLGEOM=YES, INC=200
*STATIC, RIKS
0.05, 1.0, 1e-5, 0.1
*END STEP
```

FEMaster nonlinear loadcase

```
*LOADCASE, TYPE=NONLINEARSTATIC
*NONLINEAR, CONTROL=ARC_LENGTH, INITIAL_INCREMENT=0.05, MAX_INCREMENTS=200
*END
```

4.6.1.7 *FREQUENCY

Abaqus *FREQUENCY selects free-vibration eigenfrequency extraction. The constrained generalized eigenproblem is

$$K\phi_i = \lambda_i M\phi_i, \qquad \lambda_i = \omega_i^2, \qquad f_i = \frac{\sqrt{\lambda_i}}{2\pi}.$$

The data line begins with the positive number of requested modes.

The mode shapes ϕ_i describe relative deformation patterns; their arbitrary scale is not a physical vibration amplitude. Frequencies are determined by both stiffness and mass, so density, concentrated masses, Sections, and support conditions must all be physically meaningful.

EIGENSOLVER accepts LANCZOS or SUBSPACE as supported Abaqus syntax. FEMaster's own eigensolution settings determine the numerical backend used for the calculation.

Data lines

Entry	Meaning
First value	Positive number of requested eigenpairs.
Remaining accepted positions	Additional Abaqus spectral controls not used by the currently supported mapping.

The eigenfrequency step below requests 12 modes of the constrained model. The node set FIXED must already exist, and meaningful frequencies require stiffness and mass definitions.

Parameters

Parameter	Status	Meaning
EIGENSOLVER	optional	LANCZOS or SUBSPACE; the solver backend is selected internally.

FEMaster and Abaqus example

```
*STEP, PERTURBATION
*FREQUENCY, EIGENSOLVER=LANCZOS
12
*BOUNDARY
FIXED, 1, 6, 0.
*END STEP
```

Eigenfrequency loadcase

```
*LOADCASE, TYPE=EIGENFREQ
*SUPPORTS
BC
*NUMEIGENVALUES
12
*END
```

4.6.1.8 *BUCKLE

Abaqus *BUCKLE selects linear eigenvalue buckling. The analysis first establishes the reference stress state caused by the selected preload. That stress state produces a geometric stiffness matrix K_G. Buckling factors then follow from

$$(K + \lambda_i K_G)\,\phi_i = 0.$$

Depending on the sign convention used for K_G, the equivalent generalized eigenproblem may be written with $-K_G$ on the right-hand side.

The eigenvalue λ_i is a multiplier on the reference preload, not an absolute force. If the preload corresponds to 10 kN and the first factor is 3.2, the linearized critical load associated with that mode is 32 kN. This interpretation is valid only within the assumptions of linear eigenvalue buckling; imperfections, material nonlinearity, contact changes, and nonlinear prebuckling deformation are not represented by the eigenvalue alone.

The first data value is the positive number of requested buckling modes. EIGENSOLVER accepts LANCZOS or SUBSPACE as supported Abaqus syntax.

The buckling step first forms the reference stress state from its applied load and then computes load factors. The selected nodes, load and element properties must exist before this analysis.

Parameters

Parameter	Status	Meaning
EIGENSOLVER	optional	LANCZOS or SUBSPACE; the solver backend is selected internally.

Data lines

Entry	Meaning
count	Number of requested buckling modes.

FEMaster and Abaqus example

```
*STEP, PERTURBATION
*BUCKLE
5
*BOUNDARY
ROOT, 1, 3, 0.
*CLOAD
TIP, 1, -1000.
*END STEP
```

Buckling loadcase

```
*LOADCASE, TYPE=LINEARBUCKLING
*SUPPORTS
BC
*LOADS
PRELOAD
*NUMEIGENVALUES
5
*SIGMA
0.0
*END
```

4.6.1.9 *DYNAMIC

Abaqus *DYNAMIC, DIRECT selects FEMaster's linear implicit transient analysis. The structural equation is

$$M\ddot{u}(t) + C\dot{u}(t) + Ku(t) = f(t).$$

The current supported mapping uses a fixed time increment and therefore requires DIRECT. Geometric nonlinearity is not supported by this linear transient procedure.

The first two data values are the fixed time increment Δt and the positive Step period T. The analysis runs from $t = 0$ to $t = T$ with the specified increment. Loads may vary with time through amplitudes.

Native FEMaster exposes the same analysis through LINEARTRANSIENT, *TIME, *NEWMARK, optional *DAMPING, and output-cadence controls.

The transient step integrates the response for one second using 0.01-second increments. For time-dependent loading, define an *AMPLITUDE and reference it from *CLOAD.

Parameters

Parameter	Status	Meaning
DIRECT	required flag	Fixed-increment implicit time integration.

Data lines

Entry	Meaning
dt, period	Time increment and positive step duration.

FEMaster and Abaqus example

```
*STEP
*DYNAMIC, DIRECT
0.01, 1.0
*BOUNDARY
ROOT, 1, 3, 0.
*CLOAD, AMPLITUDE=RAMP
TIP, 1, 100.
*END STEP
```

Transient loadcase

```
*LOADCASE, TYPE=LINEARTRANSIENT
*SUPPORTS
BC
*LOADS
DYNAMIC_LOAD
*TIME
0., 1., 0.01
*NEWMARK
0.25, 0.5
*END
```

4.6.1.10 *STEADY STATE DYNAMICS

Abaqus *STEADY STATE DYNAMICS, DIRECT selects direct linear harmonic response. For harmonic excitation at angular frequency $\omega = 2\pi f$, FEMaster solves the complex system

$$\left(K - \omega^2 M + i\omega C\right)\hat{u} = \hat{f}.$$

The result $\hat{u}$ contains amplitude and phase information. Resonances appear when the excitation frequency approaches a natural frequency and damping is small.

DIRECT is required. INTERVAL currently supports RANGE. FREQUENCYSCALE may be LINEAR or LOGARITHMIC.

Each data record contains lower frequency, optional upper frequency, optional point count, bias, and scale factor. If the upper frequency is omitted or zero, only the lower frequency is solved. For a range, the upper frequency must be at least as large as the lower. The default point count is 20. Bias values other than 1 and scale factors other than 1 are not supported. A logarithmic range requires a positive lower frequency.

Native FEMaster specifies a single equidistant frequency sweep per *FREQUENCIES, SCALE=LINEAR card using the lower bound, upper bound and point count. See Section 4.6.2 for the spacing formula and the corresponding native loadcase example.

This step performs a frequency-response analysis over 10–1000 Hz. The harmonic force is applied to the previously defined node set TIP; the stiffness and mass properties must be assigned to the model.

Parameters

Parameter	Status	Meaning
DIRECT	required flag	Solve the direct complex harmonic system.
INTERVAL	optional	RANGE (only supported setting).
FREQUENCYSCALE	optional	LINEAR or LOGARITHMIC.

Data lines

Entry	Meaning
f_min, f_max, count, bias, scale	Frequency interval, sample count and two unit-valued controls.

FEMaster and Abaqus example

```
*STEP
*STEADY STATE DYNAMICS, DIRECT,
 FREQUENCYSCALE=LINEAR
10., 1000., 100, 1., 1.
*CLOAD, REAL
TIP, 3, -100.
*END STEP
```

Harmonic loadcase

```
*LOADCASE, TYPE=LINEARHARMONIC
*SUPPORTS
BC
*LOADS
HARMONIC_FORCE
*FREQUENCIES, SCALE=LINEAR
10., 1000., 100
*END
```

4.6.1.11 *END STEP

*END STEP closes the active Abaqus Step. A supported procedure must have been selected within the Step.

This closing keyword executes and finishes the selected *STEP. A FEMaster *LOADCASE uses *END instead.

FEMaster and Abaqus example

```
*STEP
*STATIC
*BOUNDARY
ROOT, 1, 3, 0.
*CLOAD
TIP, 3, -1000.
*END STEP
```

The corresponding *LOADCASE scope is closed by *END, not by *END STEP.

4.6.2 Solver and Numerical Controls

The commands in this section determine how an analysis is solved. They do not change the mesh or material model; instead they control the linear algebra, constraint treatment, nonlinear path following, time integration, damping, frequency sampling, eigenvalue extraction, and selected static balancing procedures.

These settings can strongly affect robustness and computational cost, but they do not correct an inconsistent physical model. A mechanism, missing Section, disconnected mesh, or inadequate support definition remains a model-level cause of singularity regardless of the numerical regularization.

4.6.2.1 *SOLVER

*SOLVER selects the sparse linear solution strategy for supported native loadcases and the supported Abaqus *STEP procedures. In Abaqus input, it is a FEMaster extension and must appear after the procedure keyword has created the active analysis; it is not an Abaqus-native solver-selection card. DEVICE may be CPU or GPU; the default is CPU. METHOD may be DIRECT or INDIRECT; the default is DIRECT.

A direct method factorizes the sparse equation matrix and then solves by substitutions. Its convergence does not depend on an iterative stopping process, while its factorization memory can become substantial for large three-dimensional models.

An indirect method solves the linear system iteratively. For a system

$$Ax = b,$$

the method improves an approximate x until the residual

$$r = b - Ax$$

is sufficiently small. Iterative methods can reduce memory use for suitable large systems, but their efficiency is strongly affected by conditioning, matrix definiteness, scaling, and the available preconditioning strategy.

The chosen constraint formulation matters because it changes the algebraic character of the system. Nullspace and elimination methods lead to reduced structural systems and are suitable for direct or supported iterative solution. Lagrange multipliers create the indefinite saddle-point system

$$\begin{bmatrix} K & C^T \\ C & 0 \end{bmatrix} \begin{bmatrix} u \\ \lambda \end{bmatrix} = \begin{bmatrix} f \\ d \end{bmatrix},$$

which requires a linear solver that can handle indefinite matrices and is therefore restricted to supported direct configurations.

The registration applies to linear static, nonlinear static, linear buckling, topology-static, linear transient, and linear harmonic analyses. The eigenfrequency loadcase does not currently implement this *SOLVER setting. The actual CPU/GPU libraries available depend on the FEMaster build. Selecting GPU in a build without the required GPU backend cannot create GPU support by itself.

The default CPU/direct configuration avoids iterative convergence and backend-specific GPU requirements. Indirect or GPU configurations exchange this baseline for different memory, scaling, and hardware characteristics; their numerical behaviour still depends on the physical model and constraint system.

Parameters

Parameter	Status	Meaning
DEVICE	optional	CPU (default) or GPU.
METHOD	optional	DIRECT (default) or INDIRECT.

This example selects an iterative CPU solver for a linear static step. The procedure must be declared before *SOLVER; the constraint and load definitions apply to the same step. The solver keyword changes the equation solution strategy, not the physical analysis type.

FEMaster and Abaqus example

```
*STEP, NAME=SERVICE
*STATIC
*SOLVER, DEVICE=CPU, METHOD=INDIRECT
*BOUNDARY
FIXED, 1, 3, 0.
*CLOAD
TIP, 3, -1000.
*END STEP
```

The same `*SOLVER` command can be placed inside a `*LOADCASE`. It must appear after the procedure keyword when used inside an Abaqus-compatible `*STEP`.

4.6.2.2 *CONSTRAINTMETHOD

*CONSTRAINTMETHOD selects how equations of the form

$$Cu = d$$

are imposed on the structural system. Supported input values are NULLSPACE, LAGRANGE, and ELIMINATION. Linear static and topology-static analyses default to ELIMINATION; nonlinear and harmonic analyses retain NULLSPACE and currently support only that method.

Nullspace projection

The nullspace method writes every admissible displacement vector as

$$u = u_p + Tq,$$

where

$$Cu_p = d, \qquad CT = 0.$$

The columns of T span all homogeneous motions that satisfy the constraints, while u_p is one particular displacement satisfying the prescribed inhomogeneous values.

For a linear static system

$$Ku = f,$$

substitution gives the reduced equation

$$T^T K T q = T^T (f - K u_p).$$

Thus constrained directions are removed before the linear solve. The same projection applies naturally to mass and damping operators in dynamic analyses:

$$K_r = T^T K T, \qquad M_r = T^T M T, \qquad C_r = T^T C T.$$

Nullspace projection handles general linear constraint equations while keeping the reduced system in structural form. Couplings, ties, and other multi-point constraints can therefore be represented without requiring a favourable substitution ordering.

Lagrange multipliers

The Lagrange method retains the original displacement vector and adds one multiplier per constraint equation:

$$\begin{bmatrix} K & C^T \\ C & 0 \end{bmatrix} \begin{bmatrix} u \\ \lambda \end{bmatrix} = \begin{bmatrix} f \\ d \end{bmatrix}.$$

The multipliers λ represent generalized constraint reactions. The advantage is the direct enforcement of the equations without eliminating displacement variables. The disadvantage is the larger indefinite saddle-point system.

This method requires a supported direct backend. Its additional unknowns provide multiplier reactions and preserve the original displacement variables.

Elimination

Elimination chooses dependent degrees of freedom and expresses them in terms of independent degrees of freedom, again producing an affine relation

$$u = u_p + Tq.$$

Unlike a numerical nullspace basis, the transformation is constructed from explicit substitutions. This can be very efficient for constraints with a clear master/slave structure and an acyclic dependency graph.

For dense or strongly coupled general equations, nullspace projection avoids the favourable substitution ordering required by efficient elimination.

Parameters

Parameter	Status	Meaning
TYPE	required	NULLSPACE, LAGRANGE, or ELIMINATION.

FEMaster and Abaqus example

```
*LOADCASE, TYPE=LINEARSTATIC
*SUPPORTS
BC
*LOADS
SERVICE
*CONSTRAINTMETHOD, TYPE=NULLSPACE
*SOLVER, METHOD=INDIRECT
*END
```

4.6.2.3 *NONLINEAR

*NONLINEAR configures a native NONLINEARSTATIC analysis. It controls the equilibrium path, increment size, adaptive growth and cutback, Newton iteration limits, convergence tolerance, and optional weak-row regularization.

At a prescribed load factor λ, equilibrium is

$$r(u, \lambda) = f_{\text{int}}(u) - \lambda f_{\text{ext}} = 0.$$

Newton's method linearizes the residual at iteration k:

$$K_T(u_k)\Delta u_k = -r(u_k, \lambda), \qquad u_{k+1} = u_k + \Delta u_k.$$

The tangent stiffness K_T contains the current material, geometric, constraint, and contact contributions appropriate to the model.

Load control

CONTROL=LOAD is the default. The load factor is advanced by a prescribed increment $\Delta\lambda$, and Newton iterations solve only for the displacement correction. Load control is efficient on monotonic equilibrium paths but cannot generally pass a limit point where the physical equilibrium curve turns backward in load factor.

Arc-length control

CONTROL=ARC_LENGTH solves displacement and load factor together under an additional path constraint. In generic form the constraint limits a combined displacement/load increment,

$$\Delta u^T \Delta u + \psi^2 \, \Delta\lambda^2 \, \|f_{\text{ext}}\|^2 = \Delta s^2,$$

where Δs is the arc-length scale and ψ is controlled by ARC_LENGTH_PSI. This allows the solution to continue through snap-through or other limit points for which simple load control would stall.

Increment control

INITIAL_INCREMENT defines the first normalized load or arc-length step. MINIMUM_INCREMENT and MAXIMUM_INCREMENT bound adaptive changes. MAX_INCREMENTS limits the number of accepted increments. The legacy INCREMENTS=n shorthand sets the initial scale to $1/n$; an explicitly supplied INITIAL_INCREMENT takes precedence.

With ADAPTIVE=ON, quickly converged increments can grow by GROWTH_FACTOR, while rejected or slowly converged attempts are reduced by CUTBACK_FACTOR. The default thresholds are FAST_ITERATIONS=6, SLOW_ITERATIONS=10, GROWTH_FACTOR=1.5, and CUTBACK_FACTOR=0.5. MAXIMUM_CUTBACKS limits repeated reductions of one attempted path segment.

Adaptive stepping does not alter the equilibrium equations. It changes only how aggressively the solution path is traversed. Persistently small increments can result from contact transitions, material stiffness, element distortion, unstable mechanisms, or an inconsistent tangent; cutback only changes the attempted path discretization.

Newton convergence

MAXITER limits Newton iterations per attempted increment; the default is 20. TOL, default 10^{-3}, controls the normalized equilibrium-residual tolerance. Newton uses a maximum-component relative force residual, rather than the former RMS/L2 measure, and independently checks the relative displacement correction (currently 10^{-2} in both load-control and arc-length implementations). Ordinarily *both* criteria must pass.

At the first residual evaluation, where no correction has yet been calculated, immediate convergence is allowed only when the residual is at most $\min(\mathrm{TOL}, 10^{-8})$. Changing `TOL` does not change the separately configured internal displacement-correction tolerance. A small residual tolerance does not guarantee an accurate finite-element model: mesh discretization, constitutive assumptions, and load idealization remain independent error sources.

Weak-row regularization

`REGULARIZE_ZERO_ROWS=ON` enables diagonal stabilization for tangent rows whose stiffness is effectively zero. `REGULARIZATION_ALPHA`, default 10^{-4}, controls the scale of that stabilization relative to representative nonzero tangent stiffness.

This option stabilizes temporary zero-stiffness directions that can occur in nonlinear models, for example before an otherwise meaningful mechanism engages. It does not resolve a genuinely unconstrained rigid-body mode or disconnected component. A material change of the physical response with increasing regularization indicates sensitivity to the underlying model rather than only numerical stabilization.

Parameters

Parameter	Status	Meaning
`CONTROL`	optional	`LOAD` (default) or `ARC_LENGTH`.
`INCREMENTS`	optional	Legacy count mapped to initial scale $1/n$.
`MAX_INCREMENTS`	optional	Maximum accepted nonlinear increments.
`INITIAL_INCREMENT`	optional	Initial load/arc-length step scale.
`MINIMUM_INCREMENT`	optional	Lower step-size bound.
`MAXIMUM_INCREMENT`	optional	Upper step-size bound.
`ARC_LENGTH_PSI`	optional	Arc-length load weighting; default 1.0.
`ADAPTIVE`	optional	Adaptive growth/cutback; default `ON`.
`GROWTH_FACTOR`	optional	Fast-convergence growth factor; default 1.5.
`CUTBACK_FACTOR`	optional	Rejection/slow-convergence reduction factor; default 0.5.
`FAST_ITERATIONS`	optional	Fast convergence threshold; default 6.
`SLOW_ITERATIONS`	optional	Slow convergence threshold; default 10.
`MAXIMUM_CUTBACKS`	optional	Cutback limit; default 20.
`MAXITER`	optional	Newton iteration limit; default 20.
`TOL`	optional	Relative force-residual tolerance; default 10^{-3}. A separate relative displacement-correction criterion is also enforced.
`REGULARIZE_ZERO_ROWS`	optional	Enable weak-row regularization; default `OFF`.
`REGULARIZATION_ALPHA`	optional	Regularization scale; default 10^{-4}.

Arc-length controls are set inside a nonlinear static loadcase. The named collectors BC and ACTUATOR must have been defined before that loadcase.

FEMaster and Abaqus example

```
*LOADCASE, TYPE=NONLINEARSTATIC
*SUPPORTS
BC
*LOADS
ACTUATOR
*NONLINEAR, CONTROL=ARC_LENGTH,
 MAX_INCREMENTS=200,
 INITIAL_INCREMENT=0.05,
 MINIMUM_INCREMENT=1e-5,
 MAXIMUM_INCREMENT=0.1,
 MAXITER=40, TOL=1e-8,
 ADAPTIVE=ON
*END
```

Riks procedure

```
*STEP, NLGEOM=YES, INC=200
*STATIC, RIKS
0.05, 1.0, 1e-5, 0.1
*BOUNDARY
ROOT, 1, 3, 0.
*CLOAD
TIP, 3, -1000.
*END STEP
```

4.6.2.4 *TIME

*TIME defines the time interval and fixed integration step for LINEARTRANSIENT. Two forms are accepted:

$$t_{\text{start}},\ t_{\text{end}}, \Delta t$$

or the compact form

$$t_{\text{end}}, \Delta t,$$

which uses $t_{\text{start}} = 0$.

The time increment must resolve the highest response frequencies that matter to the analysis. Implicit time integration may remain numerically stable for a large Δt while still giving an inaccurate transient response. Stability and accuracy are not the same criterion.

The ratio between Δt and the shortest relevant period T_{min} determines the temporal resolution. Phase and peak-amplitude accuracy generally require many points per period.

In a linear transient loadcase, *TIME defines the integration window. The previously defined collectors BC and RAMP_LOAD supply supports and loading. This example integrates 0–2 seconds in fixed 0.005-second increments.

Data lines

Entry	Meaning
t_start, t_end, dt	Start, end, and fixed time increment; the start time is optional.

FEMaster and Abaqus example

```
*LOADCASE, TYPE=LINEARTRANSIENT
*SUPPORTS
BC
*LOADS
RAMP_LOAD
*TIME
0., 2., 0.005
*NEWMARK
0.25, 0.5
*END
```

DYNAMIC procedure

The equivalent DYNAMIC procedure specifies the increment first and the step period second; boundary conditions and loading belong inside STEP.

```
*STEP, NAME=TRANSIENT
*DYNAMIC, DIRECT
0.005, 2.0
*BOUNDARY
FIXED, 1, 3, 0.
*CLOAD
TIP, 1, 100.
*END STEP
```

4.6.2.5 *NEWMARK

*NEWMARK sets the β and γ parameters of the implicit Newmark method. Defaults are

$$\beta = 0.25, \qquad \gamma = 0.5,$$

the constant-average-acceleration scheme.

Newmark updates displacement and velocity according to

$$u_{n+1} = u_n + \Delta t\, v_n + \Delta t^2 \left[\left(\frac{1}{2} - \beta \right) a_n + \beta a_{n+1} \right],$$

$$v_{n+1} = v_n + \Delta t \left[(1-\gamma) a_n + \gamma a_{n+1} \right].$$

For the default parameters the linear method is unconditionally stable for the standard undamped problem and introduces no algorithmic high-frequency damping. Accuracy still requires a sufficiently small time step.

Data lines

Entry	Meaning
beta, gamma	Newmark integration parameters.

The integration parameters belong to the transient loadcase, together with its time interval. Here the previously defined collectors BC and RAMP_LOAD are activated; the explicit Newmark values select the default constant-average-acceleration scheme.

FEMaster and Abaqus example

```
*LOADCASE, TYPE=LINEARTRANSIENT
*SUPPORTS
BC
*LOADS
RAMP_LOAD
*TIME
0., 1., 0.01
*NEWMARK
0.25, 0.5
*END
```

4.6.2.6 *DAMPING

*DAMPING, TYPE=RAYLEIGH defines proportional damping for transient or harmonic analysis:

$$C = \alpha M + \beta K.$$

The mass-proportional coefficient α primarily affects low frequencies, while the stiffness-proportional coefficient β increasingly affects high frequencies.

For a mode with circular frequency ω_i, the approximate modal damping ratio is

$$\zeta_i = \frac{1}{2}\left(\frac{\alpha}{\omega_i} + \beta\omega_i\right).$$

This relation determines α and β from specified damping ratios at two frequencies.

Rayleigh damping is frequency dependent. Coefficients fitted at two frequencies produce different damping ratios elsewhere in the spectrum, especially for modes far below or above the fitting range.

Parameters

Parameter	Status	Meaning
TYPE	required	Currently RAYLEIGH.

Data lines

Entry	Meaning
alpha, beta	Mass- and stiffness-proportional Rayleigh coefficients.

In this harmonic loadcase, Rayleigh damping is applied to the response at every frequency in the sweep. The support and excitation collectors must already exist. Here $\alpha = 0$ and $\beta = 10^{-5}$; damping affects both resonant amplitudes and phases. The same keyword can be used in transient analysis.

FEMaster and Abaqus example

```
*LOADCASE, TYPE=LINEARHARMONIC
*SUPPORTS
BC
*LOADS
HARMONIC
*FREQUENCIES, SCALE=LINEAR
10., 1000., 100
*DAMPING, TYPE=RAYLEIGH
0., 1.e-5
*END
```

4.6.2.7 *FREQUENCIES

*FREQUENCIES defines the excitation-frequency sweep for LINEARHARMONIC. The native command currently supports linear spacing; SCALE may be omitted or set to LINEAR.

The data line contains start frequency f_0, end frequency f_1, and integer point count n. For $n > 1$,

$$f_i = f_0 + i\frac{f_1 - f_0}{n - 1}, \qquad i = 0, \ldots, n - 1.$$

For $n = 1$, the single solved frequency is f_0.

Frequency resolution determines which parts of the response curve are sampled. A coarse linear sweep can miss a narrow lightly damped resonance even though the endpoints cover the correct range; additional points near a resonance resolve its peak amplitude and phase more closely.

Parameters

Parameter	Status	Meaning
SCALE	optional	Currently LINEAR.

Data lines

Entry	Meaning
f_start, f_end, count	One data row with nonnegative start frequency, end frequency greater than or equal to start, and a positive integer point count.

Assuming that the support collector BC and harmonic-load collector HARMONIC have been defined earlier, this example solves 100 equally spaced excitation frequencies between 10 and 1000 Hz. *FREQUENCIES belongs inside the harmonic loadcase; it does not initiate an analysis on its own.

FEMaster and Abaqus example

```
*LOADCASE, TYPE=LINEARHARMONIC, NAME=SWEEP
*SUPPORTS
BC
*LOADS
HARMONIC
*FREQUENCIES, SCALE=LINEAR
10., 1000., 100
*END
```

Steady-state dynamics procedure

The corresponding Abaqus-compatible procedure defines its frequency range inside a STEP. The force on TIP is defined in the step rather than through a load collector.

```
*STEP, NAME=SWEEP
*STEADY STATE DYNAMICS, DIRECT,
 FREQUENCYSCALE=LINEAR
10., 1000., 100, 1., 1.
*CLOAD, REAL
TIP, 3, -100.
*END STEP
```

4.6.2.8 *NUMEIGENVALUES

*NUMEIGENVALUES sets the number of requested eigenpairs for EIGENFREQ or LINEARBUCKLING. The data value must be a positive integer.

The requested value is a result count, not a frequency or buckling-factor cutoff. A request for ten modes therefore returns ten eigenpairs but does not independently define the covered spectral range, particularly in the presence of multiplicities or closely spaced eigenvalues.

Here an eigenfrequency loadcase requests 20 modes from the assembled stiffness and mass matrices. BC is a previously defined support collector; the mode count is specified inside the analysis rather than at model level.

Data lines

Entry	Meaning
count	Positive number of requested eigenpairs.

FEMaster and Abaqus example

```
*LOADCASE, TYPE=EIGENFREQ
*SUPPORTS
BC
*NUMEIGENVALUES
20
*END
```

Frequency extraction procedure

The frequency extraction procedure requests the number of modes directly. The node set FIXED must already be defined.

```
*STEP, PERTURBATION
*FREQUENCY
20
*BOUNDARY
FIXED, 1, 6, 0.
*END STEP
```

4.6.2.9 *SIGMA

*SIGMA sets the spectral shift used by native LINEARBUCKLING. The shift guides the eigensolver toward the targeted part of the buckling spectrum.

For the reduced buckling problem written as

$$A\phi = \lambda(-B)\phi,$$

with elastic stiffness A and geometric stiffness contribution B, shift-invert methods become most effective when the shift is placed near, but appropriately below, the eigenvalue range of interest.

SIGMA=0 requests the automatic shift strategy. FEMaster estimates a scale from the pre-buckling solution where possible and falls back to conservative stiffness/geometric-stiffness estimates if necessary. A manual shift directly targets a known approximate spectral range.

In a linear-buckling loadcase, PRELOAD defines the reference loading and BC defines the supports. The example requests five buckling factors and an automatic spectral shift (0.0). The factors multiply the reference load; they are not forces.

Data lines

Entry	Meaning
sigma	Spectral shift; zero requests the automatic buckling shift.

FEMaster and Abaqus example

```
*LOADCASE, TYPE=LINEARBUCKLING
*SUPPORTS
BC
*LOADS
PRELOAD
*NUMEIGENVALUES
5
*SIGMA
0.0
*END
```

4.6.2.10 *WRITEEVERY

*WRITEEVERY controls how often transient results are written. It does not change the integration step or the computed transient trajectory.

With TYPE=STEPS, the data value is the number of integration steps between written states. With TYPE=TIME, it is a physical time interval. Writing less frequently can greatly reduce output size when the integration time step is small.

Output sampling must still be fine enough for the quantity being inspected. A transient can be integrated accurately but appear to miss a short peak if results are written too sparsely.

Parameters

Parameter	Status	Meaning
TYPE	optional	STEPS (default) or TIME.

Data lines

Entry	Meaning
interval	Write frequency in steps or physical time, selected by TYPE.

With a fixed time increment of 0.001 seconds over one second, the solver performs 1000 integration increments. *WRITEEVERY saves intermediate results every ten steps without changing the integration. The named collectors are defined before the loadcase.

FEMaster and Abaqus example

```
*LOADCASE, TYPE=LINEARTRANSIENT
*SUPPORTS
BC
*LOADS
RAMP_LOAD
*TIME
0., 1., 0.001
*WRITEEVERY, TYPE=STEPS
10
*END
```

4.6.2.11 *INITIALVELOCITY

*INITIALVELOCITY assigns initial generalized nodal velocity for LINEARTRANSIENT from an existing six-component nodal Field. The Field must use TYPE=NODE and component order

$$(V_x, V_y, V_z, \Omega_x, \Omega_y, \Omega_z),$$

corresponding to the translational and rotational nodal DOFs.

FIELD names the Field that completely supplies the initial velocity state. Initial displacement is taken from the analysis/model state defined by the current procedure.

Parameters

Parameter	Status	Meaning
FIELD	required	Existing six-component NODE Field.

Create the initial-velocity field before the transient loadcase. The first node starts with an x-velocity of one length unit per time unit; unspecified nodes remain at rest.

FEMaster and Abaqus example

```
*FIELD, NAME=V0, TYPE=NODE, COLS=6, FILL=ZERO
0, 1., 0., 0., 0., 0., 0.

*LOADCASE, TYPE=LINEARTRANSIENT
*SUPPORTS
BC
*TIME
0., 1., 0.01
*INITIALVELOCITY, FIELD=V0
*END
```

4.6.2.12 *INERTIARELIEF

*INERTIARELIEF enables inertia relief for a linear static analysis of an intentionally free or insufficiently supported body.

For a free body under a nonzero resultant external force or moment, the ordinary static equation has no physical equilibrium because the body accelerates. Inertia relief determines the rigid-body translational and angular acceleration associated with the applied load and model mass properties, forms the corresponding inertial forces, and balances the external resultant. The remaining static solution represents elastic deformation relative to the accelerating body.

This represents free-flight or free-body loadcases without adding an artificial fixed support. It does not represent real bearings, bolts, fixtures, or other physical supports.

All regular compiled structural elements contribute to the mass properties used by inertia relief. This includes ordinary ELEMENT, TYPE=MASS point elements: their concentrated translational mass contributes to total mass, center of gravity, and parallel-axis inertia automatically, and any point-section rotary inertia contributes directly to the inertia tensor.

CONSIDER_POINT_MASSES controls only the additional auxiliary point elements generated after compilation by native *POINTMASS, NSET=.... When enabled, their translational mass and rotary inertia are added to the regular compiled mass properties. The default is true. Disabling the option does not remove regular TYPE=MASS elements from the calculation.

Parameters

Parameter	Status	Meaning
CONSIDER_POINT_MASSES	Optional	Include auxiliary native *POINTMASS objects; Boolean, default 1.

For an intentionally unsupported body, activate inertia relief inside a linear static loadcase. Here the previously defined FLIGHT_LOADS collector supplies the unbalanced external loading.

FEMaster and Abaqus example

```
*LOADCASE, TYPE=LINEARSTATIC
*LOADS
FLIGHT_LOADS
*INERTIARELIEF, CONSIDER_POINT_MASSES=1
*END
```

Do not add artificial fixed supports to this free-body example: the inertial terms balance the rigid-body resultant.

4.6.2.13 *REBALANCELOADS

*REBALANCELOADS adjusts the active external load system of a linear static analysis so that its resultant force and moment vanish. It is intended for a load that is theoretically self-equilibrated but contains a small numerical imbalance because of discretization, rounding, approximate surface selection, or equivalent-load construction.

This differs physically from inertia relief. *REBALANCELOADS assumes a theoretically self-equilibrated applied load and corrects its numerical residual. *INERTIARELIEF accepts a genuinely unbalanced free-body load and represents the rigid-body acceleration needed to balance it.

Rebalancing a load with a physical resultant removes that resultant and thereby defines a different loadcase; it does not supply support to an unsupported model.

Here the load collector BALANCED_LOADS is assumed to contain a physically self-equilibrated loading with small numerical residuals. *REBALANCELOADS corrects that imbalance during the linear static analysis; it must not be used to remove a real resultant load.

FEMaster and Abaqus example

```
*LOADCASE, TYPE=LINEARSTATIC
*SUPPORTS
BC
*LOADS
BALANCED_LOADS
*REBALANCELOADS
*END
```

4.6.3 Topology-weighted Linear Statics

The topology-static commands use element Fields to scale stiffness and rotate material directions without changing mesh connectivity.

`LINEARSTATICTOPO` extends linear static analysis with element-wise density and optional material-orientation Fields. It is intended for topology/sizing workflows in which design variables are supplied externally or updated between solver runs.

The density Field scales each element's constitutive stiffness by

$$s_e = \rho_e^p,$$

where ρ_e is the value selected by `*TOPODENSITY` and p is set by `*TOPOEXPONENT`. Thus an element with $\rho = 1$ retains its full stiffness, while intermediate density is penalized according to the exponent.

If an orientation Field is supplied, its three components are interpreted as element-wise Euler rotation angles. The resulting additional rotation is applied to the material coordinate system for the topology-aware solid response. The Field therefore changes material orientation, not element geometry.

The analysis also produces topology-related output such as density, compliance-related quantities, density sensitivity, volume, and orientation sensitivity where applicable.

4.6.3.1 *TOPODENSITY

*TOPODENSITY selects the scalar element Field ρ_e used by an active LINEARSTATICTOPO loadcase. FIELD is required; NAME is accepted as an alternative spelling. The referenced Field must use TYPE=ELEMENT and exactly one component.

The stiffness multiplier is

$$\rho_e^p.$$

The command does not automatically force values into the interval $[0, 1]$. An optimization based on physical density fractions therefore depends on an external workflow that supplies values consistent with that interpretation. Negative density values or inappropriate exponents can produce nonphysical stiffness scaling.

For a SIMP-style topology problem, $0 < \rho_{\min} \leq \rho_e \leq 1$ is commonly used to avoid exactly singular void elements, but the choice of minimum density belongs to the optimization strategy surrounding the FEM solve.

Compliance sensitivity contains factors proportional to $1/\rho_e$. Exactly zero density therefore introduces singular sensitivity behaviour in gradient-based topology workflows even though the associated zero-stiffness element has a direct conceptual interpretation.

Parameters

Parameter	Status	Meaning
FIELD / NAME	required	Scalar ELEMENT Field containing element densities.

A scalar element field supplies the design densities; the topology-weighted linear static loadcase then reads them through TOPODENSITY. The density values are not nodal displacements or material densities.

FEMaster and Abaqus example

```
*FIELD, NAME=RHO, TYPE=ELEMENT,
 COLS=1, FILL=ZERO
0, 0.8
1, 1.0

*LOADCASE, TYPE=LINEARSTATICTOPO
*TOPODENSITY, FIELD=RHO
*TOPOEXPONENT
3.0
*END
```

4.6.3.2 *TOPOORIENT

*TOPOORIENT selects a three-component ELEMENT Field containing additional material-orientation angles for LINEARSTATICTOPO. FIELD is required and NAME is accepted as an alias.

The three Field components are Euler rotation angles

$$(\theta_x, \theta_y, \theta_z)$$

in **radians**. They define the additional material rotation

$$R(\theta_x, \theta_y, \theta_z) = R_x(\theta_x)\, R_y(\theta_y)\, R_z(\theta_z).$$

This rotation acts in addition to the Section's base material orientation. It is therefore intended for optimization or parameter studies in which the preferred anisotropic material direction varies from element to element.

An orientation Field changes only directional constitutive response. It does not rotate the mesh and has no effect on a fully isotropic material law because isotropic stiffness is invariant under rotation.

The rotation order is significant because finite rotations about different axes do not commute. An external optimization or preprocessing tool that generates *TOPOORIENT values must therefore use the same $R_x R_y R_z$ convention and radians. For example, a second-component value of 0.2 means a rotation of 0.2 rad, not 0.2°.

Parameters

Parameter	Status	Meaning
FIELD / NAME	required	Three-component ELEMENT Field of Euler angles $(\theta_x, \theta_y, \theta_z)$ in radians.

This element field supplies the orientation parameters for the topology-weighted analysis. The field must exist before it is selected by TOPOORIENT.

FEMaster and Abaqus example

```
*FIELD, NAME=ORIENT,
 TYPE=ELEMENT, COLS=3, FILL=ZERO
0, 0.0, 0.0, 0.0
1, 0.0, 0.2, 0.0

*LOADCASE, TYPE=LINEARSTATICTOPO
*TOPODENSITY, FIELD=RHO
*TOPOORIENT, FIELD=ORIENT
*END
```

4.6.3.3 *TOPOEXPONENT

*TOPOEXPONENT sets the scalar penalization exponent p used by LINEARSTATICTOPO:

$$K_e(\rho_e) = \rho_e^p K_{e,0}$$

for the element stiffness contribution represented by the topology scaling.

With $p = 1$, stiffness varies linearly with density. Values $p > 1$ penalize intermediate densities increasingly strongly and are commonly used in SIMP-style topology optimization to drive the design toward a more nearly discrete solid/void distribution.

The exponent also affects compliance sensitivity. If an element compliance contribution before the density derivative is denoted by c_e, the density sensitivity scales conceptually with

$$\frac{\partial C}{\partial \rho_e} \propto -p\, \rho_e^{p-1} c_e.$$

The exact reported Field follows FEMaster's topology result definitions, while the sign and penalization dependence follow this standard stiffness-scaling idea.

Data lines

Entry	Meaning
exponent	Scalar density penalization exponent p.

The exponent is applied inside LINEARSTATICTOPO, together with the density field selected by TOPODENSITY. Increasing it penalizes intermediate relative densities more strongly.

FEMaster and Abaqus example

```
*TOPOEXPONENT
3.0
```

4.7 Output and Diagnostics

FEMaster writes analysis results in its own result formats and provides several diagnostic commands for inspecting matrices, constraints, and the assembled model. These diagnostics do not change the physical problem; they are intended for verification, comparison, and troubleshooting.

The native `.res` format preserves FEMaster Field domains and is the most complete textual result source. FEMR provides a seekable, compressed binary representation for efficient programmatic access. FRD output provides a CalculiX/CGX-compatible visualization format for supported nodal quantities.

Abaqus/CalculiX field-output cards such as `*NODE OUTPUT`, `*ELEMENT OUTPUT`, `*NODE FILE`, `*EL FILE`, and `*EL OUTPUT` select which FEMaster result Fields are written for the active step. The first explicit field-output request replaces that step's default result selection. `*NODE PRINT` and `*EL PRINT` are accepted for compatibility but remain ignored because FEMaster does not implement a separate text-print result path.

4.7.1 *OVERVIEW

*OVERVIEW prints a diagnostic summary of the current model without changing the analysis.

The overview is intended to answer basic model-verification questions before numerical results are interpreted: which Parts and Instances exist, how many nodes and elements are present, which named sets and surfaces are available, which materials and Sections have been defined, and which loads, supports, and constraints are present.

In models with a Part/Instance hierarchy or many named regions, the overview exposes missing Instances, empty sets, unexpected entity counts, and omitted property assignments independently of the linear solver.

A structurally plausible overview establishes only the reported model organization. It does not establish the physical consistency of material units, element orientations, shell normals, surface sides, load directions, or boundary conditions.

This root-level diagnostic prints a model summary for checking the assembled input. It does not choose results or alter the finite-element solution.

FEMaster and Abaqus example

```
*OVERVIEW
```

4.7.2 *REQUESTSTIFFNESS

*REQUESTSTIFFNESS requests stiffness-matrix output from supported native analyses. It is available for linear static, nonlinear static, and linear buckling loadcases. FILE optionally specifies the output filename; otherwise FEMaster generates a deterministic default name.

For linear static analysis, the exported matrix exposes symmetry, rank, sparsity, condition-related behaviour, and differences from another finite-element implementation. For nonlinear static analysis, the relevant stiffness is the tangent operator associated with the state at which the output is produced. In buckling analysis, the elastic stiffness can be exported separately from the geometric stiffness.

Matrix output is a diagnostic and verification feature distinct from the ordinary result Field output used for engineering post-processing.

Parameters

Parameter	Status	Meaning
FILE	optional	Stiffness-matrix output filename.

The matrix request belongs inside the analysis whose assembled stiffness matrix is to be exported. The FILE name controls the output destination.

FEMaster and Abaqus example

```
*LOADCASE, TYPE=LINEARSTATIC
*SUPPORTS
BC
*LOADS
SERVICE
*REQUESTSTIFFNESS, FILE=K_service.txt
*END
```

4.7.3 *REQUESTSTGEOM

*REQUESTSTGEOM requests geometric-stiffness output for LINEARBUCKLING. FILE optionally specifies the output filename or base name.

Linear buckling uses a stress-dependent geometric stiffness K_G generated by the reference preload state. Exporting K_G exposes the destabilizing contribution of the applied preload and, together with *REQUESTSTIFFNESS, permits independent inspection of the generalized buckling problem.

The command changes output only; it does not modify the preload or buckling eigenproblem.

Parameters

Parameter	Status	Meaning
FILE	optional	Geometric-stiffness output filename/base.

This example requests the elastic and geometric stiffnesses of the linear-buckling problem, including the preload-dependent geometric contribution.

FEMaster and Abaqus example

```
*LOADCASE, TYPE=LINEARBUCKLING
*SUPPORTS
BC
*LOADS
PRELOAD
*NUMEIGENVALUES
5
*REQUESTSTIFFNESS, FILE=K.txt
*REQUESTSTGEOM, FILE=KG.txt
*END
```

4.7.4 *CONSTRAINTSUMMARY

*CONSTRAINTSUMMARY enables diagnostic reporting of the constraint equations used by the active native loadcase.

When several constraint sources act simultaneously, support collectors, RBM suppression, couplings, ties, connectors, and other kinematic relations can all contribute equations. The summary reports their number and grouping.

In an unexpectedly singular or over-constrained model, the constraint groups can expose a duplicate coupling or a support that constrains a reference node twice more directly than the final sparse matrix.

*CONSTRAINTSUMMARY does not choose the enforcement method. *CONSTRAINTMETHOD separately determines whether the equations are handled by nullspace projection, elimination, or Lagrange multipliers where supported.

The report describes the equations and ranks used to enforce supports and coupling constraints. It is useful for diagnosing underconstraint or redundant equations.

FEMaster and Abaqus example

```
*LOADCASE, TYPE=LINEARSTATIC
*SUPPORTS
BC
*LOADS
SERVICE
*CONSTRAINTMETHOD, TYPE=NULLSPACE
*CONSTRAINTSUMMARY
*END
```

4.7.5 Abaqus and CalculiX output-request keywords

FEMaster accepts the common field-output request cards `*NODE OUTPUT`, `*ELEMENT OUTPUT`, `*NODE FILE`, `*EL FILE`, and `*EL OUTPUT` inside an active step/loadcase. These spellings share one internal request mechanism: FEMaster does not distinguish between ASCII and binary output semantics at the input-card level. The selected result writer determines the actual result-file format.

The first explicit field-output request replaces the analysis step's predefined frame-output set. Subsequent node/element output cards add further requested Fields. Common Abaqus/CalculiX variable names are translated to FEMaster result quantities, for example `U` to displacement, `RF` to reaction forces, `S` to stress, and `E` to strain. Compact step metadata that defines the analysis result itself is written independently of this filtering; eigenfrequency analyses always retain eigenvalues, angular eigenfrequencies, frequencies, and modal participation, while buckling analyses always retain buckling factors.

The Abaqus container `*OUTPUT, FIELD` is accepted for compatibility but does not itself change the request set. The actual selection is performed by the following node/element output cards.

`*NODE PRINT` and `*EL PRINT` remain compatibility-only keywords. Their variables are accepted and ignored because FEMaster does not provide a separate textual print-output mechanism.

4.7.6 *NODE OUTPUT

*NODE OUTPUT selects nodal result quantities for the active step. When it is the first explicit output card, the step defaults are cleared before the listed variables are added.

The nodal output request is attached to the current analysis step. U requests displacements and RF requests reaction forces, provided those quantities are available for that procedure.

FEMaster and Abaqus example

```
*OUTPUT, FIELD
*NODE OUTPUT
U, RF
```

The CalculiX-compatible *NODE FILE keyword accepts the same nodal field list and configures the same output request.

4.7.7 *ELEMENT OUTPUT

*ELEMENT OUTPUT selects element-derived result quantities for the active step. It uses the same request set as *EL FILE; the spelling does not select a different FEMaster result format.

The element output request selects derived fields such as stress S and strain E for the active step. Output availability still depends on the elements and procedure.

FEMaster and Abaqus example

```
*OUTPUT, FIELD
*ELEMENT OUTPUT
S, E
```

The CalculiX-compatible *EL FILE keyword accepts the same element field list and configures the same output request.

4.7.8 *NODE FILE

*NODE FILE is the CalculiX-style alias of *NODE OUTPUT. It selects the listed nodal result quantities for the active step but does not force creation of a separate ASCII node-results file.

This CalculiX-compatible alias selects the same nodal fields as NODE OUTPUT. It does not create a separate unrelated output stream.

FEMaster and Abaqus example

```
*NODE FILE
U, RF
```

The equivalent *NODE OUTPUT keyword accepts the same field list. The surrounding *OUTPUT, FIELD is an optional compatibility container and does not itself select fields.

4.7.9 *EL FILE

*EL FILE is the CalculiX-style alias of *ELEMENT OUTPUT. It selects the listed element-derived result quantities without changing the configured FEMaster result writer.

This alias selects the same element result fields as ELEMENT OUTPUT. The available variables depend on the active procedure and element formulations.

FEMaster and Abaqus example

```
*EL FILE
S, E
```

The equivalent *ELEMENT OUTPUT keyword accepts the same field list. The surrounding *OUTPUT, FIELD is an optional compatibility container and does not itself select fields.

4.7.10 *EL OUTPUT

*EL OUTPUT selects element-derived Fields using the same request mechanism as *ELEMENT OUTPUT and *EL FILE. The active result writer determines the output-file format.

The following card requests stress and strain in the active step:

FEMaster and Abaqus example

```
*EL OUTPUT
S, E
```

4.7.11 *NODE PRINT

*NODE PRINT is accepted and ignored. It does not modify the active field-output request set and does not generate an Abaqus/CalculiX text print table.

This input is accepted for file compatibility but currently ignored. It neither changes the frame output selection nor produces an additional text-results table.

FEMaster and Abaqus example

```
*NODE PRINT
U, RF
```

4.7.12 *EL PRINT

*EL PRINT is accepted and ignored in the same way as *NODE PRINT. It neither filters FEMaster result Fields nor generates a separate textual element-results table.

This command is parsed for compatibility but has no effect on the result fields. Use ELEMENT OUTPUT or EL FILE to request actual element output.

FEMaster and Abaqus example

```
*EL PRINT
S, E
```

4.7.13 Native `.res` files

The native `.res` file is organized into loadcase and Field blocks. A loadcase header has the form

Syntax

```
LC <id>
```

and is followed by the Fields produced by that analysis.

Indexed Field storage

All standard result Fields associated with model entities are written in indexed form. Every row therefore contains one or more explicit index columns followed by the Field values. The header states both counts:

Syntax

```
FIELD, NAME=<name>, TYPE=<type>,
INDEX_COLS=<nindex>, VALUE_COLS=<nvalue>, ROWS=<rows>
<index-1> ... <index-n> <value-1> ... <value-m>
...
END FIELD
```

The indices identify the corresponding model entity rather than an implicit row number. Results therefore remain traceable to the user-visible node and element labels even when the model contains Parts and multiple Instances.

Entities belonging to the implicit default Instance retain their unqualified local identifier, for example `8`. Entities belonging to an explicitly named Instance use a qualified identifier of the form

Syntax

```
<instance>.<local-id>
```

such as `TRUSS_A.8` or `TRUSS_B.10`. The same convention is used for node and element identifiers.

The number and meaning of the index columns depend on the Field domain:

Field type	Index columns	Meaning
`NODE`	1	node identifier
`ELEMENT`	1	element identifier
`ELEMENT_NODAL`	2	element identifier, local node index
`ELEMENT_IP`	2	element identifier, local integration-point index
`ELEMENT_MP`	2	element identifier, local material-point index

Local node, integration-point, and material-point indices are zero based. User node and element identifiers are non-negative labels and may themselves also start at `0`; these are separate concepts.

A typical nodal Field therefore looks like:

Syntax

```
FIELD, NAME=DISPLACEMENT, TYPE=NODE,
INDEX_COLS=1, VALUE_COLS=6, ROWS=4
               1    0.000000e+00    0.000000e+00 ...
      TRUSS_A.8    4.761905e-02   -4.761905e-02 ...
       TRUSS_B.1    0.000000e+00    0.000000e+00 ...
      TRUSS_B.10    4.761905e-02    0.000000e+00 ...
END FIELD
```

For `ELEMENT_NODAL`, `ELEMENT_IP`, and `ELEMENT_MP`, the first index is the semantic element identifier and the second is the zero-based local position within that element. This keeps the external result format independent of any internal element ordering.

Common result quantities

Typical Fields depend on analysis type:

Analysis	Typical results
Linear static	Displacement, strain, stress, shell top/bottom stresses and resultants where applicable, external forces, reactions, section forces, and related recovered quantities.
Nonlinear static	Accepted-increment displacement, stress, reaction-force, and load-factor states plus final displacement, strain, stress, internal/external forces, and reactions.
Eigenfrequency	Requested mode-frame Fields plus always-written step metadata: eigenvalues, eigenfrequencies/frequencies, and modal participation.
Linear buckling	Requested buckling-mode frame Fields plus always-written buckling factors.
Linear transient	Displacement, velocity, and acceleration at written time frames.
Linear harmonic	Real and imaginary displacement, stress, and strain amplitudes for the requested excitation frequencies.

Generalized nodal motion and force vectors use the six-component nodal order `Ux, Uy, Uz, Rx, Ry, Rz`. Stress and strain tensors instead use the engineering order

$$(x, y, z, yz, zx, xy).$$

The component meaning of raw result data therefore depends on whether the Field contains generalized nodal quantities or stress/strain tensors.

Structural stress and strain recovery

For solid elements, FEMaster evaluates constitutive stress and strain at the element's actual material integration points. Result recovery extrapolates those values to the element's local nodes with a topology-specific interpolation/extrapolation matrix; the global nodal result is then assembled from the contributions of incident elements. A reduced-integration element can only recover the spatial variation represented by its integration-point data (for example, a single-point `C3D8R` supplies a constant recovered value per element). Extrapolated element-nodal values and averaged nodal values therefore need not equal the original integration-point values.

The distinction is particularly important at a material interface, a stress discontinuity, a sharp corner, or a strongly distorted mesh: nodal averaging can smooth physically different element-side values, and extrapolation may produce extrema outside the integration-point range. When comparing results with another solver or experimental stress measures, compare the same stress measure and recovery location, not merely a field with the same name. Local `ELEMENT\_IP`, `ELEMENT\_NODAL`, and global `NODE` domains retain different indexing and recovery semantics in the native `.res` format.

FEMR version 1 stores `NODE` and `ELEMENT` result fields but does not serialize `ELEMENT\_NODAL`, `ELEMENT\_IP`, or `ELEMENT\_MP` fields. The writer skips unsupported domains with a warning; use `.res` when the local-field domain is needed. The absence of a local field in FEMR does not imply that FEMaster did not evaluate that quantity internally.

Harmonic results

A complex harmonic quantity is written as separate real and imaginary components:

$$\hat{q} = q_{\text{real}} + i q_{\text{imag}}.$$

Amplitude and phase can be reconstructed from

$$|\hat{q}| = \sqrt{q_{\text{real}}^2 + q_{\text{imag}}^2}, \qquad \varphi = \operatorname{atan2}(q_{\text{imag}}, q_{\text{real}}).$$

Harmonic results distinguish the real part at a chosen phase, the complex amplitude magnitude, and signed components. These quantities are not interchangeable. Each excitation-frequency sample retains its own real/imaginary displacement, stress, and strain Fields. An ascending, one-based sweep index is appended to the native Field names (for example, `DISPLACEMENT\_REAL\_1` and `DISPLACEMENT\_IMAG\_1`); the physical excitation frequency remains the result-frame value. This also keeps samples with repeated frequency values distinct in `.res` output. The node-wise magnitude of the complex displacement vector can be reconstructed from the first three translational components of the paired real and imaginary Fields. The console's response norm instead combines all active degrees of freedom of the full displacement vectors, including rotational degrees of freedom where present, and is not the maximum node-wise displacement amplitude.

The FRD writer preserves this separation rather than discarding the imaginary component. Harmonic displacement Fields are written as `DISPREAL` and `DISPIMAG`; harmonic stress as `STRREAL` and `STRIMAG`; and harmonic strain as `STRNREAL` and `STRNIMAG`. This permits direct visualization or reconstruction of the complex response from the paired FRD blocks.

4.7.14 FEMR binary result format

FEMR version 1 is FEMaster's seekable binary result container. Multi-byte integers and floating-point values are little-endian. Unknown chunks are skipped using their stored size, allowing compatible format extensions.

Binary output is selected explicitly, for example:

```
FEMaster model.inp --output-format femr
FEMaster model.inp --output-format res frd femr
```

FEMR version 1 always compresses each independently loadable field-data chunk with LZ4. There is no compression setting in the command-line interface.

4.7.14.1 Chunk envelope

Every chunk begins with a 32-byte envelope containing, in order:

- a four-byte chunk code;
- a `uint32` compression code (0 for none, 1 for an LZ4 block);
- `uint64` stored and uncompressed payload sizes;
- a `uint32` CRC32 of the uncompressed payload; and
- a reserved `uint32`, which is zero in version 1.

Only `FDAT` is compressed in version 1. Each field is a separate LZ4 block, so loading one field never decompresses another field.

4.7.14.2 Chunks

HEAD Must be first. It stores the `FEMR` magic, major and minor version, byte order, scalar width, default compression and reserved data.

INST Stores each instance identifier once together with its length-prefixed UTF-8 name.

MESH Starts with `uint64` node and element counts. Each node stores its dense/global, instance and part-local identifiers as three `int32` values, followed by three `float64` coordinates. Each element likewise stores its dense/global, instance and part-local identifiers, followed by a length-prefixed UTF-8 type name, node count and dense/global node identifiers.

LCAS Stores the load-case identifier and step type.

FRAM Stores load-case identifier, dense frame identifier and value. Eigenfrequency and buckling frame identifiers are derived from the numeric field-name suffix, so equal spectral values remain distinct frames.

FMET Stores field identifier, load-case and frame identifiers, name, domain, scalar width, row count and component count.

FDAT Stores the field identifier followed by its dense row-major scalar array.

CSUM Must be last. Its payload is the CRC32 of every preceding chunk envelope and stored payload byte.

Strings use a `uint16` byte length followed by UTF-8 bytes. Version 1 supports only the field domains `UNKNOWN` (0), `NODE` (1), and `ELEMENT` (2). Writers skip fields from other domains with a warning; readers reject unsupported domain codes found in a version 1 file.

4.7.14.3 Stable mesh and field indexing

MESH ordering defines the stable dense result indexing. Node entry i maps to row i of every NODE field, and element entry i maps to row i of every ELEMENT field. Node and element identifiers are nevertheless stored explicitly; they must not be reconstructed from element connectivity. The accompanying instance and part-local identifiers preserve semantic labels such as `17` and `bolt.17`.

4.7.14.4 Lazy access

A reader scans only chunk envelopes and the small metadata chunks. It seeks over MESH and FDAT payloads and retains their offsets. Mesh or field payloads are read, decompressed and checked only when requested.

```
import femaster_api as femaster

with femaster.open_results("model.femr") as results:
    stress = results.loadcase(1).frame(20).field("STRESS")
    mesh = results.mesh
```

4.7.15 FRD visualization output

FRD provides a CalculiX/CGX-compatible representation of the mesh and supported nodal results. It is intended primarily for visualization. Data that do not naturally map to FRD nodal blocks—for example some element-local, integration-point, material-point, scalar-history, or participation quantities—remain available in the native result representation.

Typical mappings include:

FEMaster result	FRD block	Content
Displacement / mode / buckling vector	`DISP`	Translational and supported generalized displacement components.
Harmonic displacement, real / imaginary	`DISPREAL` / `DISPIMAG`	Separate components of the complex harmonic displacement.
Velocity	`VELO`	Nodal velocity components.
Acceleration	`ACCE`	Nodal acceleration components.
Reaction force	`FORC`	Generalized reaction-force components; nonlinear static writes them for every accepted increment and for the final result frame.
Stress	`STRESS`	Nodal recovered stress components.
Harmonic stress, real / imaginary	`STRREAL` / `STRIMAG`	Separate components of the complex harmonic stress.
Strain	`TOSTRAIN`	Nodal recovered strain components.
Harmonic strain, real / imaginary	`STRNREAL` / `STRNIMAG`	Separate components of the complex harmonic strain.
Shell resultants	`SHR`	Membrane, bending, and transverse-shear resultants.

FRD requires numeric node identifiers and therefore cannot use the qualified string identifiers employed by `.res`. FEMaster assigns every written node the deterministic FRD identifier

$$n_{\mathrm{FRD}} = 10^8 \, i_{\mathrm{instance}} + n_{\mathrm{local}},$$

where i_{instance} is the numeric Instance identifier and n_{local} is the original Part-local node ID. The local node ID must be smaller than 10^8. The same FRD node numbering is used consistently for coordinates, element connectivity, and nodal result blocks. Thus identifiers such as `100000001` and `200000001` represent the same local node label in different Instances; they are output identifiers and do not replace the semantic `INSTANCE.local-id` notation used in `.res`.

User node and element IDs in FEMaster may include `0`. FRD numbering is an output-format concern and does not impose a positive-ID restriction on the FEMaster input model.

Higher-order element connectivity is reordered where necessary to satisfy FRD conventions. For shell resultants, an explicitly assigned Section orientation defines the local output basis. Without an explicit orientation, FEMaster uses a deterministic tangent basis derived from the shell geometry.

Models containing nodes and nodal features but no supported structural elements remain valid FRD models. In this case FEMaster writes the nodal geometry and applicable nodal result fields and simply omits the element-connectivity block.

Transient FRD frames use physical time. Eigenfrequency and buckling frames use the corresponding spectral value where supported by the writer.

4.8 Keyword Index

This section indexes the keywords accepted by the unified FEMaster input language. The compatibility column distinguishes supported Abaqus-compatible syntax from FEMaster extensions and legacy aliases; it does not imply that every option offered by another solver is implemented. Detailed keyword pages explain supported alternatives.

Command names are displayed in the conventional spaced form used in the manual, for example `*END PART`, `*SOLID SECTION`, and `*STEADY STATE DYNAMICS`. `*INERTIALOAD` is shown with the exact native spelling currently accepted by FEMaster.

Table 4.4: Input keywords and their canonical documentation locations.

Keyword	Compatibility	Canonical topic
`*HEADING`	Abaqus-compatible	Input Structure
`*MODEL`	FEMaster extension	Input Structure; optional native root marker
`*PART`	Abaqus-compatible	Model Definition
`*END PART`	Abaqus-compatible	Model Definition
`*ASSEMBLY`	Abaqus-compatible	Model Definition
`*END ASSEMBLY`	Abaqus-compatible	Model Definition
`*INSTANCE`	Abaqus-compatible	Model Definition
`*END INSTANCE`	Abaqus-compatible	Model Definition
`*NODE`	Abaqus-compatible	Model Definition
`*ELEMENT`	Compatible + extensions	Model Definition; `*ELEMENT` formulations
`*NSET`	Abaqus-compatible	Model Definition
`*ELSET`	Abaqus-compatible	Model Definition
`*SURFACE`	Abaqus-compatible	Model Definition
`*SFSET`	FEMaster extension	Model Definition
`*MATERIAL`	Abaqus-compatible	Materials
`*ELASTIC`	Abaqus-compatible	Materials
`*PLASTIC`	Abaqus-compatible	Materials; isotropic J2 plasticity
`*HYPERELASTIC`	Abaqus-compatible	Materials
`*DENSITY`	Abaqus-compatible	Materials
`*EXPANSION`	Abaqus-compatible	Materials
`*THERMALEXPANSION`	Legacy alias	Materials; legacy alias of `*EXPANSION`
`*PROFILE`	FEMaster extension	Sections and Element Properties
`*SOLID SECTION`	Abaqus-compatible	Sections and Element Properties
`*SHELL SECTION`	Abaqus-compatible	Sections and Element Properties
`*BEAM SECTION`	FEMaster extension	Sections and Element Properties
`*BEAM GENERAL SECTION`	Abaqus-compatible	Sections and Element Properties
`*TRUSS SECTION`	FEMaster extension	Sections and Element Properties
`*POINTMASS`	FEMaster extension	Sections and Element Properties; NSET-based concentrated properties
`*MASS`	Abaqus-compatible	Sections and Element Properties; `MASS` point elements
`*ROTARY INERTIA`	Abaqus-compatible	Sections and Element Properties; `ROTARYI` point elements
`*SPRING`	Abaqus-compatible	Sections and Element Properties; linear `SPRING1` point elements
`*ORIENTATION`	Abaqus-compatible	Coordinate Systems and Orientations
`*TRANSFORM`	Abaqus-compatible	Coordinate Systems and Orientations
`*FIELD`	FEMaster extension	Fields
`*NORMAL`	FEMaster extension	Fields
`*INITIALCONDITION`	FEMaster extension	Fields; initial nodal temperature and velocity
`*INITIALCONDITIONS`	Alias	Fields; same syntax as `*INITIALCONDITION`
`*AMPLITUDE`	Abaqus-compatible	Loads and Boundary Conditions

Keyword	Compatibility	Canonical topic
*SUPPORT	FEMaster extension	Loads and Boundary Conditions
*BOUNDARY	Abaqus-compatible	Loads and Boundary Conditions
*CLOAD	Compatible + extensions	Loads and Boundary Conditions
*DLOAD	Compatible + extensions	Loads and Boundary Conditions; Abaqus labels and FEMaster vector form
*PLOAD	FEMaster extension	Loads and Boundary Conditions
*DSLOAD	Abaqus-compatible	Loads and Boundary Conditions
*VLOAD	FEMaster extension	Loads and Boundary Conditions
*INERTIALOAD	FEMaster extension	Loads and Boundary Conditions
*TEMPERATURE	FEMaster extension	Loads and Boundary Conditions
*HEATFLUX	FEMaster extension	Loads and Boundary Conditions
*CONVECTION	FEMaster extension	Loads and Boundary Conditions
*COUPLING	Compatible + extensions	Constraints and Contact
*KINEMATIC	Abaqus-compatible	Constraints and Contact; child of scoped *COUPLING
*DISTRIBUTING	Abaqus-compatible	Constraints and Contact; child of scoped *COUPLING
*EQUATION	Abaqus-compatible	Constraints and Contact
*MPC	FEMaster extension	Constraints and Contact; two-node kinematics
*TIE	FEMaster extension	Constraints and Contact
*RIGID BODY	CalculiX-compatible	Constraints and Contact
*CONNECTOR	FEMaster extension	Constraints and Contact
*RBM	FEMaster extension	Constraints and Contact
*CONTACT	FEMaster extension	Constraints and Contact
*LOADCASE	FEMaster extension	Analysis Procedures
*SUPPORTS	FEMaster extension	Analysis Procedures
*LOADS	FEMaster extension	Analysis Procedures
*END	FEMaster extension	Analysis Procedures
*STEP	Abaqus-compatible	Analysis Procedures
*STATIC	Abaqus-compatible	Analysis Procedures
*FREQUENCY	Abaqus-compatible	Analysis Procedures
*BUCKLE	Abaqus-compatible	Analysis Procedures
*DYNAMIC	Abaqus-compatible	Analysis Procedures
*STEADY STATE DYNAMICS	Abaqus-compatible	Analysis Procedures
*END STEP	Abaqus-compatible	Analysis Procedures
*SOLVER	FEMaster extension	Solver and Numerical Controls; in Abaqus *STEP after the procedure
*CONSTRAINTMETHOD	FEMaster extension	Solver and Numerical Controls
*NONLINEAR	FEMaster extension	Solver and Numerical Controls
*TIME	FEMaster extension	Solver and Numerical Controls
*NEWMARK	FEMaster extension	Solver and Numerical Controls
*DAMPING	FEMaster extension	Solver and Numerical Controls
*FREQUENCIES	FEMaster extension	Solver and Numerical Controls
*NUMEIGENVALUES	FEMaster extension	Solver and Numerical Controls
*SIGMA	FEMaster extension	Solver and Numerical Controls
*WRITEEVERY	FEMaster extension	Solver and Numerical Controls
*INITIALVELOCITY	FEMaster extension	Solver and Numerical Controls
*INERTIARELIEF	FEMaster extension	Solver and Numerical Controls
*REBALANCELOADS	FEMaster extension	Solver and Numerical Controls
*OVERVIEW	FEMaster extension	Output and Diagnostics
*REQUESTSTIFFNESS	FEMaster extension	Output and Diagnostics
*REQUESTSTGEOM	FEMaster extension	Output and Diagnostics
*CONSTRAINTSUMMARY	FEMaster extension	Output and Diagnostics
*OUTPUT	Abaqus-compatible	Output; FIELD container accepted, selection is made by child node/element output cards
*NODE OUTPUT	Abaqus-compatible	Output; selects nodal result Fields for the active step
*ELEMENT OUTPUT	Abaqus-compatible	Output; selects element-derived result Fields for the active step

Keyword	Compatibility	Canonical topic
*NODE FILE	CalculiX-compatible	Output; CalculiX-style alias selecting nodal result Fields
*EL FILE	CalculiX-compatible	Output; CalculiX-style alias selecting element-derived result Fields
*EL OUTPUT	CalculiX-compatible	Output; element-derived frame Fields
*NODE PRINT	Accepted, ignored	Output; accepted and ignored
*EL PRINT	Accepted, ignored	Output; accepted and ignored
*TOPODENSITY	FEMaster extension	Topology-weighted Linear Statics
*TOPOORIENT	FEMaster extension	Topology-weighted Linear Statics
*TOPOEXPONENT	FEMaster extension	Topology-weighted Linear Statics

4.8.1 Supported element formulations

The complete list of supported element names, Abaqus-compatible aliases, node connectivity, numerical formulations, and limitations is provided directly under `*ELEMENT` in Section 4.2. The element-type table and individual descriptions are maintained there as the canonical reference.

4.8.2 Coverage notes

Abaqus-compatible and CalculiX-compatible describe supported input spellings and data subsets, not full behavioral equivalence to those solvers. For example, `*DLOAD` accepts body accelerations, force-density components, element-face pressure labels, and a FEMaster-specific vector surface-traction form. The detailed keyword page specifies the valid data layouts.

`*COUPLING`, `*KINEMATIC`, and `*DISTRIBUTING` support Abaqus-compatible scoped coupling definitions with inclusive degree-of-freedom ranges. An additional FEMaster direct form uses `TYPE=KINEMATIC` or `TYPE=STRUCTURAL` and six selector values. Other Abaqus-specific coupling options are outside the supported subset.

`*EQUATION` and concentrated-element properties have Abaqus-compatible input forms. The supported point-element subset consists of isotropic `MASS`, diagonal `ROTARYI`, and constant linear point-to-ground `SPRING1`. Off-diagonal rotary inertia, oriented point properties, `SPRING2`/`SPRINGA`, nonlinear springs, and frequency- or field-dependent spring data are outside the supported subset. The FEMaster extension `*POINTMASS` combines diagonal properties directly on an NSET.

The optional native `*MODEL` marker is documented as input-file framing rather than as a separate level in the model-definition hierarchy.

The `*PLASTIC` material card is available with Abaqus-compatible syntax, with the same tabulated yield-stress/equivalent-plastic-strain rows. `*SOLVER` is a FEMaster extension accepted inside `*STEP` after the active procedure has been declared. Other numerical-control keywords have the scopes and restrictions documented on their individual reference pages.

Field-output cards `*NODE OUTPUT`, `*ELEMENT OUTPUT`, `*NODE FILE`, `*EL FILE`, and `*EL OUTPUT` actively configure the per-step frame-result selection. The first explicit request replaces the step's frame-output defaults. Compact analysis metadata such as eigenvalues, frequencies, modal participation, and buckling factors is always written when the corresponding analysis provides it. `*OUTPUT, FIELD` is accepted as a compatibility container only, while `*NODE PRINT` and `*EL PRINT` are accepted and ignored.

The index, detailed thematic documentation, and applicable complete examples together describe the supported input keywords and element formulations. This keeps the manual aligned with the syntax users can actually run.

5 Examples

The examples in this chapter combine individual keywords into complete modelling workflows. Each physical problem is first shown as a complete, supported Abaqus-compatible input deck. Alternative FEMaster-specific definitions appear below the main example when they provide a useful additional modelling approach.

Alternative blocks contain only the cards that change; they are replacement fragments, not independent complete input files. They may therefore rely on nodes, sets, materials and other definitions retained from the primary example. The physical problem is the same unless an explicitly mentioned control, such as damping or nonlinear stepping, changes the analysis.

Several examples intentionally use node or element ID `0`. This demonstrates that zero is a valid FEMaster user ID; user IDs need not begin at one.

5.1 Linear static truss

The complete introductory truss example, including both input styles, the solver invocation, and the analytical displacement and reaction checks, is given in Section 2.4. The following examples extend that workflow to additional model and analysis features.

5.2 Nonlinear axial response with tabulated J2 plasticity

This two-node bar illustrates the same `*ELASTIC`/`*PLASTIC` material definition in the native and supported Abaqus parsers. The initial yield force for the $100\,\mathrm{mm}^2$ section is $A\sigma_y = 25\,000$ N. The prescribed 27 000 N tensile force is above this initial threshold; the nonlinear solution therefore requires constitutive history rather than merely using the elastic stiffness. The table gives *equivalent plastic strain*, not total strain. No mass density is required for this quasistatic case.

Example

```
*NODE
1,    0., 0., 0.
2, 1000., 0., 0.
*ELEMENT, TYPE=T3D2, ELSET=BAR
1, 1, 2
*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3
*PLASTIC
250., 0.
300., 0.02
*SOLID SECTION, ELSET=BAR,
 MATERIAL=STEEL
100.
*STEP, NLGEOM=YES, INC=80
*STATIC
0.125, 1., 1e-5, 0.25
*SOLVER, DEVICE=CPU, METHOD=DIRECT
*BOUNDARY
1, 1, 3, 0.
2, 2, 3, 0.
*CLOAD
2, 1, 27000.
*END STEP
```

FEMaster truss section and nonlinear loadcase

The following cards replace the corresponding definitions in the complete example above; unchanged geometry and material definitions are retained.

```
*ELEMENT, TYPE=T3, ELSET=BAR
1, 1, 2
*PLASTIC, TYPE=J2
250., 0.
300., 0.02
*TRUSS SECTION, ELSET=BAR, MATERIAL=STEEL
100.
*SUPPORT, SUPPORT_COLLECTOR=BC
1, 0., 0., 0., NAN, NAN, NAN
2, NAN, 0., 0., NAN, NAN, NAN
*CLOAD, LOAD_COLLECTOR=TENSION
2, 27000., 0., 0., 0., 0., 0.
*LOADCASE, TYPE=NONLINEARSTATIC
*SUPPORTS
BC
*LOADS
TENSION
*NONLINEAR, INCREMENTS=8, MAXITER=40, TOL=1e-5
*SOLVER, DEVICE=CPU, METHOD=DIRECT
*END
```

Because the Abaqus deck uses geometric nonlinearity, compare results using the same load, deformation, constitutive stress measure, and nonlinear kinematics when checking against another implementation.

5.3 Reusable Part and translated Instance

This example demonstrates the optional Part/Assembly hierarchy. The material is defined once at model level. The reusable rod topology and Section assignment are defined inside a Part. One Instance translates the complete Part by 500 mm in x. Assembly-level node sets then identify the local endpoint IDs on that particular Instance.

The Part coordinates remain local to the reusable component. The Instance placement determines the actual position of that copy in the assembled structure. If a second Instance were added with a different placement, both copies could still use local node IDs 0 and 1 without manual renumbering.

Example

```
*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3

*PART, NAME=ROD
*NODE
0,   0., 0., 0.
1, 500., 0., 0.
*ELEMENT, TYPE=T3D2, ELSET=BAR
0, 0, 1
*SOLID SECTION, ELSET=BAR,
 MATERIAL=STEEL
50.
*END PART

*ASSEMBLY, NAME=ASSEMBLY
*INSTANCE, NAME=ROD_A, PART=ROD
500., 0., 0.
*END INSTANCE
*NSET, NSET=LEFT, INSTANCE=ROD_A
0
*NSET, NSET=RIGHT, INSTANCE=ROD_A
1
*END ASSEMBLY

*STEP
*STATIC
*BOUNDARY
LEFT, 1, 3, 0.
RIGHT, 2, 3, 0.
*CLOAD
RIGHT, 1, 500.
*END STEP
```

FEMaster truss and reusable collectors

The following cards replace the corresponding definitions in the complete example above; unchanged geometry and material definitions are retained.

```
*ELEMENT, TYPE=T3, ELSET=BAR
0, 0, 1
*TRUSS SECTION, ELSET=BAR, MATERIAL=STEEL
50.
*SUPPORT, SUPPORT_COLLECTOR=BC
LEFT, 0., 0., 0., NAN, NAN, NAN
RIGHT, NAN, 0., 0., NAN, NAN, NAN
*CLOAD, LOAD_COLLECTOR=P
RIGHT, 500., 0., 0., 0., 0., 0.
*LOADCASE, TYPE=LINEARSTATIC
*SUPPORTS
BC
*LOADS
P
*END
```

The left endpoint of the placed rod lies at $x = 500$ mm and the right endpoint at $x = 1000$ mm. No change to the Part's local coordinates was required.

5.4 Homogeneous shell cantilever

A rectangular shell panel is clamped along one edge and loaded by two equal transverse nodal forces at the free edge. The native model uses the current four-node finite-rotation shell `MITC4FRT`. The Abaqus label `S4` is read as the same FEMaster FRT shell family. Both Sections represent a homogeneous 2 mm shell.

This example is intentionally simple enough that shell normals and load direction can be checked visually. Nodes are ordered counter-clockwise when viewed from the positive shell-normal side. The free-edge load acts in negative global z.

Example

```
*NODE
0,    0.,   0., 0.
1, 1000.,   0., 0.
2, 1000., 500., 0.
3,    0., 500., 0.
*ELEMENT, TYPE=S4, ELSET=PANEL
0, 0, 1, 2, 3
*NSET, NSET=ROOT
0, 3
*NSET, NSET=TIP
1, 2

*MATERIAL, NAME=ALUMINIUM
*ELASTIC
70000., 0.33
*SHELL SECTION, ELSET=PANEL,
 MATERIAL=ALUMINIUM
2.0, 5

*STEP
*STATIC
*BOUNDARY
ROOT, 1, 6, 0.
*CLOAD
TIP, 3, -50.
*END STEP
```

FEMaster finite-rotation shell and loadcase

The following cards replace the corresponding definitions in the complete example above; unchanged geometry and material definitions are retained.

```
*ELEMENT, TYPE=MITC4FRT, ELSET=PANEL
0, 0, 1, 2, 3
*SHELL SECTION, ELSET=PANEL, MATERIAL=ALUMINIUM, TYPE=INTEGRATED
2.0
*SUPPORT, SUPPORT_COLLECTOR=BC
ROOT, 0., 0., 0., 0., 0., 0.
*CLOAD, LOAD_COLLECTOR=P
TIP, 0., 0., -50., 0., 0., 0.
*LOADCASE, TYPE=LINEARSTATIC
*SUPPORTS
BC
*LOADS
P
*END
```

Because `*CLOAD` applied to a node set acts on every node in that set, the total transverse force is -100 N in both decks. A total force of -50 N corresponds to -25 N at each free-edge node or to one -50 N reference-node load transferred through a distributing coupling.

5.5 Solid cube under pressure

The solid example demonstrates an element-face surface and scalar pressure loading. The main input uses `*DSLOAD` with `P`; the FEMaster alternative uses `*PLOAD`. In both cases the load direction follows the selected surface orientation.

Example

```
*NODE
0, 0., 0., 0.
1, 1., 0., 0.
2, 1., 1., 0.
3, 0., 1., 0.
4, 0., 0., 1.
5, 1., 0., 1.
6, 1., 1., 1.
7, 0., 1., 1.
*ELEMENT, TYPE=C3D8, ELSET=CUBE
0, 0, 1, 2, 3, 4, 5, 6, 7
*NSET, NSET=BASE
0, 1, 2, 3
*SURFACE, NAME=LOADED_FACE,
 TYPE=ELEMENT
CUBE, S2

*MATERIAL, NAME=MAT
*ELASTIC
10000., 0.3
*SOLID SECTION, ELSET=CUBE, MATERIAL=MAT

*STEP
*STATIC
*BOUNDARY
BASE, 1, 3, 0.
*DSLOAD
LOADED_FACE, P, 1.0
*END STEP
```

FEMaster pressure and support collectors

The following cards replace the corresponding definitions in the complete example above; unchanged geometry and material definitions are retained.

```
*SUPPORT, SUPPORT_COLLECTOR=BC
BASE, 0., 0., 0., NAN, NAN, NAN
*PLOAD, LOAD_COLLECTOR=P
LOADED_FACE, 1.0
*LOADCASE, TYPE=LINEARSTATIC
*SUPPORTS
BC
*LOADS
P
*END
```

For `C3D8`, the physical face represented by `S2` and its normal direction determine the pressure sign. The surface side is part of the model definition.

5.6 Eigenfrequency analysis

The modal example uses a two-element truss with distributed material density and requests the lowest five eigenpairs. Only axial motion is left free. It demonstrates distributed structural mass; the following complete example separately demonstrates the shared concentrated `MASS` element/property syntax.

The natural frequencies are governed by

$$K\phi_i = \omega_i^2 M\phi_i.$$

Changing density therefore changes the frequencies even though the static stiffness is unchanged.

Example

```
*NODE
0,    0., 0., 0.
1,  500., 0., 0.
2, 1000., 0., 0.
*ELEMENT, TYPE=T3D2, ELSET=BAR
0, 0, 1
1, 1, 2
*NSET, NSET=FIXED
0
*NSET, NSET=GUIDED
1, 2
*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3
*DENSITY
7.85e-9
*SOLID SECTION, ELSET=BAR, MATERIAL=STEEL
100.

*STEP, PERTURBATION
*FREQUENCY
5
*BOUNDARY
FIXED, 1, 3, 0.
GUIDED, 2, 3, 0.
*END STEP
```

FEMaster frequency extraction

The following cards replace the corresponding definitions in the complete example above; unchanged geometry and material definitions are retained.

```
*ELEMENT, TYPE=T3, ELSET=BAR
0, 0, 1
1, 1, 2
*TRUSS SECTION, ELSET=BAR, MATERIAL=STEEL
100.
*SUPPORT, SUPPORT_COLLECTOR=BC
FIXED, 0., 0., 0., NAN, NAN, NAN
GUIDED, NAN, 0., 0., NAN, NAN, NAN
*LOADCASE, TYPE=EIGENFREQ
*SUPPORTS
BC
*NUMEIGENVALUES
5
*END
```

The first mode shapes identify whether the corresponding numerical frequencies belong to structural deformation or to an unintended mechanism. A plausible frequency attached to an unintended mechanism is not a valid modal result.

5.7 Eigenfrequency with a concentrated MASS element

This benchmark isolates the new one-node concentrated-mass path. Two identical axial rods connect a central mass node to fixed supports on both sides. The rods carry no material density, so the only inertia is the `MASS` element at the centre. The centre node is guided so only U_x is free.

The example uses N, mm, and s. In this consistent unit system the mass unit is tonne, because

$$1\ \mathrm{N\,s^2/mm} = 1\ \mathrm{t}.$$

Thus $m = 0.01$ represents 10 kg.

Each rod has

$$k = \frac{EA}{L} = \frac{210000 \cdot 1}{1000} = 210\ \mathrm{N/mm}.$$

A positive displacement of the central node extends one rod and shortens the other, so both provide restoring stiffness and

$$k_{\mathrm{eq}} = 2k = 420\ \mathrm{N/mm}.$$

The exact circular eigenfrequency and frequency are therefore

$$\omega_1 = \sqrt{\frac{k_{\mathrm{eq}}}{m}} = \sqrt{\frac{420}{0.01}} \approx 204.939\ \mathrm{s^{-1}},$$

$$f_1 = \frac{\omega_1}{2\pi} \approx 32.617\ \mathrm{Hz}.$$

Example

```
*NODE
1, -1000., 0., 0.
2,     0., 0., 0.
3,  1000., 0., 0.

*ELEMENT, TYPE=T3D2, ELSET=BARS
1, 1, 2
2, 2, 3
*ELEMENT, TYPE=MASS, ELSET=POINT_MASS
100, 2

*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3
*SOLID SECTION, ELSET=BARS,
 MATERIAL=STEEL
1.0
*MASS, ELSET=POINT_MASS
0.01

*STEP, PERTURBATION
*FREQUENCY
1
*BOUNDARY
1, 1, 3, 0.
2, 2, 3, 0.
3, 1, 3, 0.
*END STEP
```

FEMaster truss and eigenfrequency loadcase

The following cards replace the corresponding definitions in the complete example above; unchanged geometry and material definitions are retained.

```
*ELEMENT, TYPE=T3, ELSET=BARS
1, 1, 2
2, 2, 3
*TRUSS SECTION, ELSET=BARS, MATERIAL=STEEL
1.0
*SUPPORT, SUPPORT_COLLECTOR=BC
1, 0., 0., 0., NAN, NAN, NAN
2, NAN, 0., 0., NAN, NAN, NAN
3, 0., 0., 0., NAN, NAN, NAN
*LOADCASE, TYPE=EIGENFREQ
*SUPPORTS
BC
*NUMEIGENVALUES
1
*END
```

NSET-based concentrated mass instead of MASS element

```
*NSET, NSET=CENTRE_NODE
2
*POINTMASS, NSET=CENTRE_NODE
0.01
```

This alternative replaces the regular `MASS` element and `*MASS` property assignment; the truss elements remain unchanged.

This model is useful as a direct regression check for `*ELEMENT, TYPE=MASS` and `*MASS`: the stiffness comes entirely from the two trusses, the inertia comes entirely from the point element, and the expected finite eigenfrequency is approximately 32.617 Hz.

5.8 Linear buckling of a compressed member

A linear buckling analysis uses a reference preload to construct geometric stiffness and then extracts load multipliers. The example below uses a slender truss only to demonstrate the input structure; a flexural column requires a beam, shell, or solid model with transverse/bending degrees of freedom.

The buckling factors satisfy a generalized eigenproblem involving the elastic stiffness and preload-dependent geometric stiffness. They multiply the reference load rather than representing absolute force directly.

Example

```
*NODE
0, 0., 0., 0.
1, 1000., 0., 0.
*ELEMENT, TYPE=T3D2, ELSET=MEMBER
0, 0, 1
*NSET, NSET=LEFT
0
*NSET, NSET=RIGHT
1
*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3
*SOLID SECTION, ELSET=MEMBER, MATERIAL=STEEL
100.

*STEP, PERTURBATION
*BUCKLE
5
*BOUNDARY
LEFT, 1, 3, 0.
RIGHT, 2, 3, 0.
*CLOAD
RIGHT, 1, -1000.
*END STEP
```

FEMaster buckling loadcase

The following cards replace the corresponding definitions in the complete example above; unchanged geometry and material definitions are retained.

```
*ELEMENT, TYPE=T3, ELSET=MEMBER
0, 0, 1
*TRUSS SECTION, ELSET=MEMBER, MATERIAL=STEEL
100.
*SUPPORT, SUPPORT_COLLECTOR=BC
LEFT, 0., 0., 0., NAN, NAN, NAN
RIGHT, NAN, 0., 0., NAN, NAN, NAN
*CLOAD, LOAD_COLLECTOR=PRELOAD
RIGHT, -1000., 0., 0., 0., 0., 0.
*LOADCASE, TYPE=LINEARBUCKLING
*SUPPORTS
BC
*LOADS
PRELOAD
*NUMEIGENVALUES
5
*SIGMA
0.0
*END
```

The model above is deliberately a syntax example rather than a flexural-buckling benchmark: an axial truss does not possess beam bending stiffness. A column model based on flexural buckling requires `B33`, shell, or solid elements and boundary conditions that admit the relevant buckling mode.

5.9 Arc-length snap-through

A shallow two-bar arch illustrates geometrically nonlinear path following. The main deck selects `*STATIC, RIKS`; the alternative uses `CONTROL=ARC_LENGTH` in a FEMaster loadcase. The centre node is constrained to remain in the $x = 0$ plane and to have zero out-of-plane motion, while vertical displacement remains free.

Arc-length control permits the equilibrium path to pass a limit point at which load factor no longer increases monotonically with displacement.

Example

```
*NODE
0, -1000.,   0., 0.
1,     0., 100., 0.
2,  1000.,   0., 0.
*ELEMENT, TYPE=T3D2, ELSET=BARS
0, 0, 1
1, 1, 2
*NSET, NSET=ENDS
0, 2
*NSET, NSET=APEX
1
*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3
*SOLID SECTION, ELSET=BARS, MATERIAL=STEEL
100.

*STEP, NLGEOM=YES, INC=200
*STATIC, RIKS
0.05, 1.0, 1e-5, 0.1
*BOUNDARY
ENDS, 1, 3, 0.
APEX, 1, 1, 0.
APEX, 3, 3, 0.
*CLOAD
APEX, 2, -20000.
*END STEP
```

FEMaster arc-length continuation

The following cards replace the corresponding definitions in the complete example above; unchanged geometry and material definitions are retained.

```
*ELEMENT, TYPE=T3, ELSET=BARS
0, 0, 1
1, 1, 2
*TRUSS SECTION, ELSET=BARS, MATERIAL=STEEL
100.
*SUPPORT, SUPPORT_COLLECTOR=BC
ENDS, 0., 0., 0., NAN, NAN, NAN
APEX, 0., NAN, 0., NAN, NAN, NAN
*CLOAD, LOAD_COLLECTOR=P
APEX, 0., -20000., 0., 0., 0., 0.
*LOADCASE, TYPE=NONLINEARSTATIC
*SUPPORTS
BC
*LOADS
P
*NONLINEAR, CONTROL=ARC_LENGTH, MAX_INCREMENTS=200,
 INITIAL_INCREMENT=0.05, MINIMUM_INCREMENT=1e-5,
 MAXIMUM_INCREMENT=0.1, MAXITER=40, TOL=1e-8
*END
```

For nonlinear verification, plot load factor against apex displacement. A converged sequence of increments is much more informative than the final state alone because the characteristic feature of snap-through is the shape of the equilibrium path.

5.10 Linear transient ramp load

The transient model applies an axial load with a tabular amplitude. The main deck uses `*DYNAMIC, DIRECT` to specify the time increment and step period. The FEMaster alternative uses `*TIME`, `*NEWMARK`, and `*WRITEEVERY`. The default Newmark parameters are $\beta = 0.25$ and $\gamma = 0.5$.

Example

```
*NODE
0,    0., 0., 0.
1, 1000., 0., 0.
*ELEMENT, TYPE=T3D2, ELSET=BAR
0, 0, 1
*NSET, NSET=FIXED
0
*NSET, NSET=TIP
1
*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3
*DENSITY
7.85e-9
*SOLID SECTION, ELSET=BAR, MATERIAL=STEEL
100.
*AMPLITUDE, NAME=RAMP
0., 0., 0.1, 1., 1., 1.

*STEP
*DYNAMIC, DIRECT
0.01, 1.0
*BOUNDARY
FIXED, 1, 3, 0.
TIP, 2, 3, 0.
*CLOAD, AMPLITUDE=RAMP
TIP, 1, 1000.
*NODE FILE
U
*END STEP
```

FEMaster Newmark time integration and output cadence

The following cards replace the corresponding definitions in the complete example above; unchanged geometry and material definitions are retained.

```
*SUPPORT, SUPPORT_COLLECTOR=BC
FIXED, 0., 0., 0., NAN, NAN, NAN
TIP, NAN, 0., 0., NAN, NAN, NAN
*CLOAD, LOAD_COLLECTOR=P, AMPLITUDE=RAMP
TIP, 1000., 0., 0., 0., 0., 0.
*LOADCASE, TYPE=LINEARTRANSIENT
*SUPPORTS
BC
*LOADS
P
*TIME
0., 1., 0.01
*NEWMARK
0.25, 0.5
*WRITEEVERY, TYPE=STEPS
10
*END
```

The `*NODE FILE` card actively filters FEMaster frame output for this step. Because the example requests only `U`, FEMaster writes displacement frames but omits the transient default velocity and acceleration Fields. The native `*WRITEEVERY` setting controls only the write cadence; it does not change which Fields are selected.

5.11 Linear harmonic frequency sweep

The harmonic example evaluates a real axial harmonic force from 10 to 1000 frequency units at 100 linearly spaced points. The dynamic stiffness is

$$K_{\mathrm{dyn}}(\omega) = K - \omega^2 M + i\omega C.$$

The alternative FEMaster loadcase adds Rayleigh damping through a small stiffness-proportional coefficient. The main Abaqus-compatible example does not specify equivalent damping; consequently, their frequency responses are not numerically identical unless the damping settings are made consistent.

Example

```
*NODE
0, 0., 0., 0.
1, 1000., 0., 0.
*ELEMENT, TYPE=T3D2, ELSET=BAR
0, 0, 1
*NSET, NSET=FIXED
0
*NSET, NSET=TIP
1
*MATERIAL, NAME=STEEL
*ELASTIC
210000., 0.3
*DENSITY
7.85e-9
*SOLID SECTION, ELSET=BAR, MATERIAL=STEEL
100.

*STEP
*STEADY STATE DYNAMICS, DIRECT,
 FREQUENCYSCALE=LINEAR
10., 1000., 100, 1., 1.
*BOUNDARY
FIXED, 1, 3, 0.
TIP, 2, 3, 0.
*CLOAD, REAL
TIP, 1, 1.
*END STEP
```

FEMaster harmonic analysis with Rayleigh damping

The following cards replace the corresponding definitions in the complete example above; unchanged geometry and material definitions are retained.

```
*SUPPORT, SUPPORT_COLLECTOR=BC
FIXED, 0., 0., 0., NAN, NAN, NAN
TIP, NAN, 0., 0., NAN, NAN, NAN
*CLOAD, LOAD_COLLECTOR=HARMONIC
TIP, 1., 0., 0., 0., 0., 0.
*LOADCASE, TYPE=LINEARHARMONIC
*SUPPORTS
BC
*LOADS
HARMONIC
*FREQUENCIES, SCALE=LINEAR
10., 1000., 100
*DAMPING, TYPE=RAYLEIGH
0., 1e-5
*CONSTRAINTMETHOD, TYPE=NULLSPACE
*END
```

For a numerical comparison, remove `*DAMPING` from the FEMaster alternative or otherwise ensure that both analyses use the same damping law. Harmonic response can change dramatically near resonance even for small damping differences.

5.12 Using the examples as templates

The examples follow the dependency order of the model: geometry and element connectivity precede regions, materials, and Sections; loads and constraints depend on those definitions; numerical solver controls act on the resulting analysis system.

The unified input language accepts the documented Abaqus-compatible subsets together with FEMaster extensions. A keyword or option available in the full Abaqus product is not automatically supported in FEMaster. When adapting an example, retain the definitions not replaced by an alternative block and consult the corresponding keyword page for scope, parameter, and data-line restrictions.

A Mathematical Derivations

This appendix derives the topology-related quantities written by FEMaster. The keyword reference sections describe the DSL commands, while this appendix gives the mathematical background for compliance, density sensitivities, and orientation sensitivities.

A.1 Element stiffness

For a linear elastic finite element, the unmodified element stiffness is

$$K_{e,0} = \int_{\Omega_e} B^T D B \, d\Omega.$$

B is the strain-displacement matrix, D is the constitutive matrix, and Ω_e is the physical element domain. With an isoparametric mapping from natural coordinates $r = (\xi, \eta, \zeta)$, the integral is evaluated as

$$K_{e,0} = \int_{\Omega'_e} B(r)^T D B(r) \, |\det J(r)| \, d\xi \, d\eta \, d\zeta.$$

For numerical integration with quadrature points r_q and weights w_q, this becomes

$$K_{e,0} \approx \sum_{q=1}^{Q} w_q B(r_q)^T D B(r_q) \, |\det J(r_q)|.$$

A.2 Compliance

For a linear static analysis,

$$Ku = f.$$

The external work measure commonly used for topology optimization is the compliance

$$C = f^T u.$$

Using the equilibrium equation $f = Ku$, this can also be written as

$$C = u^T K u.$$

Because the global stiffness matrix is assembled from element stiffness matrices, compliance can be decomposed into element contributions:

$$C = \sum_{e=1}^{n_e} u_e^T K_e u_e.$$

This expression is the basis for element-wise topology sensitivities. u_e is the element displacement vector extracted from the global solution.

A.3 Density penalization

In a density-based topology formulation, an element density variable ρ_e scales the stiffness contribution. With a penalization exponent p,

$$K_e(\rho_e) = \rho_e^p K_{e,0}.$$

The exponent p makes intermediate densities less efficient and therefore encourages designs closer to a solid-void distribution.

The element compliance contribution is

$$C_e(\rho_e) = u_e^T K_e(\rho_e) u_e = \rho_e^p u_e^T K_{e,0} u_e.$$

If u_e is held fixed for the local sensitivity expression, the derivative is

$$\frac{\partial C_e}{\partial \rho_e} = p\rho_e^{p-1} u_e^T K_{e,0} u_e.$$

Depending on the optimization convention, the objective derivative may be reported with the opposite sign because minimizing compliance through the equilibrium equation often uses the adjoint result

$$\frac{dC}{d\rho_e} = -u_e^T \frac{\partial K_e}{\partial \rho_e} u_e = -p\rho_e^{p-1} u_e^T K_{e,0} u_e.$$

The sign convention is therefore defined together with the optimizer interface and the Field name written by the analysis.

A.4 Orientation-dependent stiffness

For orientation-based topology or material steering, the constitutive matrix is rotated before the element stiffness is evaluated. Let the material orientation be represented by three angles

$$\alpha = (\alpha_1, \alpha_2, \alpha_3).$$

The elementary rotation matrices are

$$R_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\alpha_1 & -\sin\alpha_1 \\ 0 & \sin\alpha_1 & \cos\alpha_1 \end{bmatrix},$$

$$R_2 = \begin{bmatrix} \cos\alpha_2 & 0 & \sin\alpha_2 \\ 0 & 1 & 0 \\ -\sin\alpha_2 & 0 & \cos\alpha_2 \end{bmatrix}, \qquad R_3 = \begin{bmatrix} \cos\alpha_3 & -\sin\alpha_3 & 0 \\ \sin\alpha_3 & \cos\alpha_3 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

For the convention used here, the combined rotation is

$$R = R_1 R_2 R_3.$$

The multiplication order must be used consistently. Different conventions can describe the same physical orientation with different angle triples.

A.5 Voigt transformation

The strain vector uses engineering shear strains,

$$\varepsilon = \begin{bmatrix} \varepsilon_{xx} & \varepsilon_{yy} & \varepsilon_{zz} & 2\varepsilon_{yz} & 2\varepsilon_{xz} & 2\varepsilon_{xy} \end{bmatrix}^T,$$

and the stress vector is

$$\sigma = \begin{bmatrix} \sigma_{xx} & \sigma_{yy} & \sigma_{zz} & \sigma_{yz} & \sigma_{xz} & \sigma_{xy} \end{bmatrix}^T.$$

The strain transformation is written as

$$\varepsilon_m = T(R)\varepsilon_g,$$

where T is the 6×6 Voigt transformation matrix built from the entries of R. The rotated constitutive matrix used in global strain coordinates is

$$D_{\mathrm{rot}} = T^T D T.$$

The orientation-dependent element stiffness is therefore

$$K_e(\rho_e, \alpha_e) = \rho_e^p \int_{\Omega_e} B^T D_{\mathrm{rot}}(\alpha_e) B \, d\Omega.$$

With quadrature,

$$K_e(\rho_e, \alpha_e) \approx \rho_e^p \sum_{q=1}^{Q} w_q B(r_q)^T T(\alpha_e)^T D T(\alpha_e) B(r_q) |\det J(r_q)|.$$

A.6 Orientation sensitivity

For one element, the compliance contribution is

$$C_e = \rho_e^p \sum_{q=1}^{Q} w_q \varepsilon_q^T T^T D T \varepsilon_q |\det J_q|, \qquad \varepsilon_q = B(r_q) u_e.$$

Only T depends on the orientation angles, so the derivative with respect to angle α_j is

$$\frac{\partial C_e}{\partial \alpha_j} = \rho_e^p \sum_{q=1}^{Q} w_q \varepsilon_q^T \left(\frac{\partial T^T}{\partial \alpha_j} D T + T^T D \frac{\partial T}{\partial \alpha_j} \right) \varepsilon_q |\det J_q|.$$

If D is symmetric, the scalar expression can also be evaluated as

$$\frac{\partial C_e}{\partial \alpha_j} = 2 \rho_e^p \sum_{q=1}^{Q} w_q \varepsilon_q^T T^T D \frac{\partial T}{\partial \alpha_j} \varepsilon_q |\det J_q|,$$

provided the same transformation convention is used.

A.7 Rotation derivatives

The derivatives of the elementary rotation matrices are

$$\frac{\partial R_1}{\partial \alpha_1} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & -\sin\alpha_1 & -\cos\alpha_1 \\ 0 & \cos\alpha_1 & -\sin\alpha_1 \end{bmatrix},$$

$$\frac{\partial R_2}{\partial \alpha_2} = \begin{bmatrix} -\sin\alpha_2 & 0 & \cos\alpha_2 \\ 0 & 0 & 0 \\ -\cos\alpha_2 & 0 & -\sin\alpha_2 \end{bmatrix},$$

$$\frac{\partial R_3}{\partial \alpha_3} = \begin{bmatrix} -\sin\alpha_3 & -\cos\alpha_3 & 0 \\ \cos\alpha_3 & -\sin\alpha_3 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

For $R = R_1 R_2 R_3$, the total rotation derivatives are

$$\frac{\partial R}{\partial \alpha_1} = \frac{\partial R_1}{\partial \alpha_1} R_2 R_3,$$

$$\frac{\partial R}{\partial \alpha_2} = R_1 \frac{\partial R_2}{\partial \alpha_2} R_3,$$

$$\frac{\partial R}{\partial \alpha_3} = R_1 R_2 \frac{\partial R_3}{\partial \alpha_3}.$$

The derivative $\partial T / \partial \alpha_j$ is obtained by differentiating every entry of $T(R)$ with the product rule. For example, an entry $R_{ab} R_{cd}$ contributes

$$\frac{\partial}{\partial \alpha_j} \left(R_{ab} R_{cd} \right) = \frac{\partial R_{ab}}{\partial \alpha_j} R_{cd} + R_{ab} \frac{\partial R_{cd}}{\partial \alpha_j}.$$

This product-rule structure is the key point of the orientation derivative: once R, $\partial R / \partial \alpha_j$, T, and $\partial T / \partial \alpha_j$ are known, the sensitivity follows directly from the quadrature expression above.